

TEAM IQ

*why your best people
don't make your best teams*

Andrew Phillips

Network Advansys Limited, trading as Systemhelp

Team IQ: why your best people don't make your best teams

Copyright © 2026 Network Advansys Limited. All rights reserved.

First edition, 2026. Published by Network Advansys Limited, 49 Station Road, Polegate, East Sussex, BN26 6EA, United Kingdom. Company number 3503850. TEAM IQ Creator is a registered trademark of Network Advansys Limited, and is used in this book to refer to the method and the accompanying instruments.

No part of this publication may be reproduced, distributed or transmitted in any form without the prior written permission of the publisher, except for brief quotations in a review or in internal business documents, which are permitted with attribution.

ISBN (paperback): to be assigned

ISBN (ebook): to be assigned

Edition: 1.0

The exercises and instruments described in this book are published so that teams can use them without buying anything. You do not need to work with us to use them.

Every statistic in this book is either drawn from published research, named in the text and listed in the references, or identified as our own observation. Where a figure is quoted, it is quoted as published. Nothing here is a guarantee of a commercial result, and nothing here is professional advice for your particular circumstances.

Written in the United Kingdom in British English. Set in EB Garamond and Inter.

systemhelp.co.uk

Contents

- Why the focus is on teams 1
- Part One — Why capable teams underperform 8**
 - 1. The finding nobody acts on 9
- Part Two — The value question 19**
 - 2. Output is not value 20
- Part Three — The measurement cycle 30**
 - 3. The measurement cycle 31
- Part Four — Target and Expression 41**
 - 4. True and explainable 42
 - 5. Achievable and required 51
 - 6. Give 60
 - 7. Traceable 70
 - 8. Create safety and lead by example 79
 - 9. Expectations: the clarity code 88
 - 10. Listening, and the habit of team building 97
 - 11. Conflict and feedback 106
- Part Five — Accountability and Mindset 115**
 - 12. Roles, and modelling accountability 116
 - 13. Check-ins, empowerment and agreements 126
 - 14. Being, not doing 136
 - 15. Distributed ownership 145
 - 16. The whole is more than the parts 154

Part Six — Insight and Quality	163
17. Deciding where the information is	
.....	164
18. Interconnected metrics	
.....	174
19. Clear channels	
.....	184
20. Value stream mapping without the ceremony	
.....	194
21. Customer-centric design	
.....	203
22. Quality, cost and the cross-functional team	
.....	213
The fourteen instruments	223
References	229
About the author	233
Working with TEAM IQ Creator	235

How to use this book

This book has three jobs. The first is to convince you that the performance of a team is a measurable property of the team, not a sum of the people in it. The second is to give you six things to work on. The third is to give you a way to show the movement to whoever holds your budget.

Six parts follow. The first three set up the argument, the value question and the measurement cycle. Parts four, five and six are the six pillars in practice, and they are the working part of the book: each chapter ends with an exercise you can run with your team, something to measure, and one thing to do this week.

You can read it straight through in an evening or two. You will get more from it if you run the exercises as you go, on one real team, rather than making notes for later. Every exercise is designed for a single session of forty-five to sixty minutes, with the team in the room.

Nothing here requires you to buy a framework, a licence or a subscription. The instruments are published in the book and on the website, and they are yours to use.

A note on the evidence

Business writing has a credibility problem, and most of it comes from numbers that nobody can trace. Three rules were applied to every page of this book.

First, every published figure is named where it appears and listed in the references at the back. If you want to check a number, you can. Second, where a claim comes from our own work rather than from

published research, the text says so in the sentence, and the numbers come from programmes we can name. Third, where we could not source something, it is described rather than quantified.

That third rule cost us some good lines. A widely quoted figure about rework reduction appeared on our previous website without a source; we removed it rather than repeat it. You will find that attitude throughout: occasionally irritating, consistently honest, and the reason the arguments in this book survive being taken to somebody sceptical.

Why the focus is on teams

The transformation is complete. The framework is in place, the ceremonies are running, the tooling has been bought and the training has been delivered. Three years and a great deal of money later, delivery looks much as it did before: capable people, sensible plans, and a programme that still finishes late and costs more than the case predicted.

Ask why, and the answers are familiar. Priorities moved. A supplier slipped. The dependencies were harder than they looked. None of that is wrong, and none of it explains the thing leaders actually notice: that the same organisation can run one programme brilliantly and another badly, with much the same people and much the same process.

The difference is not in the process. It is in the team. This book is about that difference, why it is about to matter more than it has, and how to measure it and improve it.

Three pressures that are not going away

Speed. The interval between a decision and its consequence keeps shrinking. A competitor ships in weeks what used to take a year. A customer commitment made in a slide deck becomes a fact that the delivery team has to live with. When the market moves faster than the plan, the plan stops being the constraint and the team's ability to decide well, quickly, becomes the constraint.

Complexity. Very little of the work now sits inside one team. A release crosses product, engineering, architecture, data, operations, security, legal, a supplier and a regulator. Each holds part of the truth. Nobody holds all of it. Complexity of that kind does not respond

to effort; it responds to whether the people holding the parts can combine them into a decision.

Competition. Standing out now depends on making something better or different, not on doing the same thing more cheaply. That requires the people closest to the work to generate and test ideas — which is a property of the team, not of the individuals in it.

Add one more pressure, which is the one most organisations are actually feeling: being asked to do more, with less, faster.

The map is not the terrain

Process work has given what it can. That is not a criticism; it is an observation about what a process is. A process is a description of how work normally flows, captured at a point in time. It is a map. A map is useful, and a map is not the terrain. Walk a route from a map and you will meet the closed road, the washed-out bridge and the path that was never there.

Watch how fast the maps themselves move. The Scaled Agile Framework has been through six major versions since 2011, with many smaller releases in between. Every version is an improvement, and every version also means that whatever was rolled out, trained and embedded across a large organisation is now a version behind. Rollout at that scale commonly takes a year or more. You can finish the training just as the map changes.

This is the trap: treating the framework as the answer, and the team as the thing that implements it. When the framework meets a situation it did not anticipate — which is most days — the framework cannot adapt. The team can. The German field marshal Helmuth von Moltke the Elder put it in a way that has survived: no plan of operations

extends with certainty beyond the first encounter with the enemy's main strength. Mike Tyson later reduced it to a sentence everyone remembers.

Four kinds of problem, one habit of thinking

It helps to know what kind of problem you are in, because different kinds need different thinking.

- *Simple*. Manufacturing, largely. There is a best practice, and following it works. Instruction is appropriate.
- *Complicated*. Most software delivery. There are good practices and no single best one; expertise and analysis decide the route. Instruction alone stops working.
- *Complex*. The situation where nobody has a proven answer and the answer emerges from the people in it. Here you need everyone's context, not the most senior opinion.

- *Chaotic*. Something has broken and the priority is to stabilise.

The habit that causes damage is treating every problem as simple: find the best practice, mandate it, and hold people to the instruction. That works in manufacturing and fails in software, because software is always complex. A team that has only one way of thinking, whatever the situation, will be wrong in three of these four cases.

The alternative is not a better process. It is a team that can read the situation, change how it is thinking, and find a route that nobody has written down yet. Teams that can work without a map, and draw a new one.

What the research actually found

Two bodies of work sit behind this book. The first is from MIT: studies of *699 people working in groups of two to five*, published in *Science* in 2010. They found a general collective-intelligence factor — a property of the group, not of the individuals in it — that predicts how the group performs across a wide variety of tasks. It is *not strongly correlated with the average or maximum intelligence of the members*. It is correlated with members' social sensitivity, with how evenly conversational time is shared, and with the proportion of women in the group. The finding is uncomfortable for anyone whose answer to a struggling team is to add a cleverer person.

The second is Google's own study, begun in 2011 and running for about four years. It looked for what distinguishes effective teams and landed on five dynamics: psychological safety, dependability, structure and clarity, meaning, and impact. The strongest of the five was psychological safety — whether people can say what they actually think without paying a price for it.

Neither study gives you a method. MIT establishes that the property exists and can be measured. Google shows which conditions make it visible. What to do about it, week by week, is the gap this book fills — and the six-pillar model in it is ours, not theirs. The research is the reason to take the subject seriously. The model is the reason you can act on it.

Why artificial intelligence makes this more urgent

Artificial intelligence is the most useful tool to arrive in delivery work for a generation. It is also, in the wrong place, a multiplier of the wrong thing.

An AI tool applied to a broken process does not repair the process. It produces the wrong output faster, in more volume, with more confidence, and it makes the mistake harder to spot because it arrives well presented. Applied to a team that cannot say what it actually knows, it accelerates the thing the team already does: generating material nobody has checked and decisions nobody agreed.

The tool is not the variable. The team is. A team with high collective intelligence will ask the model better questions, judge its output against evidence, notice what looks plausible and is not, and integrate it into a decision. A team without it will generate more, faster, and find out later. The same is true of every previous accelerator: cloud, automation, offshoring, a new framework. Each one increased the consequence of how well the team thought together.

What this book is

This book is not against process. It is against the assumption that process is the lever.

It sets out six pillars — Target, Expression, Accountability, Mindset, Insight and Quality — a five-stage measurement cycle, and fourteen instruments that turn each pillar into something a team can inspect and change. Every chapter ends with an exercise you can run in one session, something to measure, and one thing to do this week. None of it requires a licence, a subscription or a purchase.

The claim is modest and testable: collective intelligence is a property of a team, it can be measured, and the conditions that produce it can be improved deliberately. If that is true, then the fastest perfor-

mance gain available to most organisations is not a new framework. It is the team they already have.

Before you start

One thing, which takes ten minutes and no meeting.

Think of a decision your team got wrong in the last quarter — not a disaster, just a decision that had to be revisited, or that produced work nobody wanted. Write down two things: what information existed somewhere in the team at the time, and who held it. Then ask whether the decision could have been made correctly if everyone had pooled what they already knew.

Almost always, it could. That is the subject of this book.

Part One — Why capable teams underperform

1. The finding nobody acts on

At the start of the delivery meeting, the room looks strong. The product lead knows the customer case. The architect can explain the technical constraints. The delivery manager has the plan, risks and dependencies. The operations lead knows where the handover will hurt. Each person has a sensible update and has done the work expected of them.

Then the discussion moves to a decision. A dependency has changed, a supplier date has slipped, and the release still has a public commitment. The product lead protects scope. The architect protects the design. Operations asks for a longer proving period. The delivery manager asks for a revised date. Nobody is wrong. Nobody can show how the choices fit together.

The meeting ends with actions, each assigned to a capable person. The status report will be refreshed. A decision will be taken elsewhere. By the next meeting, the programme has moved, but not in a direction that anyone chose together. This is not a failure of effort or talent. It is good people producing a result that falls short of their combined judgement.

The principle

Collective intelligence is a measurable property of a team: its ability to perform across work that requires people to share information, reason together, decide and adapt. It is not a compliment for a friendly team, nor a claim that every member thinks alike. It concerns what the group can reliably produce when the work needs more than one perspective.

The MIT collective intelligence studies found a group-level capability not accounted for simply by the intelligence of individual members. Google's Project Aristotle reached a related managerial conclusion: what happens between team members matters to effectiveness; selecting impressive individuals is not a sufficient design for a dependable team.

That finding is routinely acknowledged, then ignored in the machinery of an organisation. Recruitment inspects qualifications, experience and individual performance. Appraisal rewards the person who rescues difficult work. Promotion panels look for personal authority, subject knowledge and visible results. These are reasonable things to inspect. They are easier to inspect than the quality of a team conversation, the clarity of a handover, or the way a group changes course when its evidence changes.

The mistake comes in the inference. A leader sees capable people and assumes the team will make capable decisions. A programme is staffed with specialists and assumed to be safe. Work is divided, names are placed beside tasks, and the result is treated as the sum of the people. It does not follow. A group can contain expertise and still fail to bring it into the decision. It can be busy and leave a consequential assumption untested. It can have an owner for every task and nobody accountable for the outcome across those tasks.

The cost is disguised as ordinary delivery friction: late discovery, rework, slow escalation, decisions revisited after work starts, and senior people drawn back into matters the team ought to resolve. A weekly status report may show activity. It cannot tell you whether

people share a target, have heard the same evidence, or will act together when a constraint shifts.

The six pillars in this book are *Target, Expression, Accountability, Mindset, Insight* and *Quality*. They make a team's intended result, communication, ownership, learning, information and standard of value visible. The rest of the book takes each pillar in turn, turns it into habits and artefacts, and uses the TEAM IQ Measurement Cycle to choose where to improve next.

The six-pillar model and its practical measures are ours, not a published result from MIT or Google. Our claim is modest: inspect and improve the conditions through which a team sets a target, exchanges evidence, owns outcomes, learns, makes decisions and judges quality. You then have a disciplined way to raise collective capability and see whether it is happening.

What it looks like when it is missing

Return to the delivery meeting. The product lead has a customer commitment in a slide deck, but the team has not seen the evidence behind it. The architect has logged a design concern in a document read only by the technical group. Operations has a service risk in a separate register. The delivery manager has a plan whose dates assume all three views can be satisfied.

When the supplier date changes, people speak from their own artefact. The discussion becomes a negotiation between local commitments. The architect protects a design decision because its effect on the customer target is unclear. The product lead protects scope

because the service risk is not visible in prioritisation. Operations is asked for an answer after the decision has hardened.

The actions are individually tidy. Someone will ask the supplier for a revised date. Someone will estimate an alternative. Someone will update the plan. What is absent is a shared outcome, the evidence that should alter the decision, the person who can make the call, and the point at which the team will review the consequence. The group has distributed work without distributing understanding.

This can look orderly from outside. Meetings occur, work is assigned and papers are produced. It becomes expensive because uncertainty travels quietly between roles until it arrives as a late defect, a blocked release or a request for an executive decision.

What it looks like when it works

In the same meeting, the changed supplier date is placed beside a visible target that names the customer or business change the release must create. The team has agreed the evidence that would justify a change in scope, date or design. The product lead explains the value at risk. The architect explains the constraint in terms the whole room can use. Operations identifies the condition required before the service can change.

The discussion is still difficult. There may be a trade-off between date, scope and assurance. The team is working on one decision rather than defending private versions of it. A concern is information that may alter the route to the shared outcome, not a challenge to someone's authority.

The meeting ends with a decision record: what has been decided, the evidence used, the open assumption, the owner of the next

test and when the group will read the result. The delivery manager can change the plan because the reason is clear. The product lead can explain it to a sponsor without hiding uncertainty. Operations is involved before the handover.

This is not consensus theatre. A team can disagree sharply and still be collectively intelligent if it makes its reasoning visible, uses relevant expertise and commits to clear conditions for revisiting a course. The aim is a team that can think and act together when the original plan no longer fits the work.

How to do it

Do not begin by rating personalities. Begin with live work that exposed a gap between individual effort and team performance: a delayed decision, failed handover, late issue or differently interpreted priority. Inspect the way the team worked, not a person to blame.

1. *Choose one outcome, not a general complaint.* Put a live delivery outcome in the centre. State the customer, business, risk or service change required; the date or decision point; and the consequence if it is not achieved. A label such as poor communication gives the group nothing observable to inspect.
2. *Collect the existing evidence before the meeting.* Bring the plan, decision record, risk entry, customer information, service data and work board. Do not produce a new pack to make the team look aligned. Gaps between the existing artefacts are evidence. If a key statement exists only in one person's head, record it as a finding.
3. *Ask for individual accounts before group discussion.* Give each person time to write, without consulting colleagues: the outcome they believe the team is pursuing; evidence they trust; the main

uncertainty; the decision needed; and who owns the next move. This exposes differences before the most senior or confident voice sets the answer.

4. *Compare the accounts against the work.* Put answers beside the artefacts. Look for differences that would change a decision or fail a handover. Is the target the same in each account? Is a risk known to people who must alter the plan? Does the named owner have authority, information and support? Does a technical or operational condition appear only after a business commitment? Record the difference as a condition in the system, not a flaw in a person.
5. *Make the next decision explicit.* For the most material difference, agree the decision, evidence still needed, person obtaining it, contributors and review date. Write the assumption plainly. A decision with no stated assumption cannot be intelligently revised; it can only be defended or abandoned.
6. *Use the TEAM IQ Measurement Cycle to keep evidence connected.* The cycle is not a reporting layer added after delivery. It makes intentions, evidence, ownership, learning, decisions and outcomes visible in the work. At the end of an interval, read outcome, evidence, decision and learning together. Then decide whether the constraint concerns Target, Expression, Accountability, Mindset, Insight or Quality. Work on that before adding another ceremony.
7. *Read patterns, not isolated incidents.* One poor meeting may be a timing problem. A repeated difference between the plan's target, a risk register's evidence and a decision's assumption is a team

pattern. Look across live decisions before changing a role, meeting or process. Ask whether the team can make the connection without a senior person stitching the pieces together.

8. *Keep the intervention small and observable.* Agree one change to test in the next interval: a shared target in a decision record, an operations review before commitment, or a named evidence owner. Decide what you will look for. If it does not improve the quality or timing of the decision, keep the learning and change the practice.

This procedure is our applied method, built around the course's Measurement Cycle. It is not a clinical assessment and does not turn a team into a score. It exposes whether the team can connect the information and ownership required for the work. That is a better starting point than a general request to collaborate more.

The exercise

Run a shared-outcome review with the intact team around one live decision or delivery outcome. This is a practical book exercise, not a validated research instrument. Prepare a wall or shared board with space for target, evidence, uncertainty, decision, ownership, learning and outcome. Bring the plan, risk material, decision record and customer or operational evidence. Ask only people who affect the decision to attend.

Allow five minutes to state the outcome and decision point. Give each participant ten minutes to write their account of the target, evidence, uncertainty and expected owner. Keep this silent. Find differences before discussion smooths them away.

Spend fifteen minutes placing accounts and artefacts on the board. Group similar statements, but do not force agreement. Mark differences that could change the route, timing, scope, risk or standard of the outcome. Use fifteen minutes to select the most material difference. Agree the evidence to settle or narrow it, who will obtain it, contributors, and when the team will return to the decision.

Use the final ten minutes to capture a one-page review record: shared outcome, unresolved assumption, next evidence action, named ownership, review point and the likely constraining pillar. Read it aloud. If anyone cannot explain the outcome or their next move, correct it before leaving.

Save a photograph or export of the board with the record for the next review.

The session takes fifty-five minutes. Its output is not a verdict on team quality. It shows where understanding breaks and tests whether a clearer connection improves the next decision.

What to measure

The first artefact is a one-page review record for a live outcome, used in the TEAM IQ Measurement Cycle. It links the intended result with available evidence, open uncertainty, decision, owner, learning and observed outcome. Keep the original record. The comparison between what the team assumed and what happened makes learning visible.

Read it at the next delivery review and whenever the decision changes. Look for recurring breaks: differently explained targets, evidence after commitment, ownership without authority, or learning

that never changes a decision. These signal a pillar for attention, not individual worth.

This is a TEAM IQ design choice, not a published universal score. At this stage, measure whether the team can show its reasoning, test it and alter work when evidence requires it.

One thing to do this week

Take a delayed or revisited decision. Ask each person involved to write the outcome, evidence they trust and uncertainty that matters. Compare the answers before anyone explains them. Do not repair every difference. Choose the one most likely to alter the decision, name the evidence needed and set the next review point. You will have a clearer diagnosis than a status update.

Part Two — The value question

2. Output is not value

The monthly delivery review starts well. The team has a full status pack, a release calendar, a burn-up chart and a page of completed work. The delivery lead has highlighted the items that went live, the tickets that were closed and the documents that passed review. The room can see that the team has been busy and that work has moved through its process.

The person funding the work listens, then asks their portfolio lead to say what is different outside the team as a result. There is a pause. Someone points to the release notes. Someone else explains that a dependency was removed and that the backlog is healthier. Those things may be useful, but they do not tell the funder whether a customer can now do something they could not do before, whether a business cost has fallen, or whether a material risk has changed.

The meeting turns into a defence of effort. The team has done what it said it would do. The funder has not received an account of what their money changed. Both statements can be true, which is why this conversation so often ends with more reporting rather than a better decision.

The principle

Output is the work a team produces: a release, a closed ticket, a set of story points, a design, a policy, a test result or a document. It is visible because the team controls most of it. Output matters. You need to know whether work is moving and whether technical standards are being met. It is not, on its own, a reason to continue funding a team.

Value is a meaningful change for someone who cares about the result. That person may be a customer, a colleague handling an operational task, a regulator, a business owner or the organisation that carries the cost and risk. Value is not a feature list with a better label. It is the difference the feature, service or decision makes when it is used.

A senior reader will recognise the distinction. A team may release a self-service form. The output is the form and the integration behind it. The value may be that a customer completes a request without calling the contact centre, that the contact-centre colleague has fewer avoidable contacts, or that a request is completed more accurately. A team may also close a large batch of security findings. The output is the closed findings. The value is lower exposure to an identified risk, provided the closure is real and the remaining risk is understood.

The value question is plain: for whom will something change, what will change and how will you know? Add timing and cost to create and sustain it. Until a team can answer, it cannot be measured in a way that settles a performance conversation. Every available measure will describe activity: items started, items completed, time booked, tests run or pages produced. Those measures can be accurate and still leave the main question unanswered.

That is why arguments about performance become unwinnable. One leader points to delivery volume; another to an unchanged business result; finance to cost. Each has a partial view, but none shares a defined result against which the team can be judged. The argument is then conducted through confidence, seniority and slide quality.

Google's team-effectiveness research cautions that measures such as lines of code and bugs fixed can mislead when used alone. The point is not to discard operational measures. It is to put them in their proper place: as evidence and guardrails around the result that matters. A team needs enough evidence to show that it delivered the right value, to the right standard, at the right cost and at the right time.

What it looks like when it is missing

In the next review, the same team returns with a fuller dashboard. The programme office has added separate counts for releases, defects, test cases, approvals and work in progress. The delivery lead can show that the queue is moving faster, but no one has agreed which customer or business result that movement should serve.

Work is selected because it is ready, because a senior stakeholder asked for it, or because it will make the plan look healthier. A feature reaches production and is treated as complete at deployment. Support receives new questions, operations changes a manual work-around and customers find another route, but those signals sit in different meetings. The team has no reason to bring them together because the work item is already closed.

Cost is discussed only when a request needs approval. The build estimate is compared with the remaining budget, while maintenance, training, support, security and retirement are treated as someone else's concern. When the outcome is weak, the response is to ask for more features, faster delivery or a more detailed plan. No one can show whether that extra activity is likely to improve the result. No one

owns the connection between delivery data and the result the work was meant to change.

What it looks like when it works

The team enters the review with a short statement at the top of its pack. It names the people affected, the change they should experience, the evidence that will show it and the date by which the team expects to learn. Its delivery measures are still there, but they are no longer the headline.

For an internal service, the team might state that people making a particular request should complete it without rekeying information, and that the organisation will see fewer avoidable contacts and less manual handling. The team brings task-completion evidence, operational data and a technical guardrail. If completion improves but service demand does not, the team investigates rather than declaring victory. If a release causes a reliability issue, the guardrail makes that visible even if the customer measure initially improves.

The funder can now ask useful questions. Is this the change we still need? Is the evidence credible? What will it take to achieve and sustain it? What should stop if this becomes the priority? The conversation moves from whether the team worked hard to whether continuing the work is a sound use of capacity. The statement stays open until the relevant evidence has been read and acted on.

How to do it

Use this method for a live workstream where its effects can be observed. It is a practical TEAM IQ method, not a claim that a sentence

creates value. Its purpose is to give the team and funder a shared result to test.

1. *Choose one workstream.* Select a live stream, not the whole portfolio or a single technical task. Bring its objective, operating data and the person who can explain its funding. One stream prevents a vague statement that tries to cover everything.
2. *Name the person who must experience the change.* Write down the customer, operational colleague, business owner or other affected group. Describe their present situation in observable terms. Do not begin with the feature. Begin with the friction, cost, delay, risk or missed opportunity the work is intended to change. If the team cannot name a group and a current condition, pause. It has a delivery request, not yet a value case.
3. *Write the change and the evidence.* Use this form: For [named group], change [current condition] to [desired condition] by [timeframe], shown by [evidence], while maintaining [necessary guardrail]. Make every field specific enough that another leader could disagree with it. A statement that says an experience will be better is not testable. A statement that says a named group can complete a named task with fewer avoidable contacts, shown through service data and task evidence, can be tested.
4. *Set the threshold and the cost view.* Apply the 7 C's Value Engineering Standard (CVE) before development begins. Define what good enough means for the customer result and the relevant quality requirements, such as accessibility, reliability, accuracy, security, performance or maintainability. Record the guardrails that cannot be traded away. Then make the unit economics visible.

A delivery team costs more than build effort: include people, contractors where used, platforms, licences, supplier commitments, support, operations, security, training, maintenance and retirement. Bring costs held elsewhere into the conversation.

The board-level view is value per team. Set the value achieved in the period against the full cost of keeping that team available for the period, and identify costs that the change creates, removes or avoids over its life. Do not turn uncertain effects into a precise financial claim. State the assumptions, distinguish evidence from expectation and revise the view as use and operating data arrive. A cheap build that creates recurring support work may be poor value; a larger initial outlay may be justified where it removes recurring cost or material harm.

1. *Test the statement before committing more capacity.* Ask a leader who did not write it to explain the promised change. Ask the funding owner whether the change would alter a priority decision. Ask people closest to delivery whether evidence can be collected without a new reporting programme. Finally, ask what result would show that the work should be stopped, changed or not extended. If the team cannot answer, rewrite the statement. Make uncertainty part of the decision.
2. *Review results, not just release status.* Build a 7 C's Outcome Metrics (COM) scorecard around the statement: at least one customer measure, one business measure and one technical guardrail. Review the evidence after release and at a cadence that suits the decision. Use it to scale, improve, reposition or retire the work.

Retain delivery measures, but read them as leading evidence, not as a declaration of value.

The exercise

Run this as a fifty-five-minute session with the manager, delivery team and, where possible, funding owner. Prepare one active workstream, its objective and any customer, service, finance or operational evidence. Do not prepare a polished business case. Expose what is known and assumed.

1. *Set the workstream and affected group — 10 minutes.* The manager names the stream and group expected to benefit. Members write the current condition independently: what happens now, where the friction is and who bears the consequence. Compare descriptions and agree one that can be observed.
2. *Draft the value statement — 15 minutes.* Use the form in the method: group, current condition, desired condition, timeframe, evidence and guardrail. Read it aloud. Remove feature names unless needed to describe the change. Capture unresolved assumptions beside the draft.
3. *Check cost and quality — 15 minutes.* Use the 7 C's Value Engineering Standard (CVE) to identify the good-enough threshold and guardrails. List lifecycle costs that could change: build, support, operations, security, training, maintenance and retirement. Mark what is known, what needs validation and who owns it.
4. *Run the board test — 10 minutes.* One person who did not write the statement explains the value, the evidence and the cost trade-off without looking at the draft. The group then names the result that

would cause it to stop, change or lose priority. If the explanation is unclear, revise the statement once.

5. *Close with ownership — 5 minutes.* Capture the statement, evidence sources, guardrails, lifecycle-cost assumptions and next review date. Assign one owner for evidence collection and one for the funding conversation. The manager sends the record before the next delivery review.

The output is not a promise to produce a particular feature. It is a testable account of why the work deserves capacity and what the team will learn.

What to measure

The primary artefact is a value statement supported by a 7 C's Outcome Metrics (COM) scorecard. The statement records the affected group, the intended change, the evidence, the timeframe, the guardrail and the lifecycle-cost assumptions. The scorecard holds at least one customer measure, one business measure and one technical guardrail, so the team does not replace one narrow output count with another.

Use the 7 C's Value Engineering Standard (CVE) to record the agreed good-enough threshold before work begins. Where the work is substantial, add a Lifecycle Value Review to bring support, operations, finance, security and product perspectives into the cost view. Read the statement and scorecard in the normal delivery review, then read the outcome evidence at the decision cadence set by the workstream. The funding owner should see the same record, not a separate account of progress.

One thing to do this week

Choose the workstream that consumes the most attention in your next delivery review. Before the meeting, ask its funding owner to complete one sentence: the group whose situation should change, the change they expect and the evidence they would accept. Put that sentence at the top of the status pack. If nobody can write it clearly, make defining it the first item of work rather than adding another output chart.

Part Three — The measurement cycle

3. The measurement cycle

At 08:30 on a Tuesday, the portfolio office has the numbers ready. There is a delivery dashboard, a risk register, a burn-up chart, a financial forecast and a weekly pack for the executive meeting. Each team has sent its update. Green, amber and red boxes fill the screen.

A director asks a simple question: is the claims team getting better? The room goes quiet. The delivery lead can say how many items moved across the board last week. Finance can say what has been spent. The test lead can say how many defects were found. Nobody can show the change the team set out to create, the starting point, what the latest work was expected to alter, or what they will do because the evidence has changed.

The team is not short of measurement. It is short of a route from measurement to action. The reports describe activity in several places, with different owners and review dates. A problem noticed in a meeting is copied into the next pack, then the next. Work continues because it was planned, not because the latest evidence supports it. By Friday, another report is being prepared. The organisation has measured a great deal and still cannot say whether the team is improving.

The principle

The *TEAM IQ Measurement Cycle* turns an intended outcome into evidence and then the next decision. Its stages are Set, Communicate, Own, Learn, and Decide and improve. A decision at the end changes the target, work or risk view for the next pass.

Set establishes the valuable change, scale, baseline, target, guardrail, owner and review date. The team and sponsor agree it;

the named metric owner maintains it. A shared target statement and T.A.R.G.E.T Value Index (TVI) are the artefacts. Skip Set and later measures may answer different questions.

Communicate makes claims, assumptions, evidence and changes visible. People doing the work bring evidence; the team maintains a common record. The C.L.E.A.R.L.Y Evidence Standard (CES) evidence log and Expression of Value Delta (EVD) update result. Skipped, a repeated opinion can pass as fact.

Own turns a hoped-for result into a commitment with accountability for progress and evidence. The primary owner is responsible; contributors make dependencies explicit. A D.R.E.A.M Outcome Contract (DOC) and, where authority or handoffs need clarity, Quantified D.R.E.A.M Boundaries (QDB), are the artefacts. Skip it and a target belongs to everyone in theory and no one when a trade-off is needed.

Learn tests assumptions that could affect the outcome or invalidate the plan. The outcome owner runs the test with the needed people; the team reviews it. A M.I.N.D.S.E.T Value Engineering (MVE) hypothesis and Risk-First Learning Loops (RLL) record leave the stage, alongside an updated risk view. Skip Learn and commitment rises as uncertainty does.

Decide and improve uses evidence to continue, change, stop or scale work. The named decision owner convenes the decision within the agreed boundary, with the team supplying relevant evidence. A D.I.R.E.C.T Value Routing (DVR) decision, IRT threshold, decision-log entry or DDA is the artefact. Skip this stage and the cycle produces reports that nobody uses; capacity and priorities remain unchanged.

The fourteen TEAM IQ measurement tools are instruments in the loop, not a separate reporting layer. Set uses T.A.R.G.E.T Value Index (TVI), Value-to-Effort Ratio (VER) and Target Risk Burn-Down (TRB). Communicate uses C.L.E.A.R.L.Y Evidence Standard (CES) and Expression of Value Delta (EVD); Own uses D.R.E.A.M Outcome Contract (DOC) and Quantified D.R.E.A.M Boundaries (QDB); Learn uses M.I.N.D.S.E.T Value Engineering (MVE) and Risk-First Learning Loops (RLL). Decide and improve uses D.I.R.E.C.T Value Routing (DVR), Insight Risk Triggers (IRT), D.I.R.E.C.T Decision Audits (DDA), 7 C's Value Engineering Standard (CVE) and 7 C's Outcome Metrics (COM). Each leaves a visible artefact for the next stage, rather than a second report beside the work.

Evidence moves deliberately. The TVI supplies the measure and guardrail for a DOC. CES separates knowledge from what needs testing. MVE and RLL update TRB, EVD and confidence. DVR routes capacity using value, effort, confidence and risk. The decision becomes a changed backlog, test, boundary or target, not a comment in a meeting record.

What it looks like when it is missing

The team starts the month with a list of commitments inherited from a planning meeting. The list has dates and named task owners, but no shared measure of the customer or business change expected. A dashboard shows completed work, defects and spend. When a senior stakeholder asks whether the work is helping, the team points to the growing list of completed items.

In the weekly meeting, people report that an integration is difficult. Nobody writes the assumption behind the design or decides what evidence would settle it. The risk stays amber because no one has defined a smallest test or named the decision it should inform. The work owner asks three colleagues for approval because the boundary of their authority is unclear.

At month end, the team has a slide describing the difficulty and a request for more time. It has no before-and-after measure, no record of what was learned and no explicit choice to stop, reduce or change the work. The next month begins with the same list, a revised date and another request for an update. Reporting has become the work that proves reporting is happening.

What it looks like when it works

The same team begins with one TVI: a stated outcome, current baseline, target date and guardrail. It names the person who maintains the measure and the person who owns each material outcome. A short DOC says what evidence will demonstrate progress, what support is needed and what the team will do if the expected movement does not occur.

In the weekly review, the owner uses EVD rather than an activity update. The update shows the before state, current state, learning and next action. One result has not moved. The team records the assumption in CES, puts a small test into the current work and updates the TRB. The difficult integration is now an exposure with a reduction experiment, not a paragraph in a status pack.

At the monthly decision review, the decision owner compares options through DVR. The evidence supports reducing one piece of work, testing another and continuing a third. Those choices are recorded with their rationale and review point. A trigger states what result would cause a pivot. The quarterly review looks across the decisions: whether targets still matter, whether the measures conflict, and whether the team is becoming more able to act on what it learns. The dashboard is shorter because each measure has a decision use.

How to do it

Run the first pass with one intact team and one material outcome. Do not build a portfolio dashboard or apply every tool. Establish a repeatable loop around work already under way.

1. *Days 1–3: Set the target and initial risk view.* Bring the sponsor, team lead, people closest to customer or service data, and the person who maintains the measure together. Write one TVI with value, scale, baseline, target, guardrail, owner and review date. List work that could move it. Use VER only for items on the same value scale. Create a TRB for the risks most likely to prevent the target, with a smallest risk-reduction experiment. Leave with the target statement, value-ranked work and initial TRB.
2. *Days 3–5: Make evidence and ownership testable.* Start a CES log for claims behind the selected work: supporting evidence, assumption, test, owner and date. Convert the next material commitment into a DOC: outcome, relevant measure, baseline and intended movement, evidence source, timeframe, dependency

and learning rule. Use QDB where the owner needs a decision boundary or measurable handoff. Keep these live, not filed.

3. *Weeks 1–3: Run evidence updates and tests.* In the existing weekly meeting, reserve fifteen minutes for EVD. The work owner states what changed, which measure should move, the current observation, learning and next action. Run the MVE test or RLL experiment agreed in week one, favouring the risk with highest exposure. Update CES and TRB as results arrive. If no measure has moved, record learning and the next test rather than inventing progress.
4. *Week 4: route evidence into a decision.* Hold a monthly review with the decision owner and team. Read the TVI, EVD updates, TRB, CES assumptions and MVE or RLL results. Use DVR to compare next moves on value, effort, confidence and risk. Set IRT thresholds for a pause, pivot, escalation or further test. Record the choice, owner, rationale and review date: continue, adapt, stop or scale.
5. *At the quarter boundary: inspect the system, not only the result.* Use DDA on one material decision. Check purpose, evidence, assumptions, risks, options, authority and review point. Where quality is a trade-off, use CVE to set the value target, scale, good-enough threshold, guardrail, cost and stop-or-continue rule. Read COM beside TVI for customer and business impact. Confirm or revise the next target.

Weekly is the rhythm for EVD, CES changes, active DOCs and TRB movement. Monthly is for a DVR decision, IRT review and capacity change. Quarterly is for DDA, target relevance and COM measures. This is a practical starting cadence, not a universal prescription. A

fast operational team may need shorter decisions; a slow-moving outcome may not yield useful weekly movement. Keep the review date in the artefact and change cadence deliberately.

The exercise

Run a 60-minute cycle launch with one team before the first thirty-day pass. Prepare one live outcome that matters now, the best available baseline, the current work list, known risks, and one person able to confirm how the measure is sourced. Bring the sponsor or decision owner if a decision boundary will be needed. Use a shared screen or board that everyone can see.

1. *Set — 15 minutes.* Write the TVI together: value, scale, baseline, target, guardrail, owner and review date. Ask each person to explain the target in their own words. Capture the shared target statement and the one or two risks that most threaten it.
2. *Communicate — 10 minutes.* Choose the claim behind the next item of work. Create one CES entry: claim, available evidence, assumption, experiment, owner and learning needed. Capture the evidence source and date rather than a vague confidence statement.
3. *Own — 10 minutes.* Turn that item into a DOC. Name the outcome, measure, evidence source, timeframe, dependencies and learning rule. Confirm the primary owner and, if needed, the QDB decision boundary. Capture unresolved support needs separately.
4. *Learn — 10 minutes.* Design the smallest MVE or RLL test that could reduce the most consequential uncertainty. State what ob-

servation would change the team's view. Add it to the current work and capture the planned test date.

5. *Decide and improve — 10 minutes.* Agree the first decision point, decision owner and IRT threshold. Write the future DVR question: continue, change, stop or scale what? Capture where the EVD, TRB and decision log will be read.
6. *Close — 5 minutes.* Schedule the weekly evidence review and monthly decision review. Photograph or save the board. The launch does not replace the test; it makes the first test and the decision it should inform visible.

What to measure

The first measure is the TVI: a current baseline, scale, target, guardrail, named owner and review date for the outcome. Its artefact is a target card or shared record that the team reads weekly. It prevents a pile of delivery measures from standing in for an outcome.

Read EVD and the CES log weekly to see whether the evidence behind current work is changing. Read the TRB with them when risk is material. The monthly artefact is the DVR decision record, supported by IRT thresholds and the decision log: what the team continued, changed, stopped or scaled, why, and when it will be reviewed. At quarter end, use DDA and COM to test whether the decision process and outcome measures remain connected to value, quality and risk.

One thing to do this week

Choose one team and one decision due in the next month. Before its next planning meeting, write a one-page TVI: the outcome, baseline, target, guardrail, owner and review date. Ask every attendee to

explain it without reading the page. Where their accounts differ, do not move to the work list yet. Resolve the target first, then schedule a fifteen-minute weekly EVD review and name the decision owner for the month-end choice.

Part Four — Target and Expression

4. True and explainable

The launch meeting has reached the last slide. A sponsor points at a coloured mock-up and says that account opening needs improving before launch. The product lead writes it into the plan. The compliance lead assumes the problem is the identity check. Operations hears a request to shorten the time between application and activation. The engineers begin discussing the screens that are hardest to change. Each person has taken away a reasonable interpretation. None has taken away the same assignment.

By Friday, there is activity everywhere. A designer has changed the welcome screens. An analyst has pulled a list of abandoned applications. A technical lead has asked about a third-party dependency. The status report says the work is under way. It cannot say whether the work will improve the result that prompted it, because nobody has agreed what that result is. The team has been given a direction, not a destination.

The principle

A *TARGET* is a team's shared line of sight to a valuable result. It is not a wish, a broad corporate ambition or a list of activity. It tells people what must change, why it matters, and how they will know it has changed. That distinction is easy to lose when a programme is moving quickly. "Improve onboarding" sounds useful until a team tries to decide which item should be done first, which compromise is acceptable and which risk can wait.

The Primary T.A.R.G.E.T Pathway® begins by making a target TRUE and EXPLAINABLE. TRUE means *Timely, Relatable, Urgent and Explicit*. There is a real date, connected to the team's work. The reason

for urgency is real as well: a customer commitment, a dependency, a regulatory requirement or a value opportunity. A date copied from a steering-paper template is not urgency. It is a signal that people will learn to disregard, along with the next genuinely urgent request.

Relatable matters because teams cannot act on a target that lives only in senior language. A customer outcome, an operational change or a risk reduction has to connect to work the team can see and influence. Explicit means that scope has boundaries. It says what is included, what is not, and what counts as enough. Timely does not mean hurried. It means that the date has a reason the people doing the work can repeat.

EXPLAINABLE is the test that exposes whether those elements have been done properly. A target is explainable when a person in the team can describe it without translating it into their own specialist version. They should be able to say the outcome, the stakeholder, the measure, the intended change and the date. If several answers sound sensible but differ, the target is still cloudy. The resulting disagreements will surface later as rework, delayed decisions or a dispute about whether the team has finished.

Google's Project Aristotle identified structure and clarity, including clear expectations and specific, challenging and attainable goals, as a dynamic of effective teams. TEAM IQ takes that finding into delivery practice: the team agrees a measurable stakeholder outcome rather than merely accepting a delivery activity.

The T.A.R.G.E.T Value Index (TVI) makes the agreement visible. It records the value that should change, the scale used to observe it, the present baseline, the desired target and the review date. Its purpose

is not to make a report look more managed. It is to prevent output from being mistaken for outcome. A completed screen, a signed specification or a released feature may matter, but none alone proves that the value has moved.

A team without this discipline can be extremely busy and still be unable to state what winning looks like. That is expensive. It fills meetings with local opinions, allows each specialist to optimise their own slice, and makes a missed date look like a delivery problem when the original problem was a target nobody could properly explain.

What it looks like when it is missing

Back in the account-opening work, the delivery plan contains “improve the process” as a heading. There is a launch date, but no one can say whether it represents a customer promise, a dependency or simply the date chosen for the plan. The product lead asks for a shorter form. Compliance asks for more evidence. Operations asks for fewer contacts. The engineers hear all three requests as in scope.

At the weekly review, progress is described in completed tasks: screens built, test cases drafted, a supplier meeting held. The commercial sponsor asks whether more eligible customers will complete the process. The room goes quiet, then someone promises to find a number. A baseline has not been agreed, so the number cannot be compared with anything useful.

The team then protects itself with broad language. Every item is urgent because nobody has separated the real deadline from the noise around it. A change that would help one audience is accepted because no agreed scope says otherwise. People work hard, but

they are coordinating around interpretations. The first time the gap becomes obvious is when a late decision forces work to be changed. The missing agreement means risks remain personal concerns until they become a programme problem.

What it looks like when it works

The same team pauses before adding work to the plan. It establishes that the commercial concern is early abandonment by eligible customers in the regulated account-opening process. It agrees which part of the process it can influence, what completion means, how the result will be observed and the date on which the result will be reviewed. Identity-verification accuracy is recorded as a condition that must not be damaged while the outcome improves.

The target is written in ordinary language and read back by the product, compliance, operations and technical leads. The wording survives the test: each can state the same result and the same boundary. The designer can now decide whether a proposed screen change supports completion. The analyst knows which baseline to validate. The technical lead can judge whether a dependency threatens the date or merely affects a lower-value option.

At the weekly review, the team talks about movement in the measure and what it has learned, not only what it has produced. A request that does not support the target is deferred with an explanation. A risk to identity-verification accuracy is treated as a decision about the target, not as a technical detail to hide in a separate meeting. The work becomes easier to coordinate because the destination is stable enough to guide choices.

How to do it

1. ***Start with the outcome, not the requested activity.*** Put the current request on the table exactly as it was received. Then ask the people closest to the customer, operation, risk or commercial need to state what should be different if the work succeeds. Keep pressing until the answer describes a change in value rather than a deliverable. "Release the new form" is output. "More eligible customers complete account opening within the intended period" is an outcome that can be tested.
2. ***Make the target TRUE.*** Agree the date and write down why it is the date. A real reason can be a customer commitment, a dependency, a regulatory requirement or a value opportunity. If no reason can be found, do not invent urgency to make the plan look decisive. Identify the team's actual area of influence so the target is relatable. Set clear scope boundaries. This is where a team distinguishes the essential outcome from worthwhile but separate improvements.
3. ***Make the target EXPLAINABLE.*** Write one sentence that includes the stakeholder outcome, the measure, the intended change and the review date. Use language that a delivery manager, a service colleague and a technical specialist can all repeat. Remove internal shorthand that only one function understands. Do not settle for nods around the table. Ask each person to say the target in their own words. Where the accounts differ, improve the wording rather than assuming the difference does not matter.
4. ***Create the T.A.R.G.E.T Value Index (TVI).*** Record five things: the value that should change; the scale used to observe it; the present

baseline; the desired target; and the review date. Keep the TVI close to the work, rather than filing it with a monthly pack. It should be visible when the team makes a trade-off. A short card is often enough, provided every field is complete and understandable.

5. *Validate the baseline before forecasting improvement.* Teams are often under pressure to promise a result before they know the current state. Do not turn a weak measure into false precision. If the baseline cannot be reliably established, say that the first cycle is a discovery cycle. The team can still make useful work visible, but it cannot honestly forecast the scale of improvement until it has evidence of where it starts.
6. *Use the target as the filter for delivery decisions.* Bring the TVI into refinement, planning, design reviews and progress conversations. For each significant item, ask how it helps move the stated value, what condition it must protect and whether it fits the agreed scope. This changes the quality of disagreement. Instead of arguing from a function's preference, people can argue from the shared destination. When circumstances change, revise the target openly and record why; do not allow separate informal versions to emerge.
7. *Keep urgency credible.* A new request may be genuinely urgent, but it should have to show the same evidence as the original target. If it displaces planned work, name what it displaces and what value is delayed. This is not resistance for its own sake. It protects the team from being asked to treat every instruction as a crisis and protects the real deadline when one exists.

The method is deliberately plain. Its strength is not in clever wording. Its strength is that the team has one account of the result, one way of observing it and one date with a reason behind it. That gives people permission to stop, challenge and choose without waiting for someone outside the room to interpret the target for them.

The exercise

Run a target-definition session for the team's highest-priority outcome. Prepare the current request, any known customer or business need, the available data for a baseline, and the relevant date or dependency. Give the group a copy of the current wording, even if it is vague. The point is to expose what it leaves unresolved rather than to pretend that it was clear all along.

Spend the first ten minutes asking what value must change and for whom. Capture competing interpretations without trying to settle them too quickly. Spend the next fifteen minutes agreeing what makes the target Timely, Relatable, Urgent and Explicit. Write the reason for the date, the scope boundary and the condition that must be maintained. Use the next fifteen minutes to draft the sentence that makes the target EXPLAINABLE, then ask every participant to repeat it without reading the draft.

Use the final ten minutes to complete one T.A.R.G.E.T Value Index (TVI): value, scale, baseline, desired target and review date. If the baseline is unreliable, record that the first cycle is a discovery cycle. Within five working days, publish the TVI card. In the next team meeting, ask every member to describe the target without looking at the card. Record the number of accurate explanations out of the

number of people present. Capture any difference in wording as work to fix, not as a personal failure. Keep the card in the working area, not only in the meeting record, for later use.

What to measure

The artefact is the *T.A.R.G.E.T Value Index (TVI)* card for the highest-priority outcome. It holds the value to change, the scale, current baseline, desired target and review date. Read it when the team plans work, reviews progress and makes a material trade-off. In the next team meeting, measure shared understanding by recording accurate explanations against people present. The goal is *100% shared understanding* before major work begins. If that standard is not met, improve the target and repeat the explanation test rather than treating the card as complete. A failed explanation test is evidence that the wording, scope or measure still needs work before delivery becomes costly for the people who must carry it out.

One thing to do this week

Choose the work item currently described with the broadest phrase on your plan. Book a short conversation with the people who interpret it differently. Do not discuss the solution first. Agree the stakeholder outcome, the measure, the baseline and the reason for the date. Write one TVI card and ask each person in the room to explain it without reading it.

5. Achievable and required

The quarterly planning board is full before the meeting starts. A sponsor has marked every request as a priority. A regulatory change has a date. Sales wants a commitment for a customer. The platform lead has added an architectural concern. A team member has put coaching and technical health at the bottom of the list, under a heading called “if time allows”. The delivery manager is asked to confirm that the team can take it all.

Nobody in the room wants to be the person who says no. The requests are all plausible. Each has an advocate with a reason to care. So the list is accepted with caveats, which means it is accepted. Work begins before the team has discussed capacity, uncertainty, dependencies or what quality will require. By the middle of the cycle, the team will be called slow. In fact, it was asked to commit to more work than a credible plan could contain.

The principle

A clear target can still fail if it is not achievable. Ambition is useful when it sharpens a team’s choices and harmful when it asks people to pretend that complexity, uncertainty and capacity do not exist. The ACHIEVABLE part of the Primary T.A.R.G.E.T Pathway® requires the people doing the work to review those realities before a commitment is made. It also requires an honest view of effort needed for quality, rather than treating quality as something to negotiate away once the date becomes uncomfortable.

Achievable does not mean cautious or low ambition. It means that a commitment has been tested by the team that must deliver it. They consider the work itself, the dependencies around it, the gaps in what

is known, the capacity available and the effort required to do it properly. Where evidence is missing, they make the uncertainty visible. That creates a plan that can be managed. A forced promise creates a plan that needs constant explanation.

There is another discipline inside the target: REQUIRED. A request is not a priority merely because somebody has requested it, or because it has reached a senior forum. The team should be able to trace every significant item to a customer, business, risk or strategic value and explain what will change if it is delivered. Work without that rationale may be desirable. It is not automatically required.

This matters because capacity is not limited only by development work. Learning, coaching, technical health and collaboration support the team's ability to meet the target. They do not happen in the gaps between supposedly real tasks. When leaders plan as though they do, the first pressure point removes the very work that keeps future delivery possible.

The Value-to-Effort Ratio (VER) makes these conversations visible. Its calculation is simple:

$$\text{VER} = \text{Estimated Value Impact} \div \text{Estimated Cost or Effort}$$

The number is not a promise of mathematical certainty. Both parts are estimates, and estimates are judgements. The value of the Value-to-Effort Ratio (VER) is that it makes those judgements discussable. Options can be compared using the same logic. Confidence can be challenged. A team can update its choices as evidence changes instead of defending the first list because it appeared in a plan.

Google's Project Aristotle found that a sense that work is making a difference is important for effective teams. A visible value rationale

does more than motivate. It gives people grounds to challenge work that consumes capacity without helping the target, and it gives a delivery manager a clear explanation for deferring it.

The cost of skipping this step is predictable. Teams are loaded with committed and non-committed work without distinction. Their best people spend time recovering from interruptions and late discoveries. Progress is reported as activity because nobody can explain the value case for the queue. The organisation then concludes that delivery needs more pressure, when it needs better choices.

What it looks like when it is missing

In the planning meeting, every request is called essential. Nobody has written the customer, business, risk or strategic value alongside it. The architectural concern is treated as an optional inconvenience because its value is not described. Coaching is dismissed as overhead. The team is given a date before it has seen a dependency that could determine whether the work is possible.

The plan does not distinguish what the team has committed to from what it might do if circumstances allow. A stakeholder hears a discussion item and reports it as promised. Another adds a new item because it sounds small. The delivery manager carries the resulting confusion into the weekly status report, where red, amber and green labels hide the fact that the team has no agreed basis for choosing.

When a problem appears, the response is to work harder on everything. Quality checks are compressed. Learning is postponed. The people doing the work know which requests have weak value, but no common measure gives them permission to say so. The loudest

request wins, not because it is most valuable, but because it arrived with more authority. The team is then judged against commitments it never had enough evidence or capacity to make.

What it looks like when it works

The same board starts with the T.A.R.G.E.T Value Index (TVI) for the active outcome. Each proposed item is placed beside that outcome and given a stated value case. The team marks whether it is a commitment for the next cycle or work that may be considered later. A request with no clear link is not quietly smuggled into the queue. It is returned for a clearer rationale or deferred.

For the remaining options, the people who will do the work discuss complexity, uncertainty, capacity, dependencies and the effort required for quality. They also reserve room for learning, coaching, technical health and collaboration. An architectural change may still be selected because it reduces a material risk to the target. A customer-visible enhancement may wait because its value case is weak or its assumptions need testing.

The team calculates the Value-to-Effort Ratio (VER) and records the judgement behind it. The ratio does not decide for them. It gives them a common starting point and exposes where confidence is low. The decision is published with what the team will do, what it will not do yet and why. A sponsor may disagree, but the disagreement is now about value, effort and uncertainty, rather than an accusation that delivery lacks commitment.

How to do it

1. *Make the upcoming work visible as choices.* Gather every item proposed for the next work cycle in one place. Include requests from sponsors, operational concerns, technical work, learning, coaching and collaboration. Do not start by arranging them into a delivery sequence. First distinguish between work that is being proposed and work the team is genuinely able to commit to. This avoids the familiar mistake in which a list of possibilities is heard as a list of promises.
2. *Connect each item to value.* For every item, state the customer, business, risk or strategic value it is expected to affect. Then connect it to the relevant target and its TVI. If nobody can say what changes when the item is delivered, it has not met the REQUIRED test. That does not make the request foolish. It makes it unsuitable for priority status until its reason is clearer. Capture the unanswered question and give it back to the person best placed to answer it.
3. *Test ACHIEVABLE with the people who do the work.* Ask about complexity, uncertainty, capacity, dependencies and the effort needed for quality. Take each answer seriously. A dependency is not covered by writing "dependency" in a plan; the team needs to understand how it affects the target and when it will be resolved. Uncertainty is not failure. It is a reason to adjust a commitment, sequence a learning activity or test an assumption before committing further.
4. *Calculate the Value-to-Effort Ratio (VER).* Estimate value impact and estimated cost or effort for each item, then use the same calculation across the list. Discuss the confidence behind each

estimate. A ratio based on a weak assumption should not be presented as a fact. The purpose is to compare choices consistently and reveal where the team needs more evidence. Keep the calculation alongside a short explanation, because a bare number can conceal the judgement that produced it.

5. *Protect work that sustains delivery.* Put learning, coaching, technical health and collaboration into the capacity discussion before the queue is full. These are not rewards for completing the programme work. They are part of the conditions under which the team can complete it. Where there is no capacity for all proposed work, make the trade-off explicit. Defer an item, reduce its scope or revisit its date; do not simply erase the work that makes reliable delivery possible.
6. *Choose, publish and revisit.* Select the items with the strongest combination of value, effort and evidence for the next cycle. State what is committed and what is not. Record why a lower-VER item was selected if a risk, dependency or other known condition justifies it. That preserves an honest decision trail. At the end of the cycle, compare the estimated impact with observed TVI movement. The objective is not perfect forecasts. It is better judgement over time.
7. *Use new evidence to change the queue.* A priority decision made at planning is not permanent. If an assumption is disproved, a dependency changes or the TVI shows an unexpected movement, revisit the VER. Explain the change in the same terms used for the original choice. Teams lose trust when work disappears without a

reason, but they gain trust when a visible value case is updated by evidence rather than by hierarchy.

The practical discipline is to make less work more honest. A team that has named its commitments can coordinate around them. A team that has named the value of its options can decline or defer work without making the discussion personal. That is how ambition becomes credible rather than merely loud.

The exercise

Run a prioritisation session for the next work cycle. Prepare the full list of proposed items, the relevant T.A.R.G.E.T Value Index (TVI), any available evidence of customer, business, risk or strategic value, and the team's known capacity and dependencies. Ask the people who will deliver the work to attend; a Value-to-Effort Ratio (VER) agreed without their view of effort is only a management preference with a calculation attached.

Use the first ten minutes to separate committed work from non-committed work. In the next fifteen minutes, ask the group to state the value case for each item and its link to the target. Mark items whose rationale cannot yet be explained. Spend the following twenty minutes discussing complexity, uncertainty, capacity, dependencies and quality, and estimating value impact and cost or effort. Calculate the VER for every item planned for the next work cycle. Capture confidence and assumptions beside the calculation.

Use the final ten minutes to select the planned work and record what has been deferred, stopped or sent back for a clearer value case. At the end of the cycle, compare estimated impact with the

observed TVI movement. Record where the team over- or under-estimated value. Do not use the review to assign blame for an estimate. Use it to identify which assumptions were sound, which were weak and what evidence should be sought earlier next time. Keep the reasoning visible so a changed priority can be tested against the original judgement when evidence changes.

What to measure

The measure is the *Value-to-Effort Ratio (VER)* for every item planned for the next work cycle. The artefact is a prioritised list that shows estimated value impact, estimated cost or effort, the ratio, its confidence and its connection to the TVI. Read it when work is selected and again at the end of the cycle. Compare estimated impact with observed TVI movement, then record over- and under-estimates. The measure is useful because it improves judgement over time; it is not a claim that estimates can remove uncertainty. Read the list with the team, not only in a sponsor meeting, so the people carrying the commitment can challenge an estimate before it becomes a promise and creates avoidable pressure in delivery.

One thing to do this week

Take the next request added to your team's plan. Before accepting it, ask the requester to state the customer, business, risk or strategic value and the TVI it supports. Ask the team what effort, dependency and uncertainty sit behind it. Calculate a provisional VER beside the existing work. If the item displaces something, name what will move and why. Write the answer beside the request.

6. Give

A service team has met the target it has carried for months. The waiting-time trend has moved in the right direction. Customers are getting through the service more quickly. At the review, the senior leader thanks the team, reads the number from a slide and moves on to the next issue. The slide is filed. The analyst who found the bottleneck goes back to a queue of work. The frontline colleague who tried a different hand-off hears nothing. The technical team that changed the slowest step sees only another request arrive.

Nothing hostile happened. The team was thanked. Yet the moment has told people that the effort disappears once the result reaches a report. It has not connected the work to the people helped by it. It has not shown how the result was achieved. It has not made visible the behaviour worth repeating. Over time, a team can learn that targets are a conveyor belt: finish one, receive a general acknowledgement, pick up the next.

The principle

TARGET includes GIVE because a team needs more than a destination. People need to see what achievement means for the customer, the business, the team and the people who contributed. The G.I.V.E model is *Gain, Indicate, Value and Edify*. It is a practical way to make reward and recognition part of delivery, rather than an occasional speech after the useful lessons have already been lost.

GIVE is not a bonus scheme in disguise. It is not generic praise, a points system or an individual contest dressed up as appreciation. It is a culture of edification: making specific contribution visible in a way that strengthens shared ownership. This matters in work where

outcomes depend on people handing work to one another, raising a difficult risk, learning a new skill or testing an idea that may not work. Individual-only reward systems can make those behaviours less likely by turning colleagues into competitors.

Gain makes the win explicit. The team should be able to say what changed and what has been achieved. *Indicate* shows progress visibly, so effort does not vanish into a black hole between one reporting cycle and the next. *Value* connects the result to the customer and organisation. It explains why the change was worth making. *Edify* recognises observable contribution: capability, courage, learning, collaboration and the specific actions that helped the team reach the outcome.

The order matters. Recognition detached from the result is easy to give but weak. It sounds kind, but it does not teach the team what value looked like or why a colleague's action made a difference. A shared result without recognition is also incomplete. The team sees a number improve, but not the contributions and behaviours that deserve attention next time.

Google's Project Aristotle found that meaning can come from the work itself, the output, supporting the team or other personally significant sources, while perceived impact helps people see how their work contributes to organisational goals. G.I.V.E makes that link deliberate. It does not assume that every person is motivated by the same thing. It gives the team enough evidence to connect effort, outcome and contribution without making claims about what should matter to any individual.

The absence of GIVE has a practical cost. Progress becomes a management artefact rather than a team event. People notice that invisible work is rarely recognised, so they may choose more visible activity over necessary support. A colleague who raises a risk can be seen as slowing things down rather than protecting the target. Teams then repeat the mistake of treating motivation as private sentiment, when part of it is shaped by what the organisation chooses to notice in public.

What it looks like when it is missing

The waiting-time result is announced as a number. The story stops there. There is no account of the customer experience that changed, no indication of the progress made during the work and no explanation of which decisions made the difference. A colleague who stayed with a difficult investigation is thanked in the same phrase as everyone else. A team member who tested an unproven hand-off is not mentioned because the contribution was less visible than a release.

The next delivery milestone becomes a race for personal visibility. People keep useful knowledge close because only the person who presents the final result is noticed. Help given across roles is absorbed into “team effort” with no detail. That may sound collective, but it gives the team no example of the behaviour it needs more of.

At the next review, the leader tries to correct this with louder praise. The praise is still detached from outcome and contribution. It does not tell the team what was gained, how progress was indicated, what value was created or what should be edified. The words are positive

but leave no usable memory. The result may be repeated, but the team has learned little about how to repeat it.

What it looks like when it works

The service team marks the result by making the Gain clear: the agreed waiting-time outcome has been reached. It indicates the path, showing the movement in the service area so people can see that progress was earned through several changes, not produced by a single announcement. It then states the Value in ordinary language: customers wait less, and the organisation has removed a source of avoidable delay.

Edification is specific. The analyst is recognised for exposing the bottleneck. A frontline colleague is recognised for testing a new hand-off in live work. The technical team is recognised for changing part of the process that was slowing the service. The team is recognised for combining those contributions and keeping the shared outcome in view. Nobody is presented as a lone hero, and nobody is hidden inside a vague “well done”.

The recognition includes learning as well as a completed outcome. A failed test that ruled out an attractive but unhelpful route can be edified when it reduced uncertainty and helped the team choose better. A difficult risk raised early can be edified when it protected the customer result. The team sees that contribution is not limited to the most visible task. It is linked to evidence, to the target and to the way the team worked together.

How to do it

1. *Set the reward component when the target is agreed.* Do not wait until a milestone has passed and then search for something pleasant to say. Alongside the target, agree what a meaningful Gain would be, how progress will be Indicated, what customer or organisational Value will be stated, and which forms of contribution should be Edified. This is a recognition plan, not a script. It makes the team alert to evidence while the work is happening.
2. *Name the Gain precisely.* Use the target and its TVI to state what changed. Avoid treating effort alone as the win. A difficult period may deserve acknowledgement, but it should not be confused with the result the team set out to achieve. If the outcome has not yet moved, recognise useful progress and learning honestly rather than declaring success early. This protects both morale and trust in the measure.
3. *Indicate progress where the team can see it.* Make the movement toward the target visible in the normal places where work is discussed. The point is not to create another dashboard. It is to prevent progress, blocked work and evidence from disappearing into a status report that few people read. Show what was tried, what changed and what remains uncertain. Visible indication allows a colleague's contribution to be linked to a real moment in the work rather than remembered only by the most senior person.
4. *State the Value beyond the metric.* Translate the result into its customer and organisational meaning. A reduced delay may mean less waiting for a customer. A risk reduction may mean greater confidence in a regulated process. A clearer hand-off may mean fewer avoidable contacts. Keep the claim proportionate to what

the team can prove. The purpose is to connect the result to a real stakeholder, not to write a grand narrative around a modest change.

5. *Edify observable contribution.* Name the person or group, what they did and how it supported the outcome. Recognise capability, courage, learning and collaboration when there is evidence for each. "Thank you for the support" is a gesture. "You exposed the bottleneck, which allowed the team to test the hand-off that improved the service result" is useful recognition. It tells the person that their work was seen and tells others what behaviour helped.
6. *Recognise both individual and team contribution.* Teams need both. An individual contribution can be specific without suggesting that the outcome was achieved alone. Team recognition can be collective without flattening the work into a generic phrase. In practice, describe the individual action and the hand-offs or combined decisions that made it effective. This reduces the destructive competition created by recognition that rewards only personal visibility.
7. *Include learning and constructive challenge.* A recognition culture that notices only successful delivery will make people hide uncertainty. Edify the person who raised a hard dependency early, the group that stopped a weak idea, or the colleague who shared a capability that improved the team's decision. Connect the recognition to the evidence and the target. GIVE should encourage ownership of the outcome, not just celebration when the result is already safe.

8. *Review the signal, not the enthusiasm.* At a milestone, check whether recognition statements name a specific outcome and observable contribution. If they do not, improve the practice. The aim is not more applause. The aim is recognition that helps the team understand what it achieved, why it mattered and how collective work produced it.

GIVE works because it makes the social part of delivery concrete. Teams see the line from target to customer value, and from contribution to result. That is more durable than a reward event because it makes useful behaviour visible while there is still another decision to make.

The exercise

Prepare a G.I.V.E review for an upcoming delivery milestone. Bring the relevant TVI, the available progress evidence, and short notes on contributions that have helped or hindered the path to the outcome. Ask participants to arrive with examples rather than names alone. The exercise is not a popularity discussion; it is a check that recognition is connected to real outcomes and shared contribution.

Spend ten minutes agreeing the Gain the team is seeking and the evidence that will show it. Use the next ten minutes to decide how progress will be Indicated visibly during the milestone. Spend fifteen minutes translating the target into its customer and organisational Value. Keep the language specific enough that a new team member would understand why the work matters. In the next fifteen minutes, identify contribution that can be Edified: capability, courage, learning and collaboration, with an observable action for each.

Use the final ten minutes to write the recognition statements the team will use when the evidence is available. For the next two delivery milestones, complete a G.I.V.E Recognition Signal review. Count the proportion of recognition statements that name a specific outcome and observable contribution. Capture whether both individual and team contribution are included. The target is that all recognition is evidence-linked and includes both individual and team contribution. Where a statement is generic, rewrite it against the outcome rather than adding more praise. Keep the statements with the milestone evidence so the review can be repeated consistently.

What to measure

The *G.I.V.E Recognition Signal* is the team's lightweight check on whether recognition connects to a real outcome and shared contribution. Its artefact is the set of recognition statements reviewed at each of the next two delivery milestones, with the stated outcome and observable contribution shown beside each. Count the proportion that meet both tests and read the result immediately after the milestone. The target is that all recognition is evidence-linked and includes both individual and team contribution. Treat a weak signal as a prompt to improve specificity, not to make recognition performative. Use the signal to refine the next milestone's recognition plan while the work and evidence are still fresh for people directly involved in that work.

One thing to do this week

At your next team review, select one contribution that would normally disappear into a general thank-you. State the outcome it supported, the observable action taken and the customer or organi-

sational value involved. Then name the team hand-off that made the action useful. Keep it brief and factual. You are setting a standard for what recognition looks like, not delivering a speech.

7. Traceable

The monthly architecture review has become a fixture in the delivery calendar. A development team brings a design, waits for the meeting, receives late questions and carries the answers into the following month. The date on the plan does not move, so development continues. When an architectural decision eventually arrives, work is changed to fit it. The status report shows milestones and approvals. It does not show the cost of waiting, the assumptions being made or the value that the review process was meant to protect.

In one of these meetings, the discussion centres on whether the team has completed the review-board process. That sounds like a reasonable governance question. It is the wrong target. The team needs sound architectural decisions in time for delivery. The review board is only one possible means of achieving that. When a process becomes detached from the result it serves, people can comply with it and still create delay, uncertainty and rework.

The principle

A target is *TRACEABLE* when the team can follow a clear line from a task, decision or deliverable to the stakeholder value it supports. That line makes work intelligible under pressure. When time is tight, the team can decide which technical debt can safely wait, which issue threatens the outcome and which feature should be deleted because it is no longer required. Without it, every item competes on local urgency and every function can claim that its own work must come first.

Traceability is not bureaucracy for its own sake. It does not require a large administrative system. It requires a shared explanation of why

work exists and what part of the target it affects. A developer should be able to connect a design choice to the value path. An analyst should be able to connect a process change to the stakeholder result. A delivery manager should be able to explain why a dependency matters before it turns into a missed commitment.

The Primary T.A.R.G.E.T Pathway® also treats risk as part of delivery rather than an unpleasant list kept for governance. The Target Risk Burn-Down (TRB) identifies the assumptions, dependencies and obstacles most likely to prevent the TVI target from being achieved. The team assigns each risk an impact and probability score, tests the highest risks early, and re-scores them after learning. It is a route to evidence, not an attempt to pretend that uncertainty has vanished.

The goal is not zero risk. A team that reports no risk in complex work may simply have stopped looking. The goal is a steadily safer and more evidence-rich path to the target. Each reduction experiment should tell the team something material: whether an assumption holds, whether a dependency can be met, whether a proposed approach protects the stakeholder outcome, or whether the target needs revisiting.

Google's Project Aristotle distinguishes an interdependent team from a work group because teams plan work, solve problems, make decisions and review progress together toward a specific project. Traceability and risk review give the team a shared basis for those actions. They replace a collection of specialist updates with a conversation about the same outcome.

When traceability is absent, rework often appears to be a technical failure or a people failure. In reality, a team was asked to make deci-

sions without seeing the value path and the uncertainty around it. A late question arrives as a surprise because the dependency was not connected to the target. A feature remains in scope because nobody can show why it is no longer required. The bill is paid in waiting, changed work and damaged confidence.

What it looks like when it is missing

In the architecture review, the team measures progress by whether a paper has reached the board. Development carries on because the delivery date is fixed. Architects hold questions until the next formal meeting because that is how the process works. By the time a decision is made, a design has been built around an assumption no one tested. The subsequent change is described as rework, but the status report records only that the approval was completed.

The risk list is maintained separately. It has assumptions, dependencies and obstacles, but they are not connected to the TVI or to the decisions in the delivery plan. Items are discussed when a governance forum asks for them, not when they are most likely to become expensive. Impact and probability scores are copied forward. There is little evidence that the team has tried to reduce the highest exposure.

A request to remove a low-value feature starts an argument because the feature has become part of the plan. Nobody can state the stakeholder value it supports, but nobody feels authorised to delete it. Meanwhile, a dependency that could affect the target is treated as someone else's problem. Work continues around it until the uncertainty becomes a delay.

What it looks like when it works

The team restates the real result: sound architectural decisions in time for delivery. It maps the active work, decisions and dependencies to the stakeholder value and the TVI. The review-board process is then assessed against that result. Where the monthly meeting creates delay without improving decision quality, architects are embedded with delivery teams and shorter weekly decision reviews are used. Dependencies are made visible where planning happens.

Each material assumption, dependency and obstacle is entered into the Target Risk Burn-Down (TRB) with impact and probability scores. The team does not wait for the full design to be complete. It tests the highest risks early. A decision review can then use evidence from the experiment, not only opinion. After the learning, the scores are revisited and the exposure trend is visible.

When a feature no longer supports the value path, the team can remove it with a clear explanation. When technical work protects a condition required by the target, it can be prioritised without pretending that it is merely an internal preference. The group has fewer arguments about whose work matters most because it can see the line from the work to the outcome. It is not free of uncertainty; it is dealing with uncertainty while choices are still affordable.

How to do it

1. *Begin at the target and the TVI.* Put the current stakeholder outcome, the value measure, baseline, desired target and review date where the team can use them. Traceability starts with a result worth tracing to. If the TVI is unclear, repair it before constructing

a detailed account of the work beneath it. A precise list connected to a vague target only creates precise confusion.

2. *Draw the line from work to value.* For each material task, decision and deliverable, state the stakeholder value it supports and the part of the target it affects. Keep the statement short enough to use in planning. Where the line cannot be drawn, ask whether the work is required, whether it is supporting a condition that must be protected, or whether it should be removed. This is the point at which low-value features and hidden assumptions become visible.
3. *Make dependencies part of the value path.* A dependency is not external background. It changes the team's ability to achieve the target. State what must happen, who or what it depends on, when the uncertainty becomes material and what stakeholder value is at risk. Bring it into the same discussion as scope and sequencing. This enables a team to act earlier than a forum that sees the issue only when it has become a late exception.
4. *Build the Target Risk Burn-Down (TRB).* Identify the assumptions, dependencies and obstacles most likely to prevent the TVI target being achieved. Give each risk an impact and probability score. Use a consistent team scale and record the reasoning, so a score can be challenged with evidence rather than treated as a private judgement. The TRB is not a catalogue of every possible problem. It is a focused view of the uncertainty that matters to the target.
5. *Test the highest risks early.* Select the risks with the greatest exposure and agree a reduction experiment for each. An experiment may test an assumption, clarify a dependency or produce

evidence on an obstacle. Make the intended learning explicit before the test begins. A meeting that simply repeats concern does not reduce uncertainty. After the learning, re-score the risk and record the evidence that changed the assessment.

6. *Use the TRB in weekly delivery decisions.* Review the top risks, their exposure scores, the latest reduction experiment and evidence of change. Ask which planned item is affected by the current risk profile and whether scope, sequencing or a decision needs to change. If a risk cannot yet be reduced, make that fact visible and explain its effect on the target. This is more useful than reporting a nominally green plan that depends on untested assumptions.
7. *Challenge process by the result it serves.* A review, approval or document may be necessary, but it should be judged by whether it helps the team make sound decisions in time for delivery. If a process produces late learning, find a shorter feedback route that preserves the needed decision quality. The aim is not to discard governance. It is to connect governance to stakeholder value, evidence and the target.
8. *Preserve the decision trail.* When work is added, removed or changed, record the trace to value and the evidence behind the decision. This gives new team members and sponsors a reliable explanation of why the queue looks as it does. It also prevents a previous decision being defended merely because it is old. Traceability should make it easier to change course when evidence demands it.

A traceable target changes the team's relationship with uncertainty. Risks are no longer an administrative burden and dependencies are no longer someone else's issue. Both become part of the work of protecting a valuable result. That is what allows a team to move quickly without pretending that speed removes the need for thought.

The exercise

Run an initial Target Risk Burn-Down session for one active initiative. Prepare the current T.A.R.G.E.T Value Index (TVI), the list of active work and decisions, known dependencies, and any existing risks or assumptions. Ask the people who hold delivery, operational, technical and stakeholder knowledge to attend. The purpose is to create a shared risk profile connected to the target, not to transfer risk ownership into a document.

Spend the first ten minutes restating the target and identifying the line from active work to stakeholder value. Use the next fifteen minutes to list the assumptions, dependencies and obstacles most likely to prevent the TVI target being achieved. In the following fifteen minutes, assign impact and probability scores and agree the highest exposure risks. Use the final ten minutes to choose a reduction experiment for each of the highest risks and state the evidence that would justify a re-score.

Maintain the TRB weekly for the active initiative. Report the top three risks, their exposure scores, the latest reduction experiment and the evidence of change. Capture the trend of exposure over time, not only the current profile. If a risk has not changed, record why and what decision is required. If a test has reduced uncertainty, re-

score it openly. The exercise succeeds when the team can show how it is making the path to the target safer and more evidence-rich. Keep the current profile visible beside the previous review so the team can discuss direction as well as urgency.

What to measure

The *Target Risk Burn-Down (TRB)* is the artefact and the measure. It records the assumptions, dependencies and obstacles most likely to prevent the TVI target, with impact and probability scores, reduction experiments and evidence from each review. Read and update it weekly for one active initiative. Report the top three risks, their exposure scores, the latest reduction experiment and the evidence of change. The useful measure is the trend of exposure over time: it shows whether the team is reducing uncertainty before it becomes expensive rework. Use the weekly view in delivery conversations, so the people making choices can see which uncertainty has changed and which still needs a reduction experiment before it threatens the target or forces rework later.

One thing to do this week

Choose one active dependency that appears in a plan but not in a target conversation. Write the stakeholder value it affects and the assumption that makes it risky. Add it to a Target Risk Burn-Down (TRB) with impact and probability scores. Agree one small reduction experiment and the evidence you need from it. Review the result with the team before the next plan is published.

8. Create safety and lead by example

At 08:40 on the morning of a release, the test analyst opens the deployment checklist and finds that the rollback instruction points to an old environment. The delivery manager is already on a call with the operations lead. A product owner is asking when the customer notice can go out. The analyst has checked the link twice. It is wrong, but raising it will delay a release that has been described as straight-forward all week.

For a moment, nothing happens. The analyst drafts a chat message, deletes it, and looks again at the checklist. In teams without safety, this is where a small defect becomes a later incident. People wait for certainty beyond what the work can offer, or decide that the most senior person must already know. By the time the issue is raised, the choice is no longer between a short pause and a correction. It is between a difficult recovery and an explanation of why nobody spoke when there was still time.

The principle

Psychological safety is the confidence that you can take an interpersonal risk at work without being embarrassed or punished. In delivery terms, that means admitting that you do not know, asking a basic question, reporting a mistake, challenging a premise, or saying that an estimate has become unreliable. It does not remove standards. It makes it possible to see whether the standards can be met before a status report has turned a concern into a surprise.

Google's Project Aristotle places psychological safety first among its listed team dynamics. Its definition is practical: teammates do not embarrass or punish one another for admitting mistakes,

asking questions or offering ideas. The Massachusetts Institute of Technology collective-intelligence studies also associate collective intelligence with social sensitivity and a more equal distribution of conversational turn-taking. Neither finding says that politeness produces delivery. Both point to a condition for good judgement: the people closest to a risk must be able to put it into the work.

Safety is often misunderstood as a promise that every suggestion will be accepted or every decision will be comfortable. It is neither. A safe team can stop a weak proposal, test an uncertain claim, set a demanding date, and hold an owner to an agreement. The difference is that it deals with the work rather than making the person who raised the issue the problem. "What are we seeing, what does it mean, and what do we need to do next?" is a better first response to bad news than a search for somebody to blame.

Leaders set this condition in ordinary moments. A leader who becomes defensive when challenged teaches the team to keep its doubts private. One who publicly dismisses an idea teaches people to edit themselves before they speak. A leader who says, "I may be missing something — what do you see?" makes room for information that their role does not give them. A leader who names a wrong call and asks what can be learned makes fallibility discussable rather than dangerous.

The C.L.E.A.R.L.Y Evidence Standard (CES) gives that behaviour a place to land. It moves a claim out of personality and into one of four forms: an assumption, a hypothesis, an evidence statement, or a test plan. The CES asks whether evidence is available, whether an assumption is recorded, whether an experiment is defined if needed,

and whether learning is captured. A challenge can then be directed at the claim, not at the status of the person making it.

What it looks like when it is missing

The analyst eventually sends a private message to a colleague: “This link may be old.” The colleague replies that the operations lead probably knows. On the call, the delivery manager says the release is ready because that is what the last update said. Nobody wants to be the person who reopens the plan.

The release proceeds. When rollback is needed, the team discovers the error under pressure. The first questions concern who changed the checklist and why it was not found sooner. The analyst says that the link had looked wrong that morning, then wishes they had said so earlier. The room hears a confession rather than useful evidence. The next person who spots an uncertainty will remember that exchange.

You can see the same pattern in planning. A senior stakeholder says that customers will definitely prefer a new approval route. The statement enters the plan as fact. A quieter team member who has no customer evidence stays silent. The team now carries an untested preference as if it were a requirement, and discussion shifts to defending the date built on it.

The claim gains weight with every plan that repeats it. Soon, challenging it feels like challenging the stakeholder rather than checking a condition that matters to the work.

What it looks like when it works

The analyst posts the concern where the release team can see it: the rollback link is old, the current instruction has not been verified, and the team needs ten minutes to check it. The delivery manager thanks them for catching it before deployment and asks the operations lead to validate the route. The customer notice waits. Nobody pretends that the pause is convenient; the point is that it is visible and manageable.

The team corrects the checklist, records what was learned, and continues. Later, the manager asks what in the review method allowed an obsolete link through. That question is not soft. It seeks a change in the work, not a performance of regret.

In the approval-route discussion, the leader responds to the quiet challenge by saying that the team is assuming customer preference. The group records the assumption and agrees to run a prototype test with ten customers before committing. The stakeholder's view remains useful as a hypothesis. It no longer has to win because of who said it. This is leadership by example: curiosity is shown when the cost of curiosity is a slower decision. The record distinguishes experience from customer evidence. That distinction allows the team to proceed with a visible test rather than pretending that a confident view has settled the matter.

How to do it

1. *Make the expected response to bad news explicit.* Say at the start of planning, review and delivery work that concerns, uncertainty and mistakes are inputs to the work. Use a consistent first response: establish what has been observed, what is known, what

is assumed, and what needs to happen next. You are not asking people to be fearless. You are making the route for raising a concern clear enough to use under pressure.

2. *Model uncertainty before asking others for it.* A leader can say, "I may be missing something," before giving a view; ask what evidence supports the current position; and state plainly when a previous call needs revisiting. Do this in front of the people affected, not only in a private conversation with trusted colleagues. The team watches what happens to dissent in public. It also notices whether the leader leaves enough time for an answer rather than supplying the answer immediately.
3. *Separate a claim from its owner.* When somebody says, "This will definitely work," do not answer with an argument about confidence or seniority. Ask which form the claim is in. Is it an evidence statement? Is it an assumption that needs recording? Is it a hypothesis that could be checked? Or does it need a test plan? The language is deliberately ordinary. Its value is that it makes uncertainty visible without treating it as incompetence.
4. *Use the CES at the point of commitment.* Before work is approved, a decision is made, or a delivery promise is repeated, check the four conditions: evidence available, assumption recorded, experiment defined if needed, and learning captured. Not every claim will meet all four immediately. A claim with no evidence may still be necessary to move work forward, but it should be recorded as an assumption with a next step rather than dressed up as certainty.
5. *Welcome the useful challenge in the moment.* Thank the person who notices the missing condition, then direct the group to the

work. “What evidence do we have?” and “What would test this?” are stronger than “Are you sure?” because they request something the team can act on. If you disagree, explain the reasoning and leave the challenge visible. A polite dismissal has the same long-term effect as an angry one.

6. *Close the learning loop.* When an experiment finishes or an assumption proves wrong, capture what changed in the team’s view and what will be done differently. This matters especially when a concern did not turn into an incident. If the only celebrated intervention is the dramatic rescue, people learn to wait for drama. Recognise the early question that prevented it.
7. *Review your own modelling.* After a difficult discussion, ask whether your response made the next concern easier or harder to raise. This is not a request for a personality assessment. Look at observable behaviour: did you invite a view, name your uncertainty, ask for evidence, and keep the issue separate from the person? Repetition is what makes safety credible.

The exercise

Run a 55-minute CES launch session, then use the two-week sampling task in live work. Prepare five anonymised major claims from recent planning, review or decision meetings. They should be genuine types of claim the team makes, but do not use the session to expose an individual. Bring a simple page for each claim with space for the four CES conditions.

Spend 10 minutes setting the purpose: the team is learning to test claims, not grading confidence. In the next 15 minutes, take two

claims and have small groups decide whether each is an assumption, a hypothesis, an evidence statement, or a test plan. Capture their reasoning. Keep the claims short enough to assess in the room. Do not debate whether the underlying proposal is attractive; identify what evidence or test would allow the team to treat it with greater confidence. Spend 15 minutes scoring the same claims against the CES: evidence available, assumption recorded, experiment defined if needed, and learning captured. Where a condition is absent, record the next action rather than an excuse.

Use the final 15 minutes to agree how the team will sample five major claims over the next two weeks and who will collect the record. At the end of the two weeks, score each sampled claim against all four conditions and publish the percentage meeting every condition. Capture the percentage, the claims sampled, the missing conditions, and one change to the team's next planning or review conversation. Improvement, not perfection, is the purpose.

What to measure

The CES produces a percentage: the share of five sampled major claims that meet all four conditions — evidence available, assumption recorded, experiment defined if needed, and learning captured. The artefact is the two-week claim record and its published percentage, not a score for individual people.

Read it after each two-week sample in a planning, review or decision forum. Look first for the condition most often absent. A low evidence rate may point to rushed assumptions; missing learning may show that experiments are being run without changing the next

decision. Read a small sample of the claims as well as the percentage, so the team can see the kind of statement that is escaping the standard. The measure is useful when it changes how the next claim is handled.

One thing to do this week

At your next delivery review, choose one major claim that is being treated as fact. Ask whether it is evidence, an assumption, a hypothesis, or a test plan. Record the answer and the next check. Thank the person who supplies the uncertainty, even if the review now needs a short pause. Then watch your own response when somebody challenges it.

9. Expectations: the clarity code

A programme has agreed a change to the order of a migration. The decision is made in a video call late on Tuesday. One engineer posts a short note in chat before signing off. A project manager assumes the revised order will appear in the delivery plan. The supplier sees neither item until Thursday, when their team begins work on the original sequence. By Friday, three people have different versions of what was decided, and each can point to a reasonable place where they expected the decision to be recorded.

Nothing in this scene requires bad intent. The chat note was visible to some people. The call was attended by the people who spoke. The plan had always been the place where schedules lived. The failure sits in the gap between those conventions. If a team has not agreed what makes a decision real, where it belongs, and who must see it, everyone supplies their own rule. The result looks like poor communication after the fact, but it began as an unspoken expectation.

The principle

Most communication failures in delivery are failures of agreement before they are failures of effort. One person thinks a quick chat message is enough to settle a decision; another expects a formal record. One person treats direct language as the fastest way to remove ambiguity; another experiences it as aggression. One person fills every pause; another waits for an invitation that does not come. Neither a new channel nor a longer meeting fixes rules that have never been made explicit.

The *Clarity Code* is the team's agreed account of what good communication looks like in its work. Its five elements, the 5 C's, are

Clear expectations, Courage, Clogging prevention, Choose the right medium, and Concise and clear communication. The title matters less than the discipline. A team needs to say how decisions are recorded, what must be raised, when people may challenge, how airtime is balanced, and which channel carries which kind of information.

Clear expectations remove routine guesswork. They define, for example, the difference between a discussion and a decision, what needs an owner, and when an update is complete. Courage permits people to be open and direct. It must be joined to kindness: directness is not a licence to make colleagues smaller. Clogging prevention addresses the fact that expertise and confidence are not the same thing. When the same voices occupy every discussion, information from people with less status or a different style arrives late or not at all.

Choosing the right medium protects the team from two common errors. The first is burying a significant decision in a quick channel that is useful for coordination but poor for retrieval. The second is turning every small question into a formal process, so the record becomes too slow to use. Concise and clear communication gives a reader the context, decision, action, owner and deadline without making them reconstruct the work from a chain of messages.

The Massachusetts Institute of Technology (MIT) collective-intelligence studies found that a more equal distribution of conversational turn-taking was associated with collective intelligence. This is not an instruction to make every contribution the same length. A security specialist may need more time on a material risk than a project

observer. It is a warning against treating the loudest or most fluent contribution as the whole of the team's knowledge. The Clarity Code turns that warning into a rule the team can see and revise. It gives the manager a basis for changing the process, rather than making a private judgement about confidence.

What it looks like when it is missing

On Thursday, the supplier asks why the migration order has changed. The project manager replies that it was agreed on Tuesday. The engineer says it is in chat. The supplier says chat is not a decision record. All three are correct according to their own working practice.

The team now spends an hour reconstructing the call, searching messages and asking who was present. The delivery plan is amended, but there is no agreement about the next change. A senior person says that people should "communicate better". That instruction adds nothing. It does not say what belongs in the plan, when a chat item becomes a decision, or who confirms that the people affected have seen it.

In the next call, the usual two voices fill the time explaining the correction. A delivery analyst who spotted an impact on a dependent release has no defined route into the decision. The impact remains in a notebook. The team has recorded activity, but it has not created shared understanding. The same uncertainty will surface again as an apparent delivery surprise. People spend time asking who should have known, although the team never agreed how a dependency moved from an observation into a decision record.

What it looks like when it works

Before the next migration change is discussed, the team opens its Clarity Code. It says that decisions above its agreed impact threshold are posted in a shared decision log within 24 hours. The entry uses one page: context, options, decision, evidence, owner, implications and review date. Chat remains the place for quick coordination; the log is the source of truth for the decision.

The change is therefore recorded before work begins. The supplier can find it without asking who attended the call. The delivery plan points to the decision rather than attempting to duplicate its reasoning. The analyst adds the dependent-release impact in the implications section. The decision owner checks that the affected teams have seen it.

The discussion is still direct. Someone can say that the proposed order creates risk. The Code makes that challenge expected, and the facilitator watches whether a few people are clogging the conversation. The objective is not a quieter call. It is a record and a conversation in which relevant information can arrive before the work moves. When the decision is revisited, the team can see its evidence and review date without reconstructing a memory. That makes change visible and preserves time for the next issue.

How to do it

1. *Start with the communication failures that cost the team time.*

Bring examples such as a decision that had to be rediscovered, an action with no visible owner, a risk placed in the wrong channel, or a discussion dominated by the same people. Describe the event

without making it a case against an individual. You are looking for an absent rule, not a culprit.

2. *Write Clear expectations as observable agreements.* State what makes a decision complete, where it is recorded, how quickly it is recorded, who owns the follow-through, and how an action carries its deadline. Be specific about the artefacts already used by the team. "Keep people informed" is not an expectation that can be checked. "Post decisions above the agreed impact threshold in the shared decision log within 24 hours" can be checked.
3. *Define Courage with kindness.* Agree that people may challenge an idea, ask for context, and state a concern early. Agree also that the challenge addresses the work, evidence or implication rather than a person's competence or motive. Courage is useful when it makes a difficult fact available; it is not a preference for blunt language. A manager must show this distinction when a challenge is inconvenient.
4. *Address Clogging prevention in the live conversation.* Notice who is doing the talking and who holds relevant knowledge. Build a pause into significant decisions so people can add an overlooked dependency, risk or customer effect. Invite a view by role or by the work closest to it, rather than inviting a vague "any other business" response after the conclusion has formed. Do not force every person to speak on every item. Make it possible for the right person to contribute before the decision closes.
5. *Choose the right medium deliberately.* Use quick channels for coordination that loses value once the immediate work is done. Use a shared record for decisions, commitments and material

implications that people will need to retrieve. Use a conversation when the issue needs exploration, disagreement or shared interpretation. Tell the team where the final record will be before the discussion begins. The medium should match the cost of missing the information, not the sender's habit.

6. *Make communication concise and clear.* For a decision or update, give the context first, then the decision or current position, the action, owner and deadline. Add the evidence and implications where they matter. This is not a demand for short messages at all costs. A concise record may need enough detail for someone absent from the call to understand why the work changed.
7. *Review the Code when work changes.* The team should not treat its first draft as permanent. A new supplier, hybrid working pattern, dependency or delivery pressure may expose a rule that no longer works. Revise the agreement in the open and tell people what changed. A norm that lives only in the memories of long-standing members is still an unspoken rule. Read the revised Code after its first use and correct unclear wording promptly.

The exercise

Set aside 60 minutes to create the first Clarity Code. Prepare three recent communication failures: one about a decision, one about a channel, and one about access to the conversation. Also prepare the existing decision record, delivery plan, chat channel and any routine update the team already uses. The aim is to improve the team's actual working agreements, not to publish a generic list of good intentions.

Use 10 minutes to describe the three examples and identify the unspoken rule in each. Spend 25 minutes working through the 5 C's. For Clear expectations, capture the team's rules for decisions, actions, owners and deadlines. For Courage, agree how direct challenge and kindness sit together. For Clogging prevention, agree how relevant voices enter a significant discussion. For Choose the right medium, map quick coordination, decisions and issues to the existing channels. For Concise and clear communication, agree the minimum information a decision or update must carry. Test each rule against one of the prepared failures: could somebody unfamiliar with the discussion follow the rule and avoid the same loss of information?

Use 15 minutes to turn the agreements into a short, visible Clarity Code and to nominate its owner. Spend the final 10 minutes agreeing when it will be tested. At the end of the next two work cycles, run the Clarity Code Pulse. Capture the Code itself, the average rating, the spread of responses and one rule to retain or change. Name the date and forum for that review before the session ends.

What to measure

The *Clarity Code Pulse* is a fortnightly, five-item team check. Each team member rates agreement from 1, strongly disagree, to 5, strongly agree. It measures whether the agreed Code is experienced in the work, rather than whether a document exists.

Read the average and the spread of responses at the end of the next two work cycles. Do not read the average alone. A high average with one very low score may mean that a quieter group is having a different experience of access, courage or clarity. Keep the responses

visible long enough to compare the second cycle with the first, then discuss the reason for any difference. Use the response pattern to choose one rule to examine with the team.

One thing to do this week

Choose one decision your team will make this week. Before the discussion, state where the decision will be recorded, who will own it, and when affected people will see it. Afterward, check whether the record contains the context, decision, action, owner and deadline. Send the record to one affected colleague who was not in the discussion and ask whether it can be understood without a follow-up call.

10. Listening, and the habit of team building

A product manager arrives at a security review with a launch date already committed to a partner. The security lead has spent the morning tracing a weakness in a third-party component. Both have material information. Both are tired of being asked to solve the other's problem. The product manager explains the commercial consequence of delay. The security lead makes notes, looks back at the risk assessment, and waits until the explanation is complete.

The next minute determines whether the session becomes a trade between two fixed positions or a piece of joint work. The security lead can begin with a defence of controls. That response may be technically sound and still leave the product manager unheard. Or the lead can show that the deadline has been understood, ask what is fixed and what is flexible, and then explain the risk. The difference is not softness. It determines whether the team can see a smaller, staged launch that protects both concerns.

The principle

Listening is often mistaken for waiting quietly while another person finishes. In delivery, that version of listening is expensive. People prepare their counterargument, collect the facts that support their own position, and leave a conversation believing they have communicated. The team has exchanged statements but has not tested whether the underlying concern, constraint or question was understood. Important distinctions disappear: a fixed contractual date becomes a preferred date; a control requirement becomes a preference; a request for help becomes a request for approval.

The *E.A.R.S. active-listening framework* makes listening observable. *Engage fully* means being present rather than formulating a counterargument while someone is talking. *Ask* means using clarifying questions to test understanding. *Respect* means acknowledging another perspective even when you disagree with it. *Show empathy* means recognising the experience, concern or pressure behind the words. These are not scripted phrases. They are behaviours that let a team use more of the information it already has.

The Massachusetts Institute of Technology (MIT) collective-intelligence study found that average social sensitivity was associated with groups' collective intelligence. Social sensitivity can sound like a personal trait that a manager cannot influence. E.A.R.S. gives it a practical form. You can observe whether someone is engaged, whether their questions test understanding, whether disagreement still acknowledges another view, and whether the pressure behind a concern has been recognised. That makes listening something a team can practise rather than praise in the abstract.

Regular team building supports the same work. It is not an optional social extra placed at the edge of delivery. Personal connection gives people context for interpreting a terse message, asking for help early, and giving a colleague the benefit of the doubt when pressure rises. It also creates practical knowledge: who has experience of a system, who thinks best with a written brief, who needs time to examine a risk, and who can explain a customer consequence. In hybrid teams, those connections must be designed. They will not reliably appear between booked calls.

Team building does not mean compulsory intimacy or an event disconnected from the work. Its purpose is trust and useful knowledge of one another's strengths. A regular, modest ritual is more credible than a rare away day that leaves the working pattern untouched. The test is simple: after the team has spent time together, are people more able to seek context, ask for help and interpret a colleague's intent accurately when the delivery timetable tightens?

What it looks like when it is missing

The product manager finishes the launch explanation. The security lead responds with a list of controls and the phrase "we cannot accept that risk". The product manager hears refusal. The security lead hears a demand to ignore a material weakness. Neither checks what the other has actually said.

The conversation turns into prepared speeches. A programme lead tries to settle it by asking which date is more important. The team chooses a position without distinguishing the partner commitment from the first-mover concern, or the security control from the specific exposure. The matter returns later with more frustration and less time.

Outside the review, people know little about how their colleagues work. A short message from the security lead is read as obstruction. A late request from product is read as carelessness. No one asks for the context because past exchanges have taught them that the answer will be defensive. A team can still be courteous in this state. Courtesy alone does not create understanding. The issue then moves into delivery planning as two competing stories. Neither story con-

tains enough of the other person's information to support a useful choice, so the team carries the disagreement forward instead of resolving it.

What it looks like when it works

The security lead begins by summarising: the product manager is worried that a two-week delay will miss the partner launch and lose first-mover advantage. The product manager confirms the summary. The lead then asks which part of the date is contractual, which is commercial, and whether a staged launch is possible.

When the risk is explained, the product manager does not have to agree with every control to acknowledge the concern. The conversation has established that both people are trying to protect a customer outcome. Together, they examine a smaller launch that avoids the weakness while preserving the most time-sensitive part of the partner commitment.

The team has practised this kind of exchange in routine work. People know who can explain the component, who can test the customer effect, and who needs a written question before a difficult conversation. Their team-building time has supplied useful context, not a slogan about trust. Under pressure, they can seek help earlier because they have already learned how colleagues approach a problem. That knowledge makes the next contact more direct and less speculative. The team does not need to assume intent from a terse message when it has a regular habit of learning how its members work.

How to do it

1. *Engage fully before responding.* Put the delivery artefact or decision in view, but do not use it as a place to hide while someone is speaking. Remove avoidable distractions. Let the person complete their account, especially when it contains uncertainty or bad news. You may be able to predict their conclusion; that does not mean you understand the constraint behind it.
2. *Ask questions that test understanding.* Use a summary followed by a check: "You are concerned that the partner date is fixed because of the launch. Have I got that right?" Ask what has changed, what is known, what is assumed, and what the consequence would be. Avoid questions designed only to expose a weakness in another position. The purpose is to make the issue more precise before the team chooses a response.
3. *Respect the perspective without pretending to agree.* Name the valid concern you have heard. A product lead can acknowledge a material control risk. A security lead can acknowledge the commercial consequence of delay. Respect does not require either person to abandon their professional judgement. It keeps disagreement attached to the work rather than to a judgement about the other person's motives.
4. *Show empathy for the pressure in the work.* Say what you understand about the experience, concern or pressure: a customer commitment, a risk that could become visible later, a dependency that leaves little room to move. Empathy is not agreement and it is not a promise to remove the pressure. It shows that the person does not have to exaggerate the problem to get it recognised.

5. *Use a rotating E.A.R.S. observer.* At major meetings, appoint one person who is not responsible for the main decision to notice listening behaviour. The observer records two examples of effective listening and one missed opportunity. They are watching the conversation, not scoring personalities. Rotate the role so that the team develops a shared view of what good listening sounds like. Reserve a few minutes at the end for the observer's record, otherwise the observation will be postponed when the agenda runs late.
6. *Create a regular team-building rhythm.* Put a small amount of time beside the delivery work at a regular interval. Use it to learn practical information about one another's strengths, current pressures and ways of working. A new member should be included deliberately. For hybrid teams, make the access equal: do not rely on information exchanged only by people who happen to be in the same room.
7. *Connect the ritual to the next piece of work.* At the end, ask what the team has learned that will help it seek help, make a decision or manage a dependency. This keeps team building grounded. If the activity reveals that someone has relevant experience, make that knowledge available when the next issue emerges. Trust grows when the team sees a practical result from the attention it gives one another. Keep that result in the relevant work record, not solely in a social note.

The exercise

Run a 55-minute E.A.R.S. practice session before using the observation task in live meetings. Prepare one current delivery issue with

two valid perspectives, such as a customer date and a material risk. Also prepare a short explanation of the E.A.R.S. behaviours and a simple observation sheet with spaces for two effective listening examples and one missed opportunity.

Spend 10 minutes introducing the issue and defining Engage fully, Ask, Respect and Show empathy. In the next 20 minutes, pair people to take each perspective. The listener must summarise the concern, ask clarifying questions, acknowledge the perspective and name the pressure behind it before giving a response. Ask the speaker to say whether the summary is accurate before the listener continues. Use 15 minutes for observers to share what they heard: not whether they liked the answer, but where E.A.R.S. was visible and where it was absent.

Use the last 10 minutes to agree the rotating E.A.R.S. observer for the next three major meetings and to choose one regular team-building ritual. After each meeting, record two examples of effective listening and one missed opportunity. At the end of the third meeting, capture one shared listening habit to maintain and one to improve. Keep the team-building ritual regular enough that practical knowledge of strengths does not depend on chance conversations. Capture the ritual's purpose and its next date, so it does not vanish when the delivery agenda becomes crowded.

What to measure

The *E.A.R.S Listening Observation* is the record from a rotating observer at the next three major meetings. For each meeting, the observer captures two examples of effective listening and one missed

opportunity. The artefact is a short behaviour record, not an assessment of who is naturally a good listener.

Read the three records at the end of the third meeting. Identify one shared listening habit to maintain and one to improve. Use examples from the record so the agreement describes a behaviour the team can recognise. Then use the next meetings to see whether the chosen habit is present in the work. The measure is most useful when it changes the team's next conversation, rather than becoming another meeting note.

One thing to do this week

In your next difficult delivery conversation, do not lead with your response. First summarise the other person's concern and ask them to confirm or correct you. Then name one pressure you can see in their position before explaining your own view. Write down the correction you receive. It is evidence of what you would otherwise have missed.

11. Conflict and feedback

Two engineers are reviewing a component that has failed twice during testing. One wants to rebuild it before the customer launch; the other wants to patch it, protect the date, and deal with the larger change later. Both have evidence, both can point to work the other has not considered, and neither trusts the other's preferred route. A programme lead has asked for an answer by the end of the day.

The argument does not begin with raised voices. It begins with increasingly careful messages that select only the facts supporting each position. The patch is described as reckless. The rebuild is described as academic. A decision could be forced by seniority, but that would leave the unchosen risk unowned and the losing engineer less likely to raise the next concern. The useful work is to turn a contest between preferences into a shared examination of what the customer launch and future stability require.

The principle

Healthy teams do not remove conflict. They make it useful. In complex delivery, disagreement is often evidence that people are seeing different parts of the system: customer impact, operational risk, technical debt, a dependency, cost, or the consequence of delay. A team that never disagrees may be withholding information, or it may be dealing only with work that has already been simplified for it. The risk begins when the disagreement turns personal, becomes hidden, or circles without producing a decision or a test.

L.E.A.N conflict resolution gives the team a route through. *Listening* means understanding the other view before replying to it. *Empathy* means recognising the underlying concern. *Adaption* means

changing the approach or the response in light of what has been learned. *Nurture of difference* means treating the difference as a source of stronger decisions. The spelling of Adaption is part of the named method. Its point is practical: listening and empathy have to alter the work if they are to be more than a courteous preface to the same argument.

Feedback needs the same discipline. The *Yin Yang Feedback* model balances *Yin* — reflection, receptivity and honest feedback — with *Yang* — action, innovation and adaptation. Reflection without action becomes a retrospective in which people describe the same frustrations every month. Action without reflection becomes repetition: the team moves quickly, but it cannot tell whether it is correcting the cause or simply moving the pressure to another part of the work.

The *Expression of Value Delta (EVD)* gives this feedback loop something tangible to inspect. Instead of saying, “I worked on the database” or “The task is 80% complete,” the team states what changed in value, risk, quality or learning. A database change might reduce average query time from 900ms to 540ms and move a customer-facing page-load target from 4.1 seconds to 3.4 seconds. If work did not produce movement, the update says what was learned and what changes next. The measure directs the conversation away from activity and towards consequence.

Google’s Project Aristotle describes psychological safety as confidence that teammates will not embarrass or punish one another for taking interpersonal risks. That condition allows conflict and feedback to contain the truth needed for learning. L.E.A.N, Yin Yang Feedback and the EVD do not guarantee agreement. They give a team

a way to disagree, reflect and act without leaving the value of the work hidden behind a percentage complete.

What it looks like when it is missing

The engineers defend their positions in separate messages. One repeats the test failures. The other repeats the date. The programme lead reads the exchange as a choice between delivery and quality, even though both engineers are worried about both.

A decision is made to patch because the deadline is nearest. Nobody records what failure rate would make that route unacceptable, what effort the patch actually needs, or what must happen after launch. In the next retrospective, the team says that communication was poor. The item is added to a list of lessons, but no action changes the way a similar conflict will be handled.

Status reporting makes the gap harder to see. The update says that the patch is 80% complete. It does not say whether customer risk has reduced, whether performance has changed, or what was learned from the second failure. The team has activity to report but no visible movement in value, risk, quality or learning. The unrecorded difference does not disappear. It returns as resentment or a late defect, but the team cannot see whether its decision process helped or merely ended the discussion. Feedback becomes a verdict on people rather than material for the next choice.

What it looks like when it works

The engineers begin by stating the shared concern: protect the customer launch and avoid future instability. They then use L.E.A.N. Each listens to the other route, identifies the concern underneath it,

and acknowledges what the other has seen. The rebuild advocate hears the date risk. The patch advocate hears the cost of carrying a known weakness into production.

They agree to a small experiment that measures failure rate and delivery effort for the patch route. The result provides evidence for adapting the approach rather than a reason to repeat a preference. The programme lead has a decision path and a visible risk, not a forced choice between two labels.

At the next retrospective, the team uses Yin Yang Feedback. It reflects on how the disagreement was handled, receives candid feedback about the point where evidence was missing, and agrees an action for the next similar decision. The update reports the effect of the chosen work or the learning obtained if movement has not yet occurred. The feedback has produced motion, and the action has been informed by reflection. The record gives a future reviewer a clear account of what the team knew, what it tested and what changed. It is a basis for learning, not a defence of the people involved.

How to do it

1. *Put the shared concern on the page first.* Before debating routes, ask what each person is trying to protect. In the component dispute, both customer launch and future stability belong in the statement. Write the evidence, constraints and uncertainty beside it. State what decision is needed and when it is needed, so the discussion does not drift into a general debate about the component. This reduces the temptation to describe one view as caring about delivery and the other as caring about quality.

2. *Listen to understand the other view.* Give each person enough time to set out their evidence and reasoning. Summarise it back before testing it. Listening does not suspend judgement; it stops the team from arguing with a weaker version of the other person's position. If the summary is wrong, correct it before moving to options.
3. *Name the underlying concern with empathy.* Identify the pressure beneath the stated recommendation: a customer commitment, the effect of a further failure, future instability, delivery effort or a dependency. Acknowledge that concern explicitly. This does not make all risks equal, but it stops the team from treating a real concern as a personal preference simply because it is inconvenient.
4. *Use Adaption to change the approach.* Ask what evidence, test or smaller piece of work would make the next decision better. The answer may be a patch experiment, an effort estimate, a failure-rate check or a staged route. Give the test an owner and a point at which the result will be read. A test that has no decision attached to it merely postpones the conflict. Adaptation is not compromise for its own sake. It is a response to what the team has learned through the disagreement.
5. *Nurture difference after the decision.* Record the view that was not chosen, the condition under which it should be revisited, and the owner of the next check. The difference may contain the warning that matters later. Treating it with care means the person who raised it can return with new evidence without being seen as reopening an old argument.

6. *Run Yin and Yang as one feedback loop.* Make room for Yin: reflection, receptivity and honest feedback on what occurred. Then insist on Yang: an action, innovation or adaptation that changes the next cycle. A retrospective should finish with a visible next action, an owner and a point at which the team will review it. A delivery action should be checked against what the team learned, not merely marked complete. If the action produces no movement, report that fact and decide what changes next.
7. *Report the EVD rather than activity alone.* For each meaningful update, state the work completed and the observable movement in value, risk, quality or learning. If the work moved nothing yet, say what was learned and what changes next. This creates a direct link between feedback and delivery. It also exposes when measures need to be defined before work begins. Keep this record visible to the people making the next decision.

The exercise

Run a 60-minute conflict-and-feedback session using one current disagreement or a recent, anonymised one. Prepare the available evidence, the two views, and a blank page for the shared concern, the next test or action, and the EVD update. The purpose is not to decide who argued better. It is to practise moving from conflict to learning and action.

Use 10 minutes for each view to state what it is trying to protect. Spend 15 minutes applying L.E.A.N: listen to the other view, name the underlying concern with empathy, identify an Adaption of the approach, and capture how the difference will be nurtured after the

decision. Write the condition under which the unchosen route must be revisited. In the next 15 minutes, use Yin Yang Feedback. Reflect candidly on the discussion, then agree the action or experiment that changes the next delivery cycle.

Use the final 20 minutes to agree how team updates will use the EVD for one delivery cycle. Capture the reported work, the observable value, risk, quality or learning change, and, where there is no movement, what was learned and what changes next. At the cycle's end, calculate the percentage of updates using complete EVD fields. Review whether at least 80% of reported work can be linked to an observable value, risk, quality or learning change. Where it cannot, capture which measures must be defined before work begins. Assign someone to bring that record to the cycle review and state the learning plainly.

What to measure

The *Expression of Value Delta (EVD)* is the record of movement caused by the work: an observable value, risk, quality or learning change, rather than activity or a percentage complete. For one delivery cycle, calculate the percentage of team updates using complete EVD fields.

Read that percentage at the cycle review and check whether the team can link at least 80% of its reported work to an observable value, risk, quality or learning change. Read a few individual updates as well as the percentage, especially those without movement. Do not use a missing link as a reason to embellish an update. Use it to

identify which measures need definition before the next work begins. Use examples honestly.

One thing to do this week

Choose one live disagreement. Ask each person to state the shared concern before they argue for a route. Then agree one piece of evidence, test or action that could alter the next decision, and make the next update describe the change it produces. At the next review, ask what the team learned from that change before deciding the following action.

Part Five — Accountability and Mindset

12. Roles, and modelling accountability

At the Friday delivery review, the programme plan was green until someone asked whether the new service could move into operational support. The delivery manager believed the service desk had accepted it. The service desk lead believed the product team was still finishing the knowledge articles. The product manager had asked an analyst to draft them, but the analyst thought the technical writer would publish them. None of those people had acted carelessly. Each had completed a sensible piece of work. The handover still failed because nobody could point to the outcome, the person leading it, the evidence required, or the moment it would be reviewed.

The meeting then did what such meetings often do. It searched for a name. People opened messages, recalled conversations and tried to establish who had promised what. That is slow, awkward and usually fruitless. A name in an email is not the same thing as a role that leads an outcome. The preceding work on clarity should have made the destination and expectations visible. Accountability makes that clarity usable when work changes hands and pressure rises.

The principle

Accountability is the team's shared certainty about who owns which outcomes, what evidence will demonstrate progress, how handoffs work, and how people will respond when conditions change. Blame produces defensive reporting and carefully narrow commitments; accountability makes work visible early enough for the team to act.

There are two levels. Individual accountability means knowing what you have committed to and following through. Team account-

ability means behaving like total football: covering, supporting, escalating and picking up when the outcome requires it. A person can fulfil an assigned task whilst the team still misses the outcome. The team can support a colleague without dissolving the clarity of who is leading the work.

A role is the contribution an outcome needs; a name is the person currently filling it. Confuse the two and your plan depends on memory. A named individual may move team, take leave, or be drawn into an urgent incident. The outcome still needs a lead, contributors, people to consult and people who need to know. The team also needs a shared answer to what complete means. That is role clarity: a working arrangement that can be inspected, not a job title copied from an organisation chart.

The cost of its absence is familiar. Two people prepare the same material. A risk lives between workstreams because each person has assumed it sits elsewhere. A task is closed because it was sent, although the receiving team cannot use it. Near the end of a release, an issue appears that was visible all along but owned by nobody as an outcome. More meetings will not repair this if the meetings merely circulate activity.

The D.R.E.A.M Outcome Contract (DOC) is the practical counter-measure. It is a short, visible commitment that states the expected outcome, the metric it should influence, the evidence that will prove the result and when that evidence will be reviewed. It changes the question from whether someone completed a task to whether the work created the expected change, and what has been learned when

it did not. A task may be necessary. It is not, on its own, the promise the team needs.

Google's Project Aristotle identified dependability and structure and clarity as important team dynamics. In that work, dependability includes completing quality work on time, while structure and clarity include understanding job expectations, how to fulfil them and the consequences of performance. A DOC turns those broad expectations into evidence that a team can read together.

The leader is part of the mechanism, not an observer of it. When a leader protects their own missed commitment with a vague explanation, the team learns to conceal uncertainty. When the leader shows the evidence, explains the changed condition or flawed assumption, states the learning and agrees the next action, the team has a standard worth copying. That example is especially powerful because it happens when the stakes are real. The next chapter takes this standard to the boundaries between people and the regular conversations that keep agreements alive.

What it looks like when it is missing

A portfolio has an objective to make a customer service available to operations. The delivery board lists several activities: finish testing, write support material, train the service desk and obtain approval. Each activity has a person's name beside it. No one has stated which role leads the operational outcome or what evidence proves that the service is usable by the receiving team.

Testing finishes and the tester reports completion. The technical writer sends a draft. The service desk manager receives an invitation

to training but cannot see the material. At the review, the programme lead asks for a status. Each person gives an accurate account of their own effort. The board hears four updates and still cannot establish whether operations can take the service.

A late defect then exposes an assumption about access rights. The discussion begins with who should have noticed it. People protect their area, because an admission may be treated as a personal failure. The issue becomes a sequence of explanations. No one records the assumption, the evidence that disproved it, or the action needed to recover. The operational outcome remains unowned. The plan stays green until it is suddenly not. The same gap can return in the next release under a different label.

What it looks like when it works

The same service has one visible outcome: operations can support it under agreed conditions. The team identifies the lead role, the contributors, the people who must be consulted and those who need to know. The person currently filling the lead role is named, but the board makes clear what the role must achieve if that person changes.

Its D.R.E.A.M Outcome Contract (DOC) links the outcome to a relevant measure and names the production evidence the team will inspect. The review date is on the board before work starts. Testing, support material, access and training remain pieces of work, but they are connected to the outcome rather than reported as separate destinations.

When the access assumption proves wrong, the lead brings the evidence to the review. They say what changed, what the team has

learned and what needs to happen next. The delivery lead does not turn this into a performance trial. The team gets help with the dependency, adjusts the next action and preserves the evidence with the DOC. Everyone can see the effect on the intended service. The review can focus on recovery, not reconstruction. The outcome may still be delayed, but the delay is explicit and the learning remains available.

How to do it

Start with critical outcomes, not the complete task list. Select commitments that affect a target, a key quality measure, a material risk or a handoff that could stop delivery. Make the outcome observable. "Prepare the support pack" describes effort. "Operations can support the service using agreed material and access" describes a condition the team can inspect.

1. *Identify the role required by the outcome.* Ask who is leading the outcome, who is contributing, who needs to be consulted and who needs to know. Then name the person filling each relevant role for this cycle. Keep the role and the name separate on the working board.
2. *Write a D.R.E.A.M Outcome Contract (DOC).* Keep it short enough to read during normal delivery work. Record the expected outcome, the metric it should influence, the evidence that will prove the result and the review point. If the entry only says that a document will be written or a meeting held, it is not yet an outcome contract.
3. *Test the contract against the actual work.* Ask what counts as complete, what assumptions must hold and what dependency

could prevent the result. If another team supplies an input, make the condition visible now. Do not make a person promise an outcome that depends on a decision they cannot obtain; make the dependency and required support visible instead.

4. *Make individual and team accountability explicit.* The individual filling the lead role owns the commitment to bring evidence and learning. The team owns the conditions around it: decisions, dependencies, support and escalation. This prevents the false choice between leaving someone alone with a difficult problem and having everyone vaguely responsible. One person leads; the group acts when the outcome requires it.
5. *Review evidence, not confidence.* At the stated review point, look at the metric and the agreed evidence before accepting a status. A contract can be achieved, partially achieved, disproved or blocked. Those classifications describe what happened without pretending every outcome was under complete control. A blocked result should show the condition that blocked it and the action needed from the team or outside it.
6. *Model the response you expect.* Leaders should put their own material commitments through the same discipline. If a commitment is missed, show the evidence. Explain the changed condition or the flawed assumption without hiding behind it. State the learning, agree the next action and record it where the team can find it. Do this before asking others for their status. It is a practical signal that bad news is information, while unexplained status is not acceptable.

7. *Keep the contract alive when conditions change.* A DOC is not a promise to ignore reality. A decision, supplier delay or new evidence can change the route. Update the contract openly, including the reason and the next review. Do not quietly replace an uncomfortable outcome with an easier task. The point is to retain the link between the intended change, the evidence and the learning. This discipline also keeps the preceding clarity work honest. Expectations that cannot be expressed as an outcome, a measure and evidence are still too loose for delivery. Once the team can see its commitments this way, it is ready to manage the boundary conditions around them rather than hoping that a name beside a task will be enough.

The exercise

Run a 55-minute working session before the next work cycle. Prepare a list of the major commitments already planned, the current targets and key quality measures, and any known handoffs. Bring the delivery board or a shared document that the team will continue to use. Ask the people who lead the relevant work to attend; this is not a status meeting for people who cannot shape the commitments.

Spend the first 10 minutes listing the major commitments in plain language. Remove activity labels that cannot be tested as an outcome. For each remaining item, take 15 minutes to identify the lead role, contributors, people to consult and people who need to know. Record the person currently filling each role, but preserve the role description beside the name.

Use the next 20 minutes to create a D.R.E.A.M Outcome Contract (DOC) for each priority commitment. For every DOC, capture the expected outcome, the metric it should influence, the evidence the team will inspect and the review point. Ask the group to challenge any contract that reports a task rather than a change. Where an assumption or dependency makes the result uncertain, record it rather than forcing a confident status.

Use the final 10 minutes for a leader-led review of one commitment. The leader should practise the response required when evidence is weaker than expected: show the evidence, explain the condition or assumption, state the learning and agree the next action. Capture the completed DOCs in the normal delivery board, together with the review dates. The output is not a separate workshop pack. It is the set of visible commitments the team will read during the cycle.

What to measure

The artefact is the *D.R.E.A.M Outcome Contract (DOC)*. Review all major commitments in the next work cycle and calculate the percentage expressed as complete DOCs. At each DOC review, classify the result as achieved, partially achieved, disproved or blocked. For every result other than achieved, retain the evidence and learning; never accept an unexplained status label.

Read the DOCs at their stated review points and inspect the percentage at the cycle review. The measure reveals whether critical work has an outcome contract, not whether people have been kept busy. It also makes missing evidence visible before the cycle ends.

This is an early warning for delivery leaders. A falling percentage points to commitments that are still being treated as tasks.

One thing to do this week

Choose one major commitment due in the next work cycle. With the person leading it, write a D.R.E.A.M Outcome Contract (DOC) on the delivery board: expected outcome, metric, evidence and review point. At the next team meeting, ask what condition could disprove it and what the team would need to do if that happened. Do not wait for a wider process change.

13. Check-ins, empowerment and agreements

On Thursday afternoon, a release manager learns that the integration test has failed against a changed data field. The software team says the change was sent to data engineering on Monday. Data engineering says the message asked for a review, not for a usable schema. The architect assumed the interface was still being explored. A customer demonstration is scheduled for the next morning. Everyone can produce a message that supports their own reading of the situation, but no message says what the receiver needed, when they needed it, or how either side would know the handoff was ready.

The immediate temptation is to make the release manager approve every future exchange. That adds a queue and leaves the real defect untouched. The problem is at the edge between contributions: boundaries were implied rather than agreed. The role work in the previous chapter establishes who leads an outcome. The next task is to make the conditions around that work visible, inspect them regularly and give people authority to act within limits they can see.

The principle

Accountability often fails between roles rather than inside them. A developer can reasonably believe that design is approved whilst an architect believes the developer is still exploring. A product manager can believe that customer validation is under way whilst the people closest to customers have not received the material. Such gaps are rarely laziness. They are the predictable result of a boundary that exists in somebody's head but not in the team's working agreement.

A *Quantified D.R.E.A.M Boundaries (QDB)* board maps people and roles to the outcomes that matter. The team places each target, key

quality measure or material risk on a visible board. It assigns a single primary owner, named contributors and measurable handoff conditions. It is not an expanded organisation chart. It is a current picture of the moments where one part of delivery needs another part to act.

The word *handoff* needs more discipline than it usually receives. Sending an item is an action by the sender. A complete handoff is a condition for the receiving role: it can use the item because the agreed information, quality and timing conditions are met. A link to a draft, a meeting invitation or a message saying “over to you” may be useful, but none proves that condition. The receiver must be able to tell quickly whether they have what they need to proceed.

Regular check-ins make the board a working agreement rather than a diagram produced once and forgotten. They provide a short, repeatable place to inspect upcoming boundaries, evidence from recent ones, decisions needed and changed conditions. They should not become a tour of individual activity. Their value lies in exposing a boundary before it becomes an urgent defect, and in agreeing the next action while the people who can act are present.

Authority close to the work follows clarity; it cannot safely precede it. A person needs to know the purpose, the boundary and the measures that matter before making a local decision. They also need to know which decisions are theirs, which must be consulted on and which require escalation. Without those conditions, a leader may call for initiative but still reserve every meaningful decision. That creates delay and teaches the team to wait.

Google’s Project Aristotle found structure and clarity to be an important team dynamic, including understanding job expectations,

the process for fulfilling them and the consequences of performance. QDB makes those expectations specific to a target, quality measure or risk, and makes the conditions of a handoff inspectable. The leader's task is not to remove every limit. It is to set clear limits, hold the agreed review rhythm and respond to evidence rather than preference.

These agreements are a bridge to the mindset work that follows. A team cannot take distributed responsibility merely because a leader asks it to. It needs practice in keeping small, observable agreements, revising them openly when evidence changes, and using a failed handoff to improve the boundary instead of blaming the sender.

What it looks like when it is missing

A release has a weekly meeting, but the agenda is a round of status updates. The software lead reports that a schema change is complete. Data engineering reports that data quality is being monitored. The architect reports that the interface design is progressing. These statements can all be true. No one asks whether the receiving team has the information, quality and timing conditions needed to use the change.

The board has a broad entry for data quality, with several names beside it. During the week, each group assumes another group will spot an incompatible field. An issue is found only when an integration test fails. The sender argues that the file was supplied. The receiver argues that it lacked the needed definition. The work is returned without a clear condition for the next attempt.

Decision-making then moves upwards. People send questions to the release manager because they cannot see what they may decide locally. The manager becomes the route for small choices and the check-in becomes a queue. When a handoff fails, the conversation concentrates on who delayed it. Missing information, a quality issue, a timing issue, an unclear owner and an untested assumption are all collapsed into the same accusation.

What it looks like when it works

The same release has a Quantified D.R.E.A.M Boundaries (QDB) board that shows the material target, quality measure and risk. The relevant boundary names one primary owner and the contributors, then states what a usable schema change requires. The data engineering lead owns the data-quality TVI. The software lead owns schema-change notice. The handoff is complete only when an automated contract test passes and the data dictionary is updated.

At a short check-in, the people involved read the condition rather than reconstructing intent from messages. The automated test has not passed, so the handoff is not presented as complete. The software lead can correct the change within the agreed boundary. The data engineering lead can assess the data-quality TVI. A decision that exceeds those limits is identified quickly and escalated with the evidence needed.

When the handoff later fails first time because an assumption was untested, the team records that cause. It adjusts the QDB condition and checks it at the next review. Nobody needs to pretend that sending the file was enough, and nobody waits for a manager to approve a

correction already within their authority. The next check-in confirms the correction before the demonstration. The agreement is visible, testable and revised by learning.

How to do it

Build the QDB around material edges in the next two work cycles, not around every conversation. Start where delivery has repeatedly slowed, where a quality problem has appeared late, where a decision waits too long, or where a risk crosses teams. Gather the targets, key quality measures and material risks already in use. The task is to make the connections between them usable in the rhythm of work.

1. *Put each material target, key quality measure and material risk on the QDB board.* Use language the delivery teams already recognise. A board full of broad labels cannot guide a decision. Make each item specific enough that people can see when it is affected and who needs to talk.
2. *Set the boundary for each item.* Assign a single primary owner and name the contributors. This is not a second role catalogue. It answers a narrower question: who brings the boundary and its evidence to the check-in, and who must contribute for the condition to be met? Where no one can take the primary owner position, the boundary is not ready to manage.
3. *Write measurable handoff conditions.* For each handoff, specify the information, quality and timing conditions the receiver needs. Make the wording receiver-centred: the receiving role can use the output because the agreed conditions are met. State the evidence that demonstrates each condition. An automated test, an updated

data dictionary, a signed decision or a reviewed item can be evidence where they are relevant. A vague assurance is not.

4. *Make decision authority visible.* For the primary owner and contributors, state the purpose of the boundary, the measure that matters and the decisions they may make to keep work moving. Also state what requires consultation or escalation. This is how authority moves nearer to the work without creating conflicting changes. The boundary gives people a safe operating limit; it does not require them to guess a leader's preference.
5. *Hold brief, regular check-ins at the working cadence.* Read the QDB board before the meeting so the conversation can focus on changes. Inspect boundaries that are due, evidence from those just completed, decision points and risks to the next handoff. Ask whether the receiver can use the output now. If not, identify which agreed condition is absent and who will act. End with recorded actions and any decision needing escalation.
6. *Treat agreements as observable commitments.* Keep the QDB where both sender and receiver can see it. When a condition changes, change it openly, name the reason and agree the new timing. Do not allow a private message to replace the shared agreement. This protects the team when people are absent and prevents an informal exception becoming the assumed process.
7. *Learn from failed handoffs.* For the next two work cycles, classify each failed material handoff as missing information, quality issue, timing issue, unclear owner or untested assumption. Discuss the classification at the check-in, then improve the QDB condition

rather than blame the sender. A failed condition may show a gap in the agreement, not a lack of effort.

The regular rhythm matters because delivery conditions move. A QDB that is precise on Monday may need revision after a new risk, a changed decision or evidence from a test. The aim is neither a large governance meeting nor unrestricted local action. It is a visible agreement that lets capable people act quickly within clear limits and raise the right issue before it becomes a release surprise.

The exercise

Run a 60-minute QDB working session for the next two work cycles. Prepare the current delivery targets, key quality measures, material risks and a list of the handoffs most likely to affect them. Bring evidence from any recent late defect, returned item or delayed decision. Include people from both sides of each selected handoff; a boundary cannot be agreed by the sender alone.

Use the first 10 minutes to select no more than three material boundaries. Put each associated target, key quality measure or risk on the shared board. In the next 15 minutes, assign the single primary owner and named contributors for each boundary. Confirm that the primary owner can bring evidence to the regular check-in and can obtain the required contribution.

Spend 20 minutes writing the measurable handoff conditions. For each one, record the information, quality and timing the receiving role requires, plus the evidence that proves the condition. Ask the receiver to read the condition back in their own words. If they cannot tell whether they can use the output, make the condition more precise.

Use the next 10 minutes to agree decision authority inside each boundary: what the people closest to the work may decide, what needs consultation and what needs escalation. Use the final five minutes to set the check-in cadence and capture the agreed QDB board in the team's normal workspace. At the first check-in, review one completed or failed handoff using the agreed conditions. Keep the record as a living working agreement, not a workshop output.

What to measure

The artefact is *Quantified D.R.E.A.M Boundaries (QDB)*. For the next two work cycles, track the percentage of material handoffs that meet their agreed QDB condition first time. Read the measure at the regular check-in and use the cycle review to identify patterns.

Record both the total material handoffs and those meeting the condition first time, so the percentage can be checked rather than assumed at each cycle review. When a handoff fails, classify the cause as missing information, quality issue, timing issue, unclear owner or untested assumption. Keep the classification with the boundary evidence. The purpose is to improve the condition, not to create a league table of senders. Repeated failure in one class tells the team where the agreement needs attention.

One thing to do this week

Pick one handoff due before the next delivery check-in. With the sender and receiver, write its information, quality and timing conditions on the Quantified D.R.E.A.M Boundaries (QDB) board. Agree what evidence will show that the receiver can use it, and what deci-

**sion the people involved can make without waiting for a manager.
Read that condition at the check-in. Keep it visible to both parties.**

14. Being, not doing

At half past four on a Thursday, the delivery lead is looking at a release plan that has no room left in it. A new customer dashboard has been marked amber for weeks. The design is approved, the backlog is detailed, and the status report says the team is on track. In the meeting, a developer says the assumed data is not yet available in the form the dashboard needs. The product manager replies that the launch date has already been communicated. The architect suggests an interface change, but nobody writes it down because it would reopen the design.

The room moves on to actions. The dashboard remains amber. A week later, the data issue becomes a build issue, then a test issue, then a date issue. By then the team has committed too much work to ask the basic question that was available on Thursday: will the thing we plan to build produce the change we expect? This is not a failure of effort. It is a team treating certainty as safer than learning.

The principle

MINDSET is a team's shared way of interpreting challenge, uncertainty, failure, difference and opportunity. In delivery, it shows up before anyone calls it culture. It is there in the reaction to a defect that nobody expected, an estimate that no longer holds, a customer finding that contradicts a requirement, or a junior colleague who spots a weakness in a familiar design.

A growth mindset is not positive thinking and it is not a licence to congratulate people for poor work. It is the practical belief that ability, knowledge and workable approaches can develop through effort, feedback and learning. A team with this habit can say that an

assumption was wrong without turning that statement into a verdict on the person who made it. It can distinguish between an experiment that did not support its premise and a delivery failure caused by unmanaged work.

That distinction matters because complex work contains assumptions whether you record them or not. A plan can look complete while its most consequential belief has never met customer evidence, operational data or a technical test. When the belief proves false late, people often call it an unforeseen issue. In many cases it was seen early but not given a safe, disciplined route to be tested.

The delivery implication is practical. If people expect blame, they will present a confident answer before they have evidence. If they expect curiosity and clear standards, they can expose the uncertain part while it is still cheap to change. A team can be demanding about preparation and still make it safe to say that it does not yet know.

Innovation needs the same discipline. New ideas should not be accepted because their sponsor is persuasive, nor rejected because the team is busy. They need a small test, a measure and a decision rule. *M.I.N.D.S.E.T Value Engineering (MVE)* gives a team a way to make that normal work. It does not make uncertainty disappear. It makes uncertainty visible early enough to act on.

This is a practice rather than a poster on a wall. A stated value about learning changes nothing if the weekly meeting rewards only assurance, if a failed test is hidden in a work item, or if people must secure permission from several layers before trying a small check. The cost is not merely slower invention. It is late rework, false confi-

dence and people learning to protect themselves instead of helping the team see clearly.

What it looks like when it is missing

The dashboard team has a detailed delivery plan but no recorded premise about the data. The analyst knows the source is incomplete. The developer has found a workaround that might affect response time. The product manager is trying to protect a date. Each person has useful information, but it remains attached to their role rather than becoming a team decision.

The status meeting asks whether work is green, amber or red. It does not ask what has been tested or what would change the plan. New suggestions arrive as extra scope, so they are deferred until after release. A test that produces an unwelcome result is described as a problem to fix rather than evidence to examine. The team builds more of the dashboard because visible progress feels safer than checking the premise.

When the issue finally reaches governance, the discussion starts with accountability for the delay. That is necessary, but it comes before learning. People become careful with language. The next risk is then less likely to be raised. The plan gets longer and the team's understanding gets thinner.

What it looks like when it works

In the same meeting, the delivery lead pauses the plan when the analyst raises the data concern. The team separates what it knows from what it assumes. It agrees that the question is not whether the

full dashboard is attractive, but whether the available data can support a useful customer view within the intended service conditions.

The developer and analyst propose a narrow technical and customer test. The product manager agrees what result would justify continuing and what result would require a change of direction. The architect is present early enough to see the consequence of either outcome. Nobody is asked to guarantee success. Each person is asked to help produce evidence.

The first test may support the idea, qualify it or show that it should stop. All are useful outcomes if they occur before a large build. The team records the decision and updates the work rather than preserving the original plan for appearances. It has not become casual about delivery. It has become more precise about what it is trying to learn, and why.

How to do it

1. Start with the initiative that carries real uncertainty, not a harmless improvement that is certain to succeed. Ask what must be true for the intended customer or operational result to happen. Keep the answer specific. A premise about data availability, customer behaviour, technical performance or an external dependency is more useful than a general wish to improve the service.
2. Put the premise where the whole delivery group can see it. Separate evidence already available from inference. This prevents a confident statement in a steering pack becoming accepted fact merely because it has been repeated. It also makes it easier for a tester, operations colleague or subject specialist to add a miss-

ing constraint without having to challenge the status of a senior person.

3. Write the MVE hypothesis in its full form: MVE hypothesis: If we do [action], then [measure] will change from [baseline] to [target] within [timeframe], because [reason]. Do not leave any bracket vague. The action must be something the team can carry out. The measure must be observable. The baseline and target make the expected movement clear. The timeframe stops a test from becoming an indefinite piece of work. The reason states the causal belief that is being examined.
4. Design the smallest viable test. That might be a prototype, a limited service path, a technical spike, a customer conversation structured around a concrete choice, or a controlled use of existing data. The test must be small in commitment, not careless in design. Decide who needs to be involved, what must be protected, what evidence will be collected and when the team will review it. A small test still needs appropriate security, operational and customer safeguards.
5. Agree the decision rule before the result is known. State what evidence would support continuing, what would lead to an adaptation and what would mean stopping. This is the discipline that separates learning from retrospective rationalisation. Without it, teams can spend weeks explaining why any result still supports the original idea.
6. Run the test and classify the hypothesis as supported, partly supported, disproved or inconclusive. These are not grades for individuals. They are a shared description of what the evidence

permits the team to say. An inconclusive result may require a better measure or a different test. A disproved result may be the most valuable outcome if it prevents a larger commitment.

7. Record the decision taken because of the evidence and the time or cost avoided through early learning. Do not invent savings to make the record look useful. If the avoidance cannot yet be estimated, state the decision and the work that did not proceed. The value of the record is that a sponsor can trace the change from premise to evidence to delivery choice.
8. Change the meeting habit around the work. In planning, ask which premise has the largest consequence if it is wrong. In delivery reviews, ask what the latest evidence changed. In retrospectives, ask whether the team made it safe to challenge an assumption early. A leader sets the tone by treating a revised view as useful information, while still holding the team to clear preparation and follow-through.

MVE is particularly useful when a team has already written a large backlog. It gives people permission to reduce a broad commitment to the next question that matters. The work item is no longer simply to build a feature. It is to test whether a particular action will produce a stated change. That is a more demanding standard, but it is also a more honest one.

The exercise

Run this as a fifty-five-minute session before significant build begins on an active initiative. Prepare one-page summaries of the intended outcome, the current evidence, the relevant constraints and

the work that the team expects to start. Bring the people who can speak for product, delivery, technology, operations and the customer context. The purpose is not to create a long list of ideas. It is to select one assumption worth testing.

Start with five minutes to state the outcome and the delivery decision that is approaching. For the next ten minutes, ask each person to write the assumptions they believe must hold. Group similar assumptions and identify the one whose failure would most alter the plan. Keep the discussion on the current initiative, rather than turning it into a general retrospective.

Use fifteen minutes to draft the MVE hypothesis together. Check every part of the statement: action, measure, baseline, target, time-frame and reason. If the team cannot name a measure or baseline, capture that as a gap rather than forcing a precise-looking claim. The next ten minutes are for designing the smallest viable test and identifying the evidence source, owner and review point.

Spend the next ten minutes challenging the test. Ask whether it is safe, small enough, connected to the premise and capable of changing a decision. Agree the decision rule before anyone starts the work. Use the final five minutes to capture the MVE card, the test date, the people involved and the decision that will be revisited. At the end of the first test, classify the hypothesis as supported, partly supported, disproved or inconclusive, then record the decision taken because of the evidence and the time or cost avoided through early learning.

What to measure

The artefact is an *MVE card* for the next major initiative, created before any significant build begins. It contains the MVE hypothesis, the smallest viable test, the decision rule, the evidence collected and the final classification. It should also record the delivery decision and any time or cost avoided through early learning, without manufactured precision.

Read the card at the review point agreed for the test and again in the next delivery planning session. A delivery lead should look across cards at the end of each work cycle for a simple pattern: are important assumptions being tested before large commitments, and are results changing what the team does? The measure is the quality of that link, not the number of cards completed.

One thing to do this week

Choose one item already planned for build and write a single MVE hypothesis with the people closest to it. Do not wait for a new initiative or a formal workshop. Identify the assumption that would most damage the outcome if it proved wrong, agree the smallest viable test and put the review date into the team's existing delivery rhythm. The useful result this week is a clear question with a route to evidence.

15. Distributed ownership

The release planning board is crowded with work that appears to belong to other teams. The product manager has a customer commitment. The delivery manager has a date. A security review is marked as an external dependency, as is a data-provider response and a change to the contact-centre script. Each line has an owner, but the owners are not in the room. The group spends most of the meeting arranging handoffs and deciding who will chase whom.

Near the end, an operations lead points out that a planned service change will create a support pattern nobody has designed for. The point is acknowledged and copied into an action log. It is not connected to the customer commitment, the data dependency or the release decision. By the next planning session, the product work has moved forward and the operational concern has not. Everybody has done their own job. The outcome has no owner who can see the whole path.

The principle

Complex delivery crosses specialisms. Product, technology, data, operations, security, legal, customer service and suppliers all hold a part of the answer. The familiar response is to make each boundary a handoff: one team completes its task, sends something on and waits for the next group to respond. That can look orderly in a plan. It often hides the important work until it is late, when the cost of changing direction is highest.

Distributed shared ownership is the team's answer to that problem. It does not mean that everyone does everything, or that specialist accountability becomes blurred. A security professional still owns

security expertise. A product manager still makes product decisions. A developer still has responsibility for sound engineering. The difference is that the project outcome belongs to the delivery group, not to the function with the most visible task.

This changes the standard for participation. A developer can raise a customer-experience risk. A product manager can bring forward a security dependency. An architect can help the delivery team make an early decision about an interface. An operations lead can challenge a release that would leave support unprepared. None needs to wait until a task is formally assigned before making a relevant risk visible.

Interdependence needs to be visible in planning, problem solving, decisions and progress reviews. That is more exacting than collaboration as a calendar invitation. If people only meet at formal handoffs, they do not have shared context when a trade-off is needed. The work may be connected on a plan, but the people making choices are not connected in time.

Risk-First Learning Loops (RLL) provide the working discipline. The team reviews its *Target Risk Burn-Down (TRB)*, identifies high-impact, high-probability risks and tackles them before polishing low-risk work. This does not mean ignoring important planned delivery. It means putting learning where the consequence of being wrong is greatest. A risk discovered in an early test is still work. A risk discovered during a launch is often disruption for customers and staff as well.

The cost of absent shared ownership is rarely a dramatic refusal to help. More often it is polite separation. People deliver their local

output, report their local status and assume the project manager will join the dots. The project manager becomes a traffic controller, overloaded with dependencies that people closest to the work could have exposed and examined together.

What it looks like when it is missing

The team plans the customer interface first because it is visible. The third-party data integration is placed later because another group owns the supplier relationship. The security colleague receives a request when a design is already fixed. Operations sees the service only when a release note arrives. Each function gives a reasonable answer within its own remit, but the answers do not meet until the date is close.

The TRB exists as a governance document. It is updated for reporting and read before a formal review, not used to choose work. The most serious risks remain on a list while the team finishes lower-risk interface details. A person who notices a cross-functional concern has to decide whether it is their place to raise it. Often they wait for a meeting with the nominal owner.

When the data provider cannot meet the required response time, the integration becomes urgent. The team now has to redesign around a dependency while preserving the work already built. The event is described as a late surprise, though the dependency was visible from the outset. Ownership was delegated in name, but the outcome was not held in common.

What it looks like when it works

At planning, the team puts the customer commitment, data dependency, security requirement and support change on the same board. The product manager describes the intended customer outcome. The data specialist explains the uncertainty in the provider response. Operations explains what a change in service behaviour would mean for staff. The architect and developer identify an early path through the integration.

The group reviews the TRB together and sees that the provider response-time risk has both high impact and high probability. It moves a data test into the first iteration, before the customer interface is polished. The supplier contact is invited to the right discussion. Security and operations help define the limits of the test rather than receiving a finished design later.

The provider limitation appears quickly. The team redesigns around cached data and adjusts the work before a large build depends on the original premise. The product manager owns the customer decision, the technical people own the design choices and operations owns readiness, but all understand why the change is necessary. The group has used its connections to protect the outcome, not merely to pass work around.

How to do it

1. Name the outcome that the group owns together. Make it concrete enough that product, technology and operations can each describe their contribution to the same result. Do not begin with a list of departmental deliverables. That list may be necessary for planning, but it does not create a shared view of success or

expose the points where one local decision damages another part of the work.

2. Map the people and functions that have information, authority or work needed for that outcome. Include the less visible functions early: service operations, security, legal, data, procurement, customer support and suppliers where they affect the path. This is networking with a delivery purpose. The aim is not to enlarge every meeting. It is to make the relevant connections before an issue becomes a formal escalation.
3. Bring the connected group to the TRB. The Target Risk Burn-Down identifies assumptions, dependencies and obstacles most likely to prevent the target from being achieved. For each current risk, make the impact and probability visible and ask who has evidence that could change the assessment. A risk held by one function becomes useful to the team only when the consequence and next learning step are understood across the relevant roles.
4. Use RLL to select the work that should happen first. Identify the high-impact, high-probability risks, then design an experiment or investigation that reduces uncertainty. Resist the pull of visible but low-risk polish. A refined screen, completed document or finished component can create a reassuring progress report while the major dependency remains untested.
5. Give the learning activity an owner and give the outcome multiple active participants. One person may coordinate the test, but product may need to define the customer consequence, technical colleagues may need to establish feasibility and operations may need to set service conditions. State who will decide after the

evidence arrives. This preserves clear accountability without pretending that a cross-functional answer can be produced by one role alone.

6. **Make connections routine.** Use a short cross-functional review in the existing work cycle to inspect the leading risks, new dependencies and changed assumptions. Ask people to bring a concern before they have a fully formed answer. The useful contribution may be a question, a data point or a constraint. If the forum only accepts finished proposals, it will recreate the handoff behaviour it is meant to replace.
7. **Make ownership visible in the actions that follow.** A record should show the risk, the people needed, the learning activity, the decision point and the consequence for the shared outcome. Avoid a long action list with no indication of why each item matters. When a decision changes the plan, tell all affected functions promptly so that a local team is not working to a superseded assumption.
8. **Review whether the group is learning in the right order.** RLL is not a demand that every activity concern risk. It is a check on priority. If high-risk learning repeatedly loses out to lower-risk feature polish, the team has a planning problem, not a motivation problem. Bring that pattern to the delivery lead and sponsor as evidence that the sequence needs to change.

Distributed shared ownership becomes credible when it is visible in decisions. It is not created by adding the word shared to a responsibility matrix. People need access to the context, an invitation to surface what they see and a method for deciding which uncertainty

deserves attention. The more functions involved, the more valuable those habits become.

The exercise

Run a fifty-five-minute RLL session at the start of an active work cycle with the people connected to the outcome. Prepare the current TRB, the planned work for the next two work cycles and any known customer, technical, operational or supplier constraints. Ask participants to arrive ready to explain a risk or dependency from their own part of the work. The session is a prioritisation discussion, not a general project update.

Use the first five minutes to restate the shared outcome and the decision the team is trying to protect. Spend ten minutes reviewing the TRB, checking that the highest risks still have a current impact and probability assessment. For the next ten minutes, identify the top third of TRB risks and ask what evidence would reduce uncertainty on each one. Keep the focus on risks that are both consequential and plausible.

Use fifteen minutes to design planned experiments for the next two work cycles. For each one, capture the risk, the relevant functions, the smallest learning activity, the evidence expected and the decision that could follow. Do not assign the risk away to one group. Make clear who needs to participate or supply information so that the learning does not become another handoff.

Reserve ten minutes to compare these experiments with lower-risk feature polish already planned. Move work where needed so high-risk learning gets priority. In the final five minutes, record the

percentage of planned experiments focused on the top third of TRB risks. The target is not complete coverage in every cycle. The point is evidence that consequential learning receives priority before lower-risk feature polish.

What to measure

The artefact is the *Risk-First Learning Loops (RLL)* record, read alongside the current TRB. For the next two work cycles, record the percentage of planned experiments focused on the top third of TRB risks. Keep the planned experiment, the risk it addresses and the evidence expected together, so the percentage does not become a reporting exercise detached from delivery.

Read the record at the start and end of each work cycle. At the end, check whether the planned experiments actually examined the risks that were prioritised and whether the results changed the TRB or work sequence. The measure is not a demand for complete coverage every time. It is evidence that high-risk learning received priority before lower-risk feature polish.

One thing to do this week

Bring the current TRB to the next planning meeting and ask the people in the room to identify the top third of risks together. Select one high-impact, high-probability risk that has been left behind a handoff, then agree the smallest learning activity and the functions needed to run it. Put that activity ahead of one piece of low-risk polish already on the board.

16. The whole is more than the parts

The service has been restored after a difficult release, but the release review is already becoming a familiar story. The delivery manager thanks the engineer who stayed late. The engineer credits the team. A slide lists completed actions: patch applied, monitoring adjusted, customer message sent. The analyst who spotted the pattern in the support data is not mentioned. Neither is the support colleague who explained the customer impact, nor the tester who had raised a related edge case earlier in the week.

The meeting closes with a request to be more careful next time. People return to their own queues. The useful knowledge is scattered across incident notes, chat messages and individual memory. A new team member could read the action list and still fail to understand how the release was recovered, which capabilities were needed, or what should be practised before the next change. The team has handled the event, but it has not yet turned the event into collective capability.

The principle

A team can contain strong people and still leave much of its capability unused. The work may be divided sensibly, but people may not know what colleagues can contribute beyond their formal role. They may not understand how one person's customer evidence changes a technical decision, how a tester's pattern recognition changes an operational check, or how a support colleague's practical knowledge can prevent a design mistake. This is where the team becomes more capable as a group, rather than merely a set of capable individuals.

Synergy and Learning Signal means collective capability is greater because people understand, trust and use one another's distinct

strengths. It is not a vague claim that everyone gets on. It is visible in the way information travels, in how a team responds when one part of the service changes, and in whether people can call on knowledge outside their own speciality before a problem becomes expensive.

Education and learning are therefore delivery work. Lunch-and-learn sessions, peer demos, communities of practice, cross-training and reflection on failures create routes by which useful knowledge moves. They should not be treated as optional activity for quiet periods. A team that cannot explain its service to one another, share a new constraint or learn from a difficult release will repeat avoidable work even when individuals are conscientious.

Recognition also has a practical role. *Team Tributes* make a strength visible by naming a specific capability and showing how it advanced a shared outcome. They are not general praise, social reward or a substitute for addressing poor performance. A useful tribute makes a contribution legible to the group: what someone noticed, did or enabled; what it affected; and why that contribution mattered to value, quality, risk or learning.

The question of whether a person's unique skills and talents are valued and used is not answered by an annual recognition scheme. It is answered in the daily allocation of work, in who is invited to solve a problem and in whether the team can see contribution types that are less visible than a presentation, a senior role or a late-night recovery.

This book's claim is a practical one rather than a guaranteed result: teams that deliberately share learning and recognise specific contributions give themselves more usable options when the next difficult

decision arrives. The alternative is to depend on a small set of people and to discover missing knowledge only after it is needed.

What it looks like when it is missing

After the difficult release, the incident record describes the technical fix. The support evidence sits in a separate system. The analyst's work is visible only to the manager who asked for it. The tester's earlier concern is treated as an item that was overtaken by events. Nobody connects these contributions into an account of how the team understood the problem.

Learning is postponed until there is time. A useful demonstration is cancelled because delivery is busy. A new operational constraint is explained once in a specialist meeting but never reaches the people designing the next change. The same questions recur, and newer colleagues learn by interrupting the few people who have seen the situation before.

Recognition follows visibility. The loudest role, the person who presents to leaders or the person who stays late receives thanks. Quieter work that prevented harm, produced evidence or enabled someone else's decision is not named. People learn what is noticed, then orient their effort towards being seen rather than towards the shared outcome.

What it looks like when it works

The review starts with the customer and service outcome, then traces the chain of contributions that changed it. The analyst shows how support data exposed the pattern. The support colleague explains the practical effect on customers. The tester describes the

earlier edge case. The engineer explains the diagnostic change and the recovery. People see the linked work without reducing it to a hero story.

The team captures one lesson that must change the next release and chooses a short peer demonstration to spread it. A colleague from another function attends because the lesson affects their part of the service. The discussion makes a new connection: the support pattern can become an early release check, and the diagnostic information can be available before customers report the issue.

Team Tributes name the distinct contributions with evidence. They connect each contribution to a value, quality, risk or learning outcome. The people involved do not need to become generalists. They become easier to work with because others understand what they know and when to involve them. The next release begins with more shared context than the last one.

How to do it

1. Treat each significant piece of delivery work as a source of learning, not only failures. Look at releases, customer findings, avoided risks, difficult decisions and changes that went well. Ask what the team now knows that it did not know before, and who else needs that knowledge. Keep the learning tied to a real service or outcome. A generic presentation about good practice will travel less well than a concrete account of a decision and its evidence.
2. Choose the learning route to fit the need. A lunch-and-learn session can introduce a topic across a wider group. A peer demo can show a concrete technique or service behaviour. A community of

practice can connect people facing recurring questions. Cross-training can reduce a single point of dependence. Reflection on a failure can expose an assumption that needs to change. Do not schedule all of them. Select the one that will help the next piece of work.

3. Prepare learning material from delivery artefacts people already use: a release record, a customer finding, a risk, a design decision, a test result or an operational observation. State the context, what the team expected, what it observed, the decision it made and what it will do differently. Remove sensitive details where required, but do not sand away the practical constraint that made the learning useful.
4. Create a short, regular slot for the chosen activity and protect it from being continually displaced by routine reporting. The time does not need to be long. It needs a named purpose and an output that will be used. For a peer demo, the output may be a changed check. For cross-training, it may be another person able to carry out a task. For a reflection, it may be an updated assumption or risk.
5. Use Team Tributes at the end of a work cycle or after a material event. Name the contribution precisely. Describe the capability, the evidence of what it enabled and the link to a value, quality, risk or learning outcome. A tribute such as thanks for working hard is too vague to teach the team anything. A tribute that identifies how a colleague's customer evidence changed a release decision tells others when that strength matters.

6. Make the recognition broad enough to reveal the real chain of work. Ask whose contribution is not normally visible in the main delivery meeting. Look across product, technical, operational, analytical, assurance and customer-facing roles. This does not mean distributing praise mechanically. It means checking whether the team's account of success reflects the interdependent work that actually occurred.
7. Turn recognition into future capability. When a tribute identifies a useful strength, ask how the team can use it again: invite that person into an earlier discussion, pair them with another role, record their method in a peer demo, or add their insight to a release check. Recognition becomes empty if it ends with applause. It becomes useful when it changes access to knowledge.
8. Review the pattern over time. If one type of role is always recognised and another is absent, investigate the work system rather than assuming some people contribute less. It may show that evidence is not travelling, that meetings exclude key voices or that the team lacks a way to see enabling work. The aim is not an even distribution of mentions. It is a more accurate picture of the capability on which delivery depends.

Education and Team Tributes work together. Education makes knowledge available; Team Tributes show which knowledge and behaviours made a difference in real work. Both need evidence, a connection to outcomes and a place in the delivery rhythm. Used this way, they help a team retain learning after the urgent moment has passed.

The exercise

Run this as a fifty-five-minute end-of-cycle learning and recognition session after a release, a material decision or a completed work cycle. Prepare the delivery artefacts that show the outcome: relevant risk or quality records, customer evidence, release notes, test results and any decision record. Invite people from the functions that contributed, including those whose work is usually visible only through a handoff. The purpose is to understand the contribution chain and make future learning usable.

Use the first five minutes to state the outcome and the evidence available. Spend ten minutes mapping the sequence of contributions: what was observed, who supplied information, what decision changed and what action followed. Keep the map factual. The group is not there to decide who deserves general praise.

Use the next ten minutes to identify one lesson that should alter future work. Choose the appropriate learning route: a lunch-and-learn session, peer demo, community of practice, cross-training or reflection on a failure. In fifteen minutes, record at least three evidence-based Team Tributes linked to a value, quality, risk or learning outcome. Each should name the specific capability and how it advanced the shared outcome.

Spend ten minutes deciding how the identified strengths and lesson will be used in the next work cycle. This might mean an earlier invitation to a decision, a peer demonstration or a changed delivery check. Use the final five minutes to confirm where the learning record and Team Tributes will be kept. At the end of each work cycle,

review the record; quarterly, review whether recognition reaches all functions and contribution types, not only the most visible roles.

What to measure

The artefact is a cycle-end record containing evidence-based Team Tributes and the learning action they produced. It records at least three tributes linked to a value, quality, risk or learning outcome, alongside the relevant delivery evidence. It should also show the education or learning activity selected and where the team will use the lesson.

Read it at the end of each work cycle, when the contributions and evidence are still clear. Review it quarterly to see whether recognition reaches all functions and contribution types, not only the most visible roles, and whether learning is changing future work. The measure is not the amount of praise. It is whether specific strengths are visible, valued and used.

One thing to do this week

At the next team review, take one completed piece of work and ask the group to name three specific contributions that changed the result. Link each to a value, quality, risk or learning outcome, then choose one strength the team will involve earlier next time. Capture the decision with the existing delivery record so the recognition also becomes a practical reminder of how the work was done.

Part Six — Insight and Quality

17. Deciding where the information is

At 16:20 on a Tuesday, a service team is preparing a change that will alter the way customers reset their passwords. The contact centre has seen a rise in calls after a recent release. A developer has found a narrow change that can be deployed before the evening peak; the service manager knows the likely effect on call handling; the product owner has the latest customer feedback. All three are in the room. The delivery lead says the change must go to the weekly steering group because that is where release decisions are normally confirmed. The steering group meets on Thursday.

By Thursday, the call pattern has moved, the developer is on another item, and the decision is no longer the same decision. The group receives a tidy summary of what was known two days earlier and approves a general direction. The people who could have acted when the evidence was freshest have learned a familiar lesson: notice the problem, assemble the paper, then wait.

The principle

INSIGHT places a sound, timely, evidence-aware decision at the point where information is richest. That does not mean removing leadership or treating every choice as local. It means deciding deliberately which decisions belong close to the customer, system, supplier, or delivery constraint that creates the evidence, and which need a wider authority because the consequence reaches beyond that setting.

Centralised decision making has a legitimate role where a choice changes funding, policy, legal position, customer commitments, or

shared risk. The mistake is to use it as a holding area for routine judgement.

The cost is more than decision latency. People closest to the work become good at escalating and poor at deciding. They may stop bringing forward a risk until they have a fully formed case that will survive a senior meeting. They may follow an instruction that no longer fits the situation because changing it would mean another approval route. Senior leaders then spend time making choices for which they have the least current information. Control becomes an appearance of control: a record that someone senior agreed, rather than a record that the best available judgement was made in time.

Purpose-based leadership is the counterweight to uncontrolled local choice. The leader states the outcome to protect and the boundary within which the team may act. A direction such as reducing customer waiting time while protecting service quality and cost gives the team a reason to compare options. It does not prescribe a single action before the team has seen the evidence. The team can then explain its choice through purpose, evidence, and guardrails rather than through rank.

Google's Project Aristotle identified structure and clarity, meaning, impact, and psychological safety among the dynamics of effective teams. Those conditions support local judgement. People need to know what matters, understand their boundary, see the effect of their work, and be able to say that a concern changes the decision. TEAM IQ turns those conditions into a decision practice; it does not claim that a slogan about autonomy is enough.

D.I.R.E.C.T Value Routing (DVR) provides the practical discipline. It uses the VER, current evidence, and confidence to direct effort towards the highest-value next move. The purpose supplies the direction. The guardrails define the edge of the decision. DVR stops decentralised authority becoming a licence for personal preference, because a team must be able to show why this next action is preferable now.

What it looks like when it is missing

The password-reset change becomes a chain of requests. The developer asks the delivery lead for approval. The delivery lead asks the product owner for a business case. The product owner asks the service manager for updated call information. Each person prepares a version of the same issue for the next meeting. Nobody has said whether the team is allowed to make a small change that stays within the release boundary.

The steering group finally receives a choice between a short summary and doing nothing. It does not see the discarded options, the details of the service constraint, or the judgement made by the people who investigated it. Members ask for more assurance, partly because the material cannot answer the questions they would ask if they were in the work. The decision returns to the team with a request for a revised paper.

In this arrangement, authority follows the organisation chart rather than the information. A low-risk, reversible action waits beside a decision that genuinely needs senior sponsorship. The team starts to protect itself with email trails. The service manager stops raising

small improvements because each one creates a further round of approval. When the incident is later reviewed, the discussion centres on who approved what, not on whether the right person could act when the evidence was available.

What it looks like when it works

The same team has agreed that it may change guidance and the password-reset route when the work remains within the stated service-quality and cost guardrails. A change that alters customer commitments, creates a material risk, or requires funding outside the agreed work goes upwards. The boundary is visible before the issue appears, so the team is not negotiating authority during the incident.

The developer brings the proposed guide, the service manager brings the current contact pattern, and the product owner brings the customer effect. They use DVR to compare the guide with the larger automation idea and with leaving the route unchanged. The guide is the highest-value next move on the evidence available. The team records why, names the uncertainty, and sets a point to look again at the contact pattern.

The delivery lead does not disappear. Their role is to test whether the purpose and guardrails are clear, to remove a dependency the team cannot remove, and to take a decision upwards when it crosses the agreed boundary. At the next steering group, the team can explain what it decided and what happened. Leaders receive a decision with its reasoning, rather than a request to recreate the reasoning from a distance.

How to do it

1. *State the purpose before discussing the option.* Write the outcome that must improve and what must remain protected. Keep it close to the work: customer waiting time, service quality, cost, a delivery commitment, or a stated risk. A purpose that only says to complete the planned item gives no help when new evidence suggests a different course. The useful version lets a team compare an action with the outcome it is trying to create.
2. *Set the decision boundary in advance.* Identify which choices the team can make, which choices require consultation, and which require a decision at a wider level. Use the real consequences, not job titles, to draw the line. A choice with a local, reversible effect can sit with the people who have the evidence. A choice that changes a customer promise, creates exposure beyond the team's agreed guardrails, or commits shared funding belongs at the appropriate wider level. Make the boundary available where the work is planned. If it only exists in a leader's head, the team will still wait.
3. *Put authority beside the relevant information.* For each material decision, ask where the current evidence sits. It may be with a customer-facing colleague, an engineer who understands the system constraint, a person tracking a supplier, or a product owner holding the current priority. Bring those people into the decision early. Do not ask them merely to supply material for a meeting elsewhere. If the information is distributed, convene the smallest group that can see the whole choice and give that group a named authority within the boundary.

4. Use *D.I.R.E.C.T Value Routing (DVR)* to select the next move.

Compare the available actions using the VER, current evidence, and confidence. The discipline matters most when a plan has already absorbed time and effort. DVR gives the team permission to change course when a better option emerges, but requires it to explain why the option is better. In the password example, an automated password-reset guide may have a high VER and low risk, while a broad chatbot replacement has uncertain value and high effort. The next move is not the most ambitious idea. It is the one the evidence supports now.

5. Make the guardrail part of the decision, not a warning added later.

A team should be able to say what would make its chosen action unacceptable: a fall in service quality, a cost consequence, a risk limit, or a conflict with the stated purpose. Where the guardrail is reached, the team pauses, asks for the right wider authority, or selects another action. This is the responsibility that comes with local authority. It also gives senior leaders a clear basis for intervention without taking every routine choice away from the team.

6. Record material decisions while their context is still available. The record need not be lengthy. It needs to preserve the purpose, the option selected, the evidence used, the relevant guardrail, the authority used, and when the team will revisit the decision. This is not paperwork for its own sake. It prevents a later review from becoming a reconstruction based on partial memory and it shows whether the team acted within its boundary.

7. Review the route, not only the outcome. Look at whether the team had the relevant information, whether the boundary was clear,

whether DVR was used, and whether an avoidable delay was created by escalation.

Purpose-based leadership is demanding for leaders because it requires them to be clear about outcomes and boundaries while allowing the method to be chosen close to the work. It is equally demanding for teams. They cannot say they were following orders when they have been given a decision. That is the point. Good local judgement is visible, reasoned, and connected to the wider purpose.

The exercise

Run a 55-minute decision-routing session with the people who regularly make or approve delivery choices. Prepare one current decision that has not yet been made, together with the available evidence, any TVI or VER reference, and the existing service, cost, or risk limits. Choose a decision that has enough consequence to matter but is not already committed to a senior forum.

- Spend 10 minutes writing the purpose in one sentence and listing the guardrails. Ask the group to remove instructions disguised as outcomes. Capture the version that tells a team what needs to improve and what must be protected.
- Spend 15 minutes mapping where the relevant information is held. Name the people or roles closest to customers, systems, delivery constraints, and risk. Then identify the smallest group that can make the choice with that information present.
- Spend 15 minutes agreeing the authority boundary. Mark what that group may decide, what it must consult on, and what must go to a wider authority. Capture the specific condition that moves the

decision upwards, rather than writing a general instruction to seek approval.

- Spend 10 minutes applying D.I.R.E.C.T Value Routing (DVR) to the available options. Record the VER, current evidence, confidence, the chosen next move, and the guardrail that would cause the team to reconsider.
- Use the final 5 minutes to agree who will record the decision, when the evidence will be reviewed, and which material decisions will be tracked during the next month.

Leave with a one-page decision record and a visible authority boundary. Do not use the session to settle every possible future choice. Its purpose is to make the next real decision faster and more explainable, then learn from the record over the month.

What to measure

The TEAM IQ measurement is *D.I.R.E.C.T Value Routing (DVR)*. Track material decisions for one month. For each decision, record decision latency from issue recognised to decision made, the decision level, and whether a TVI, VER, or risk reference was used. The artefact is the month's material-decision record, read at the end of the month by the team and the authority that receives its escalations. Look for avoidable delay: a decision held above the information, a missing reference, or an unclear boundary. Treat the pattern as evidence about the decision route, not as a tally of individual compliance. Compare it with the record from the previous month to see whether the route has improved.

One thing to do this week

Choose one decision your team routinely sends upwards. Before the next instance, write the purpose, the guardrails, and the condition that would require wider authority. Agree with the relevant leader whether the people holding the current evidence can decide it within those limits. Then use DVR on the next live choice and record what happened.

18. Interconnected metrics

On the second Friday of a pilot, a team is due to decide whether a new data feature can move beyond its initial users. The product owner wants adoption to grow. The engineer has concerns about accuracy in a narrow set of records. Operations has seen a handful of slow responses during the busiest part of the day. The meeting begins with separate statements: adoption is encouraging, the build is nearly complete, and the response-time issue needs attention.

The decision cannot be made from three good numbers viewed alone. A rise in use has little value if accuracy is not good enough. A technically complete feature is not ready if its response time creates a worse customer experience. Equally, a team can keep improving an already adequate technical measure while losing the customer opportunity it was meant to address. The work needs measures that explain the trade-off between them and a routine that makes the consequence of a change visible before the team simply carries on.

The principle

Interconnected metrics show how an individual or functional measure contributes to the shared TVI and to agreed quality guardrails. They resist the familiar pressure to optimise the number owned by one role. A development team judged only by speed can create avoidable support cost. A support team judged only by call time can transfer customers rather than solve the underlying issue. Neither group has necessarily behaved badly. They have been given a local measure without a clear connection to the outcome and constraints shared by the wider team.

A wall of numbers does not correct this. It often makes the problem harder to see. Teams can report delivery activity, response time, adoption, defects, cost, and risk in the same pack without agreeing which measures matter together or what decision a movement should prompt. More measures then produce more commentary and less judgement. Interconnection means that a change in one measure has a stated implication for another measure, for the TVI, or for a guardrail. The team can therefore ask a practical question: if we pursue this improvement, what does it do to the outcome we are here to create and to the boundary we have agreed not to cross?

The shared TVI is the anchor. Functional and individual measures should make sense in relation to it, not compete with it. Guardrails make the connection honest. They acknowledge that a team cannot pursue a desired effect at any price. In the data-feature pilot, customer adoption may be the TVI, while accuracy and response time are quality guardrails. The team is not choosing between customer value and technical quality. It is deciding whether adoption is being achieved without breaking the conditions that make the feature worth using.

Measures also need a feedback loop. A measure that is reviewed only after the work is complete cannot guide a live choice. The team needs an agreed cadence to consider what has changed, what has not changed, and what the new evidence means for the plan. That cadence is not a ceremonial update. It is the point at which a team decides whether to continue, change direction, seek a wider decision, or test a different idea.

Insight Risk Triggers (IRT) make the feedback loop explicit. An IRT is a predefined threshold connected to a date or point in the work. If a risk remains above an agreed exposure by that date, or if a test has failed to produce evidence by that point, the team must pause, pivot, escalate, or choose a new experiment. The trigger prevents a weak but common habit: recognising that the evidence is against the plan, then continuing because there is no agreed moment to decide otherwise.

One local number is rarely enough to explain whether the work is helping. TEAM IQ connects the measures the team already needs so that it can see trade-offs and act while there is still a choice.

What it looks like when it is missing

The pilot team reviews progress through separate updates. Product reports that adoption is moving in the right direction. Engineering reports that the planned work is complete. Operations reports that response time has worsened at busy periods. Each update is accepted. The meeting ends with three actions owned by three people and a plan to review them next week.

No one has connected the response-time change to the customer-adoption aim. No one has said what level of accuracy is necessary before the feature should be extended. The team has not agreed what evidence would stop a broad rollout. Because the measures are separate, every issue can be treated as another item to improve while the main decision is deferred.

The feature is extended because the completion measure is green and adoption has increased. Later, the team spends time responding

to accuracy concerns that were already visible. Operations feels that its warning was noted but not used. Engineering feels that a difficult technical constraint was made secondary to a growth target. Product believes it had a clear reason to proceed. All three accounts are understandable, and all point to the same gap: the measures did not explain their relationship or require a decision.

What it looks like when it works

Before the pilot starts, the team states customer adoption as the TVI and names accuracy and response time as guardrails. It discusses how the measures relate. Adoption that rises while accuracy remains below the agreed condition is not a reason to scale. A response-time deterioration that affects the customer experience is not a technical footnote. The team has one view of the decision, even though colleagues remain responsible for different evidence.

It also agrees an Insight Risk Trigger (IRT): if accuracy cannot exceed 97% by the end of the second pilot week, the team will not scale the feature. The condition is visible to everyone who is doing the work. It is not introduced after the threshold has been missed as an explanation for a decision that has already become difficult.

At the Friday meeting, accuracy has not reached the trigger. The team does not argue about whether the incomplete result should be described as good or bad. It follows the agreed response and pivots to a narrower use case. Product can explain the customer effect, engineering can focus on the condition that needs more evidence, and operations can test response time in the smaller setting. The decision

is timely because the team agreed in advance what the evidence would mean.

How to do it

1. *Start with the shared TVI.* For a significant initiative, name the outcome measure that tells the team whether the work is creating the intended effect. The TVI is the reference point for choosing supporting measures. If the team cannot identify the shared TVI, it is not ready to use activity or completion as a substitute.
2. *List the measures held by the different parts of the work.* Bring together the measures used by product, delivery, operations, engineering, service, or other relevant colleagues. Include the measures that affect the customer outcome and the measures that show whether the work is staying within agreed quality guardrails. Do not begin by building a large reporting pack. The task is to identify the small set that could pull the team in different directions if read alone.
3. *Describe the relationship between each measure and the TVI.* For every measure, write what a movement means for the shared outcome or a guardrail. A measure may support the TVI directly, warn of a cost of pursuing it, or test whether a stated assumption holds. In the data-feature example, adoption is connected to the intended customer effect; accuracy and response time test whether that effect is usable. If a measure has no clear relationship to a decision, question why it is being reviewed in this forum.
4. *Agree the guardrails before a target creates pressure.* State the conditions the team will protect while pursuing the TVI. Quality

guardrails stop a locally attractive result from hiding a wider cost. Make them specific enough that the team can recognise a breach. Where there is uncertainty, record what evidence is still needed. The aim is not false precision. It is to avoid discovering after a decision that different roles were protecting different boundaries.

5. *Create at least two Insight Risk Triggers (IRT) before delivery begins.* Each IRT should state the evidence or risk exposure, the threshold, the date or delivery point, and the required decision window. An IRT can cover a risk that remains above agreed exposure by a given date, or a test that has not produced evidence by a given point. State the response in advance: pause, pivot, escalate, or choose a new experiment. The response may vary, but it must be a decision rather than a vague request to monitor the position.
6. *Set a feedback-loop cadence that matches the work.* Agree when the team will read the interconnected measures together and who needs to be present to interpret them. At each review, ask what changed, what did not change, and what the evidence now means for the plan. A regular session with no decision need can be short. A trigger review needs the authority and evidence necessary to act within the stated decision window. Do not let the meeting become a sequential round of updates; return repeatedly to the TVI, guardrails, and IRTs.
7. *Respond to a trigger as a test of the feedback loop.* When an IRT is reached, make the agreed choice and record the time taken. If the team decides late, analyse why. Perhaps the data was unavailable, the right people were not in the review, the boundary was unclear, or a colleague did not feel able to report the issue. Do not dismiss

the delay as merely a delivery problem. It is evidence that the feedback loop failed to turn information into a decision.

8. *Remove measures that do not help the decision.* After several reviews, ask whether each measure has explained a trade-off, protected a guardrail, or prompted an action. If not, it may belong elsewhere or not be needed. Interconnected metrics make the cost of a trade-off visible early enough for the team to choose deliberately.

The exercise

Run a 60-minute IRT design session before the next significant initiative begins. Prepare the proposed TVI, the evidence already available, the likely quality guardrails, and the people responsible for the measures that will inform the decision. Keep the focus on the evidence that will guide a pivot-or-persevere choice.

- Spend 10 minutes agreeing the shared TVI and writing the outcome it represents. Capture the quality guardrails that must be protected while the team pursues that outcome.
- Spend 15 minutes listing the relevant individual, team, and organisational measures. For each, state how a movement affects the TVI or a guardrail. Put aside measures that do not explain a current decision.
- Spend 20 minutes creating at least two Insight Risk Triggers (IRT). For each one, record the threshold, the date or point in the work, the evidence or risk exposure it concerns, the required decision window, and whether the response will be to pause, pivot, escalate, or choose a new experiment.

- Spend 10 minutes agreeing the feedback-loop cadence and the people who must be present when an IRT is read. Make clear where the linked measures will be viewed together.
 - Use the final 5 minutes to test one trigger against a plausible result. Ask whether the group could make the stated decision with the evidence named. If not, revise the trigger before delivery starts.
- Capture a single page linking the TVI, guardrails, supporting measures, and the two IRTs. Share it with everyone who supplies or uses the evidence. Its purpose is to make an agreed future decision easier, not to predict every result.

What to measure

The TEAM IQ measurement is *Insight Risk Triggers (IRT)*. For the next significant initiative, create at least two IRTs before delivery begins and read them on the agreed cadence with the measures that explain them. The artefact is the IRT record: the threshold, timing point, decision window, and the action taken when it was reached. Measure whether trigger decisions occur within the stated decision window. A late decision should be analysed as a feedback-loop failure, not simply a delivery failure. Review the record after the initiative to see whether the triggers exposed the right trade-offs. Compare the timing with the stated decision window and capture the reason whenever the team acts after it, including missing evidence, unclear authority, or unavailable data.

One thing to do this week

For one active initiative, put its TVI and two quality guardrails on the same page. Ask the team to name one measure that could look

good while damaging one of those guardrails. Agree one Insight Risk Trigger (IRT) with a threshold, date, and decision window. Read it at the next review alongside the linked measures.

19. Clear channels

At 08:45 on release day, a security concern is raised while checks are under way. The release manager has heard it in a private message from an engineer. A delivery lead recalls a related risk from the previous week's meeting. The product owner has an email with a customer commitment, and the person who owns the service-quality guardrail is travelling. By 09:20, fragments of the issue sit in different places. Nobody can point to a decision record.

The release is delayed. That may be the right choice. Three months later, a new manager asks why it was delayed and whether the condition still applies. The answer should not depend on who happens to remember the morning. If the organisation cannot find the context of a material decision, it has not shared the information in a form that can be used. It has merely passed messages around at speed.

The principle

Clear channels and transparent information sharing give people a usable route to operational updates, evidence, decisions, risk escalation, and learning. Transparency does not mean broadcasting everything to everyone. A person cannot make better choices because they receive every message in the organisation. They need relevant information that they can find, understand, and use in time. That is a practical standard. It asks whether the right audience can locate the current item, see what it means, and know what action or authority it requires.

A channel needs a clear purpose: what belongs there, who keeps it current, and when it is read. The test is the pathway from information

to action; a meeting or shared workspace that cannot be used in time has failed.

When information is scattered across private conversations, hidden spreadsheets, and meeting memories, the organisation becomes dependent on gatekeepers. A delivery lead may have to translate between the engineer who knows the system, the product owner who knows the customer effect, and the leader who holds the decision boundary. Each translation can remove a condition, an assumption, or a warning. People who are not present may act on an old version because they have no clear source for the current position. This is not a problem solved by copying more people into email. It requires a recognised channel for the particular kind of information.

D.I.R.E.C.T Decision Audits (DDA) turn transparency into learning. A DDA asks whether the team followed a sound decision process: was the purpose clear, did it seek evidence, were assumptions and risks visible, was the VER considered when appropriate, and was the decision recorded? The audit does not punish a decision because the outcome was uncertain. Complex delivery includes choices where the result cannot be known in advance. The purpose of the audit is to improve the quality and traceability of the next choice.

Google's Project Aristotle notes that people in psychologically safe teams can bring up problems and difficult issues, ask for help, and take interpersonal risks. Clear channels are the operational infrastructure that makes that behaviour usable. Someone may be willing to raise a security concern, but the concern still needs a route that reaches the people with the evidence and authority in time. Safety

without a route can leave important information private. A route without safety can leave it empty.

Transparency therefore has a boundary. It shares the context necessary for the relevant people to understand and use a decision. It makes material choices traceable without turning every working conversation into a public record. It lets a new colleague understand why a release was delayed, what alternatives were considered, who made the call, and when the decision should be reviewed. That is enough to reduce dependence on memory and enough to support a better decision next time.

What it looks like when it is missing

The security concern moves through direct messages, a hurried call, and a meeting that has no agreed record. The engineer tells the release manager about the technical condition. The release manager tells the delivery lead that there is a risk. The product owner hears only that the release may be delayed. By the time a senior leader is asked for a decision, the description has become a choice between delaying and accepting an unspecified concern.

After the release is delayed, one colleague updates a private spreadsheet and another adds a brief line to meeting notes. A team member who was absent believes the delay was caused by a capacity issue and begins discussing a different response. The original security concern is no longer visible enough to guide the work.

The next time a similar concern is raised, people hesitate. They do not know whether to post it in the operational update, take it to a meeting, send it to a named leader, or create a risk entry. They choose

the least exposed route: a private conversation. Information then sits with the people who happened to receive it. The organisation has communication activity but no clear channel for a material decision.

What it looks like when it works

The team has already identified pathways for operational updates, evidence, decisions, risk escalation, and learning. The engineer records the security evidence in the agreed place and uses the risk-escalation route because the release condition is affected. The release manager brings together the people who hold the relevant information and authority. The operational update points to the current decision record rather than trying to reproduce it in several messages.

The record states the context, risk evidence, TVI guardrails, alternative options, who made the decision, and the planned review. It also makes clear that the release is delayed because the security condition must be understood before the team can proceed. People who need to act can find the same current position. People who only need to know the release status receive an update that tells them what has changed without exposing every working detail. At the next D.I.R.E.C.T Decision Audit (DDA), the team checks whether the evidence reached the decision makers in time and whether the record made the choice understandable. The delay is not treated as proof of failure or success on its own; the process supplies learning for the next choice and its channel.

How to do it

1. *Name the information pathways that delivery actually uses.* Start with the five types in the work: operational updates, evidence, decisions, risk escalation, and learning. Describe the route for each in plain terms. Where does a colleague put a current operational update? Where is evidence for a material choice held? How is a risk made visible to the right authority? Where is a decision recorded, and where does the team review what it learned? This exposes gaps hidden by the vague instruction to keep people informed.
2. *Give each channel a purpose and an audience.* State what belongs in the channel, who needs to read it, who keeps it current, and when it is used. The channel for an operational update may direct people to a decision record rather than become the record itself. Evidence may need to be available to the people analysing an option. A material decision needs an identifiable record that survives a meeting. Do not create several locations that claim to be the current source. If a message points elsewhere, make the destination clear.
3. *Create a decision record for material choices.* Use a decision log that captures the context, risk evidence, TVI guardrails, alternative options, who made the decision, and the planned review. Add the purpose, the authority used, the assumptions and risks made visible, and whether the VER was considered when appropriate. These are not decorative fields. Each allows someone who was not in the room to understand how the choice was made and whether a later change in conditions should alter it.
4. *Make relevant information findable in time.* Test the channel with the people who need it, not just the person who created it. Can

an engineer find the current customer constraint? Can a product owner locate the risk evidence? Can a leader see the decision that needs their authority without reading every operational discussion? Do not rely on a colleague knowing whom to ask. A clear channel reduces the number of gatekeepers between an item of evidence and the decision it should inform.

5. *Separate visibility from indiscriminate distribution.* Transparent sharing is selective enough to keep information useful. The people affected by a release delay may need the decision and its consequence; the people conducting the technical investigation may need fuller evidence. Set access and updates according to what people need to understand or act. When a decision changes, update the record first and use the operational channel to direct readers to it.
6. *Use risk escalation as a decision route, not a parking place.* A risk channel should make clear what exposure is being raised, what evidence supports it, which guardrail is affected, and who has authority to act. Give the escalation a required response point. If an issue is not material, keep it in the operational path.
7. *Run D.I.R.E.C.T Decision Audits (DDA) on a sample of records.* Audit the process rather than judging the outcome with hindsight. Ask whether the purpose was clear, evidence was sought, assumptions and risks were visible, the VER was considered when appropriate, the authority was clear, the decision was recorded, and a review date was present. A poor outcome may still show a good process in uncertain work. A favourable outcome may reveal weak practice that will fail when conditions change.

8. *Use the audit finding to improve one channel at a time.* If decision records have evidence but no clear authority, improve that field and the route into the authority. If risks are visible but decisions have no review date, correct the record and the review routine. Choose the weakest field as the next improvement focus. Avoid a large communications redesign in response to a small, observable gap. Use channels people will follow under delivery pressure.

The exercise

Run a 55-minute clear-channels session with the delivery lead, people who supply operational and risk information, and the authority that receives material escalations. Prepare one recent material decision or a live decision that is likely to be made soon. Bring the evidence available, the current decision record if one exists, and access to the places where the team currently shares information.

- Spend 10 minutes listing the team's paths for operational updates, evidence, decisions, risk escalation, and learning. Capture the current location or meeting for each path and identify where the group is relying on private messages or memory.
- Spend 15 minutes complete a decision record for the chosen item. Include context, purpose, risk evidence, TVI guardrails, alternative options, authority, assumptions and risks, whether the VER was considered when appropriate, who made the decision, and the planned review.
- Spend 10 minutes test findability. Give each participant a different question about the item and ask them to locate the information

through the agreed channels. Note where a person has to ask a gatekeeper or finds two competing versions.

- Spend 15 minutes conduct a short D.I.R.E.C.T Decision Audit (DDA). Check the record for clear purpose, evidence, authority, and review date. Agree one weakness in the pathway or record, and assign the person who will correct it.
- Use the final 5 minutes to set the four-week trial: log every material decision using the DDA fields, then audit a sample of five at the end of the period.

Capture the pathway list, the completed decision record, and the single improvement focus. The session should not decide which technology everyone must use. It should make the route and the required information clear enough that a material decision can be understood after the meeting ends.

What to measure

The TEAM IQ measurement is *D.I.R.E.C.T Decision Audits (DDA)*. For four weeks, log every material decision using the DDA fields. At the end of the period, audit a sample of five records. Report the proportion with clear purpose, evidence, authority, and review date. The artefact is the decision log and its DDA sample, read at the end of each four-week period by the team and the relevant decision authority. Choose the weakest field as the next improvement focus. A record is useful because it shows where information ceased to be usable, not because it proves a decision was certain. Compare proportions with the previous audit. Use results to improve it next period.

One thing to do this week

Take the next material decision and put its context, purpose, risk evidence, authority, and review date in the team's decision log before the meeting ends. Send one operational update that points people to the record. In the next team meeting, ask one colleague who was absent to find and explain the decision without asking its author.

20. Value stream mapping without the ceremony

At the release meeting, the team has a slide for every workstream and a date beside every work item. The product lead wants the reporting capability in front of users before the quarterly review. The design lead wants another pass at the visual animation. Security has asked for evidence that an approval was completed, but nobody is clear who owns the evidence. A developer says the code is ready. Operations has not seen it. The chair asks why a small change has taken so long.

The usual response is to produce a larger plan. People add more columns to the tracker, schedule a mapping workshop and leave with coloured notes that soon become wallpaper. That does not answer the question in the room. The team needs to know what value the work is meant to create, what standard is sufficient for that value, and where time or effort is being consumed before the customer receives anything. Until it can see those things together, it will argue about preference, urgency and blame.

The principle

Quality starts before a team begins work. It starts by continuing with the end in mind: stating the value to be created, who receives it, how the team will recognise it, and what it will cost to create, support and change. That sequence is deliberately awkward for teams used to receiving a requirement and opening a delivery board. It makes them pause before they turn activity into commitment.

In TEAM IQ, quality is the usefulness and worth of a product, service, process or decision to the customer and the organisation. It is not polish for its own sake, nor a licence to release work that does not meet a necessary standard. A reporting capability that helps

people make a decision may be high quality with a plain interface. A beautiful animation can be a poor quality choice when it delays a capability customers need more. The test is value at an appropriate cost and time.

The *7 C's Value Engineering Standard (CVE)* gives the team a way to make that test explicit. For every quality requirement, it defines a target and a guardrail. The requirements may concern usability, security, reliability, accessibility, performance, accuracy or maintainability. A target says what the work must achieve. A guardrail says what cannot be compromised while achieving it. When the agreed target has been reached, additional effort must compete with the value of other priorities. It may be worth doing, but it is no longer assumed to be good merely because it improves an attribute.

That distinction matters because gold-plating is easy to hide inside professional pride. A specialist can usually name one more refinement. A programme can usually find a reason to postpone a release. Neither observation is an argument against care. It is an argument for making the trade-off visible, whilst there is still a choice. CVE prevents a debate in which one person argues for speed and another argues for quality as if the two are opposites. The real question is whether the next piece of effort creates enough additional stakeholder value to justify what it displaces.

Conducting value stream mapping makes the cost side of that question visible. It follows the work from the moment a need is recognised to the point at which the customer can use the result. Waiting for approvals, unclear handoffs, duplicated data entry, late defect discovery and unnecessary rework are costs even when no invoice is

attached to them. They make a small request feel slow, tempt teams to start too much at once, and conceal where a decision is actually stuck.

Google's team-effectiveness research cautions against treating measures such as lines of code or bugs fixed as standalone measures of effectiveness. The same restraint applies here. A busy delivery flow and a long list of completed technical tasks do not establish that the right value reached the customer. CVE and value stream mapping put the outcome, the standard and the flow in the same conversation.

What it looks like when it is missing

The reporting capability is described as a feature with a fixed deadline. Nobody has written down whose task it improves or what successful use would look like. The design work continues because it is already under way. Security, operations and the product lead each receive a request at different times, with different descriptions of the change. An approval sits in an inbox because the person asked to approve it cannot see the context.

In the review, the team reports that development is complete. Operations reports that it cannot accept the release. Product reports that the capability is late. Each statement is true within one part of the flow, and none explains the whole delay. A defect found near the end is treated as an individual error rather than evidence that work arrived too late for someone to influence it. More meetings are added, but the route through the work remains unseen.

The schedule shows work assigned to people, but not the time it spends in queues. No one can say which handoff is causing the delay,

which work is repeated, or which decision is missing. The next status report asks for dates rather than the information needed to remove the constraint. The pattern repeats on the next item.

What it looks like when it works

Before work begins, the team records the customer task the reporting capability must support and the point at which it will regard the experience as good enough. It records the security and operational conditions that must hold. The design lead can still propose a refinement, but can show what value it adds and what it will postpone. The product lead can make a decision with that information rather than relying on rank or persistence.

The team then draws the actual path of the work. It sees that a single approval is requested only after a build is ready, although the approver needs information available at the start. It moves that conversation forward. It sees that two people copy the same data into separate documents and removes one copy. The map does not make delivery friction disappear. It changes it from a vague complaint into something the people involved can alter. At review, the team can see whether it met its CVE threshold and where it spent time getting there.

The product lead can explain the release choice in terms of the agreed outcome, not personal preference. The team has a visible basis for refusing extra work that does not meet a target or protect a guardrail.

How to do it

1. *Name the end before naming the work.* For an active item, write a short statement of the value to be created and the person or group

who receives it. State the task, decision or service that should be better when the item is complete. Then ask how the team will know. Do not accept a technical output as the answer when the work exists to change a customer's experience or a business result. Also ask what it will cost to create, support and change. The aim is not a perfect forecast. It is to stop those costs being absent from the decision.

2. *Set the 7 C's Value Engineering Standard (CVE).* Identify the quality requirements that matter for this item. Use the relevant terms directly: usability, security, reliability, accessibility, performance, accuracy and maintainability. For each relevant requirement, agree a target and a guardrail. Make the target observable enough that a review can determine whether it was met. Make the guardrail clear enough that nobody can declare success by sacrificing an obligation which was merely left unstated. Record the standard before development begins, not after a result makes it convenient.
3. *Separate necessary work from optional refinement.* Ask the team to place each proposed activity against the CVE. Does it meet an agreed target? Does it protect a guardrail? Or does it take the work beyond the required level? The third category is not prohibited. It must have a stated value rationale and compete with other priorities. This is where a team prevents gold-plating without asking people to lower their professional standard. It is choosing deliberately where its capacity goes.
4. *Conduct value stream mapping from request to use.* Put the active item at the end of the map and trace backwards through the actual

steps that enable it. Include decisions, handoffs, approvals, information received, information re-entered, build work, checking and release. Ask the people who perform each step to describe what happens, rather than drawing the intended process from a policy document. Mark where work waits, returns for clarification, is duplicated or discovers a defect late. Keep the map to the path the item really follows. It does not need decorative symbols or a day-long workshop to be useful.

5. *Find the constraint that changes the whole flow.* Review the map for the earliest practical point at which a missing decision, condition or piece of information could be brought forward. A late operational question may need to be raised when the item is shaped. A repeated data entry may need one shared source. An approval may need a clearer request. Choose one change that removes waiting, repeated work or late discovery. Do not turn the map into a catalogue of every irritation in the organisation. A small change that is adopted is more useful than a large map that is filed away.
6. *Review the decision when the item is ready.* At review, return to the CVE record. Record whether the team stopped at the agreed threshold, exceeded it deliberately with a value rationale, or fell short. If it exceeded the threshold, record what capacity that consumed. If it stopped at the threshold, record what capacity was saved for other priorities. Revisit the value stream map when the work exposed a handoff or wait that had not been visible. This turns quality from a retrospective opinion into a repeatable delivery decision.

The exercise

Run this exercise with the people responsible for one active work item. Choose an item that has not yet entered development if possible; if it has started, use it to make the next decision clearer. Prepare the current request, any known constraints, and a blank space where everyone can see the CVE and the flow of work. Allow 55 minutes.

- Spend the first 10 minutes agreeing the value the item should create, who receives it, how the team will know, and the expected cost to create, support and change. Capture short statements rather than a new requirements document.
- Spend 15 minutes agreeing the 7 C's Value Engineering Standard (CVE). List the relevant quality requirements and record a target and guardrail for each. Ask which proposed work is necessary to meet the standard and which goes beyond it.
- Spend 20 minutes tracing the work backwards from customer use to the initial request. Add each actual handoff, approval, repeated entry, wait and checking point. Ask people to describe the work they do, including work that is normally invisible in a plan.
- Use the final 10 minutes to select one source of waiting, duplication or late discovery that the team can change. Record an owner and when the change will be checked.

At the item review, bring back the CVE record. Record whether the team stopped at the agreed threshold, exceeded it deliberately with a value rationale, or fell short. Estimate the capacity saved or consumed by that decision. Keep the map with the item until the selected change has been tested; otherwise it becomes evidence of a meeting, not an aid to delivery.

What to measure

The artefact is the *7 C's Value Engineering Standard (CVE)* record for an active work item. It shows the agreed quality targets and guardrails, then the review decision: stopped at the threshold, exceeded deliberately with a value rationale, or fell short. It also records the estimated capacity saved or consumed. Read it when the item is prepared, before development begins, and at the item review. For a team with several active items, read the current records in the regular delivery review to spot repeated choices that are consuming capacity without a stated value rationale. Over time, compare records to see whether the same optional refinement keeps returning and whether it displaces higher-value work or extends delay for customers without a stated rationale.

One thing to do this week

Choose one item that is about to begin. Before anyone starts development, ask the team to write the customer value, one quality target and one guardrail. Then trace only the next three handoffs it must make before a customer can use it. Do not schedule a mapping event. Put the result beside the item and use it in the next decision.

21. Customer-centric design

A team has released an improved online form. The delivery board is green. The automated checks have passed, the service is available and the release note has gone out. In the following days, the service desk receives messages from people who cannot tell which option applies to them. Some leave the form unfinished. Others submit it and then contact the organisation to ask what they have agreed to. The team has evidence that the release happened, but very little evidence that the person using it can complete the intended task.

At the next review, the conversation begins with technical activity because that is what the team has ready to show. A test lead explains the checks. A developer explains the changes. The product lead has a list of requests from different customers, none of which can simply be accepted as a priority. What is missing is a shared account of the customer's need, context, behaviour and constraints. Without it, the team will either build the loudest request or defend its existing work. Neither is a reliable way to judge quality.

The principle

The most expensive quality failure is building the wrong thing perfectly. Customer-centric design begins with the people who receive the work: their need, the setting in which they act, what they do now and the constraints that shape their choices. It asks for enough evidence to understand the problem, test the proposed value and improve the work as customers respond. It does not mean that every request becomes a commitment. It means requests are examined against a real customer problem instead of being sorted by volume or seniority alone.

For a delivery leader, this changes the starting point of design. A feature description is rarely a sufficient account of value. "Add a field", "make the screen clearer" and "automate the notice" may each conceal several different customer tasks. The team needs to know which task matters, where it currently fails and what a person needs in order to complete it. That inquiry can be brief, but it cannot be replaced by an internal debate about what users probably want.

The *7 C's Outcome Metrics (COM)* moves the primary quality conversation from outputs to outcomes. Output measures continue to have a place. Test coverage, defects, uptime and delivery speed can protect necessary conditions. They are technical guardrails. They do not, by themselves, show whether a customer completed a task, whether retention improved, whether revenue changed or whether the experience was worth its cost. COM connects the team's work to the customer's actual experience and to the business value that follows.

This is a disciplined rather than sentimental approach to the customer. A team may discover that a requested change cannot be justified, that an existing feature should be altered, or that a capability should be retired. Customer-centric design does not promise that the customer will get every proposed answer. It makes the problem, the evidence and the decision visible. That gives product, delivery and operational colleagues a common basis for disagreement.

Continuous customer feedback is what keeps the team from treating a release as the final moment of learning. Before release, the team should have an explicit outcome it expects to see. After release, it should seek feedback and outcome evidence on a regular rhythm.

The feedback may show that the technical work is sound but the language is confusing, the steps are in the wrong order, or a constraint was misunderstood. The response is not to abandon technical quality. It is to keep technical evidence as a guardrail and change the primary target to the outcome that matters.

Google's team-effectiveness work combined leader, member, executive and quantitative perspectives because each saw a different part of effectiveness. COM applies the same discipline to quality. Technical, customer and business evidence should be visible together. A green delivery board is useful information. It is not a conclusion about the value of a release.

What it looks like when it is missing

The online form is designed from an internal policy document and a list of stakeholder comments. The team measures completed development tasks, passed checks and whether the service is up. It treats service-desk messages as an operational issue for another group. When a customer asks for help, the request is added to a backlog with no account of the task that failed.

At review, people defend the measure closest to their own role. The test lead points to coverage. Operations points to uptime. Product points to the length of the backlog. None of these accounts shows whether the person filling in the form can understand it and complete the required task. The team makes a series of small changes based on opinion, then calls the next release an improvement because activity continued. Feedback arrives late, scattered and stripped of the context needed to act on it.

Nobody is accountable for bringing together the customer experience and the business effect. The product lead therefore has to interpret isolated reports after decisions have already been made. The absence of a shared scorecard makes it easy to call a release successful before there is evidence that it solved the original problem for customers in context.

What it looks like when it works

Before changing the form, the team identifies the customer task and listens to people who carry it out in the relevant context. It records where language, sequence or constraints make the task difficult. It does not accept every suggested feature; it uses the evidence to decide which obstacle is worth addressing first. The team writes the expected customer outcome and the business result alongside the technical conditions that must remain true.

After release, the team makes it easy to collect feedback at the point where customers use the form and brings that feedback into the review. It reads a customer measure, a business measure and a technical guardrail together. If customers are still struggling, the team can see whether the problem is understanding, flow, reliability or something else. It chooses to scale, improve, reposition or retire from evidence, not from attachment to the work already delivered.

Customer feedback is not passed around as anecdote. It is linked to the COM scorecard, so the team can see whether a complaint reflects a repeatable obstacle, a temporary effect or an issue outside the feature. The business representative can also see whether the

intended value follows, rather than waiting for a technical report to declare success.

How to do it

1. *Choose the customer task, not the requested output.* Select the feature, service or decision under discussion and state who receives it. Describe what that person is trying to complete, decide or avoid. Ask for the context in which the task happens and the constraints that affect it. A person using an internal service under time pressure may need something different from a person using the same service occasionally. Keep the statement close to observable behaviour. It should help the team test whether it has improved the task.
2. *Gather evidence before settling on an answer.* Use the customer contact already available to the organisation: direct feedback, service-desk contacts, existing usage evidence or conversations with the people who support the service. Ask what the person was trying to do, what happened, what they did instead and what constraint mattered. Do not ask only whether they like a proposed feature. That question invites a preference. The team needs evidence of the problem and its effect before it decides what to build.
3. *Set the 7 C's Outcome Metrics (COM).* For the proposed release, create a small COM scorecard with at least one customer measure, one business measure and one technical guardrail. The customer measure should show whether people can complete the relevant task or experience the intended benefit. The business measure should show the organisational value that follows, such as reten-

tion or revenue where relevant. The technical guardrail should retain a condition the team must protect, such as test coverage, defects, uptime or delivery speed. Choose measures that can influence a decision, rather than measures selected because they are easy to obtain.

4. *Make feedback part of the delivery flow.* Decide before release how feedback will reach the people who can use it. The mechanism must be close enough to the customer experience that it retains context, and regular enough that the team does not wait until a large amount of dissatisfaction has accumulated. Give the team a named point in its review to read feedback alongside COM. The aim is not to collect praise or create an endless survey. It is to find out whether the proposed value is occurring and what needs to change if it is not.
5. *Review at 7, 30 and 60 days.* Use the same COM scorecard at each review. At the earlier review, look for immediate evidence of whether the release can be used as intended and whether a technical guardrail has been affected. At the later reviews, look for the business effect and whether the customer outcome is holding. Record the evidence that supports the next choice. The intervals prevent a release from being declared complete before customers have had a fair chance to respond.
6. *Decide from evidence.* At each COM review, choose one of four actions: scale, improve, reposition or retire the feature. Scaling is justified when the outcome is occurring and the cost remains appropriate. Improving is justified when the customer need remains valid but the current work is not achieving it. Repositioning is

justified when the work has value for a different use or group. Retiring is justified when continuing would consume effort without sufficient value. Record the decision and the evidence behind it. This makes it possible to change direction without presenting that change as a failure of the people who built the release.

The exercise

Use this exercise for one recently released feature. Invite the product lead, a delivery representative, someone who sees customer feedback, and the person responsible for the relevant technical guardrail. Prepare the feature description, any feedback already received and the information needed to read the selected measures. Run a 55-minute session to create the COM scorecard, then schedule the follow-up reviews at 7, 30 and 60 days.

- In the first 10 minutes, state the customer task the feature was meant to improve. Record the customer need, context, behaviour and constraint that the release addresses. Separate this from the team's description of the technical output.
- In the next 15 minutes, identify at least one customer measure and one business measure that would show value, plus one technical guardrail. Capture them as the 7 C's Outcome Metrics (COM) scorecard. Agree where the information will come from and who will bring it to review.
- Spend 15 minutes reviewing available early feedback. Note what customers were trying to do and whether any repeated obstacle is visible. Do not decide a change from a single comment; use

the discussion to test whether the scorecard will expose the right evidence.

- In the final 15 minutes, book the 7, 30 and 60 day reviews and agree that each will decide whether to scale, improve, reposition or retire the feature.

At each later review, read the same scorecard rather than replacing it with a new list of delivery measures. Record the decision, the evidence behind it and any change needed to the feature or feedback mechanism. The session establishes the loop; the later reviews close it.

What to measure

The artefact is the *7 C's Outcome Metrics (COM)* scorecard. For each recently released feature, it contains at least one customer measure, one business measure and one technical guardrail. Its purpose is to show customer and business impact without discarding technical evidence. Read the scorecard at 7, 30 and 60 days after release, alongside customer feedback, and record whether the feature should scale, improve, reposition or retire. Keep the technical guardrail visible, but do not allow it to stand in for the customer outcome. Make it a standing item in the feature review until the 60-day decision has been recorded, even if technical delivery has moved on. This prevents the customer outcome being lost when the next release takes attention.

One thing to do this week

Take one recent release to the next team review. Ask the people present to name the customer task it was meant to improve. Add one

customer measure and one business measure beside the technical guardrail you already read. Book a 30-day review now, with someone who can bring customer feedback. That is enough to begin treating the release as a test of value rather than a completed output.

22. Quality, cost and the cross-functional team

The team is ready to approve an automated customer-notification process. The business case says it is a short build. Development has selected an approach. Near the end of the meeting, someone from customer support asks who will handle messages that do not arrive. Operations asks how delivery failures will be monitored. A colleague responsible for customer communications asks about message templates and opt-out rules. Security asks how contact data will be protected. The room becomes quiet because none of these questions belongs neatly inside the original build estimate.

The wrong response is to treat the questions as delay caused by people outside the team. Each one describes work or risk that will exist after the build is complete. If the questions are left until release, the team may find a defect, a compliance concern or an operating cost when change is difficult and expensive. If they are heard early, the team can compare its initial approach with an alternative that may cost more to build but less to maintain, support, secure or retire. Quality requires that wider view.

The principle

Corrective quality assurance is not merely testing near the end in the hope of finding defects before customers do. It is the practice of asking quality questions while decisions are still cheap to change. Security, accessibility, operational support, customer communications, data protection, performance, legal constraints and maintainability are not add-ons that can be attached when a build appears complete. They are conditions that can alter the design, the cost and the value of the work.

This is preventative in effect, even though the TEAM IQ practice is called Corrective QA. It corrects the course early by bringing evidence into the decision. A team that waits for a final checking stage often turns quality into a contest between the people trying to release and the people asked to approve. The later the question is raised, the more likely it is to be framed as an obstacle. Asked when the work is still being shaped, the same question can inform the design and prevent avoidable rework.

Lifecycle cost analysis completes the picture. Build cost is visible because it has a plan, people and a date. The cost of maintaining message templates, handling opt-out rules, monitoring delivery failures, responding to customer complaints, securing data and retiring old channels may be spread across several groups and future budgets. That does not make it optional. A feature that is inexpensive to create may be expensive to support, maintain, secure or retire. Conversely, an approach that looks costly at the outset may remove recurring cost or customer harm.

The purpose is not to select the lowest build cost, or to insist on the most elaborate answer. It is to make the value-versus-cost trade-off visible over the full life of the work. That requires more than the view of a single discipline. Development understands build cost. Operations sees reliability. Support sees customer pain. Finance sees economic effect. Security sees risk. Product sees desired value. Customer communications sees what a message means to the person receiving it. Each account is incomplete by itself.

Cross-functional collaboration brings those accounts into the decision early, around a shared outcome rather than a late approval

gate. Woolley and colleagues' findings on social sensitivity and balanced turn-taking indicate that groups benefit when different perspectives can be expressed and integrated. In delivery terms, diversity of function is only useful when the information changes a decision. Inviting colleagues to a meeting after the scope has been fixed is not collaboration. Giving their evidence a place before approval to build is.

The *Lifecycle Value Review* is the point at which the team does this visibly. It looks across quality, cost and risk before a major commitment. It makes the assumptions in a business case open to challenge and gives the team a record of issues identified before development. The result may be a different approach, a clearer condition for proceeding or a decision not to pursue the work. Those are quality decisions, not failures of momentum.

What it looks like when it is missing

The notification process is handed from product to development with a short build estimate. Development completes the work against the stated requirement. Operations sees it during release preparation, when monitoring is hard to add without changing the design. Support receives a briefing after customer messages begin to fail. Customer communications finds that templates do not cover the situations customers encounter. Security is asked to sign off on contact data once the route through the system has already been chosen.

The team calls the resulting work unexpected. Defects and rework are entered as delivery problems, whilst the original decision remains unquestioned. Finance sees a build cost that looked low; other

groups absorb maintenance, support and customer costs elsewhere. People protect their own deadlines by handing the work on quickly. No one owns the whole lifecycle, so no one can explain whether the original value still justifies its cost.

Because the issue is found late, the team now has a choice between accepting it, delaying the release or working around it. None of those choices is as good as a design decision made when the relevant people first understood the proposed value and its lifecycle demands for every group involved in delivery from the outset.

What it looks like when it works

Before approval to build, the team brings development, operations, support, finance, security, product and the relevant customer communications perspective into the same Lifecycle Value Review. They start with the intended value and trace what the work will require to create, run, support, secure, change and retire. The questions are specific: how failures will be noticed, how customers will be helped, how opt-out rules will be handled, and what happens to legacy channels.

The initial approach remains possible, but it is no longer the only option in the room. The team may choose an existing communications platform because it reduces lifecycle cost and compliance risk, even if the build takes slightly longer. It records the material quality, cost and risk issues identified before development, together with the assumptions made. At the end of the first release, it compares those assumptions with early operational evidence and updates the busi-

ness case. Work has not crossed a wall. The disciplines have made one decision with different evidence.

The review gives support and operations time to prepare for the chosen route, while development can adjust before change becomes rework. The team can explain the trade-off plainly to sponsors, including what the chosen approach will cost after release.

How to do it

1. *Start with the value and the full life of the work.* Before approval to build, state the customer and organisational value the initiative is meant to create. Then name what will happen after the initial build: it may need to be operated, supported, monitored, secured, changed and eventually retired. Do not begin by asking each function whether it approves a completed proposal. Begin by asking what the proposed value will demand from each part of the life-cycle. This keeps the discussion attached to the outcome rather than to departmental preference.
2. *Bring Corrective QA into the shaping decision.* Identify the quality questions that could alter the design while change remains possible. Consider security, accessibility, operational support, customer communications, data protection, performance, legal constraints and maintainability. Invite the people with direct knowledge of those conditions to explain what they see, including what could fail and what evidence would show it. Corrective QA is useful when it creates a design choice or an explicit condition for proceeding. It is less useful when it becomes a late list of objections with no time left to respond.

3. ***Analyse cost through the lifecycle.*** Set the build estimate beside the work required to maintain, support, secure and retire the initiative. For the notification process, that includes message templates, opt-out rules, monitoring delivery failures, customer complaints and legacy channels. For another initiative, the details will differ, but the question remains: where will recurring effort, customer harm or risk arise after release? Do not pretend that every implication can be priced precisely before a decision. Record the implication and its owner so that the team can compare approaches without allowing unseen work to disappear from the case.
4. ***Run a Lifecycle Value Review before approval to build.*** Use the review to examine quality, cost and risk together. Ask each function what the initiative needs from it during the lifecycle and what assumption could prove false. Look for an approach that protects the intended value at an appropriate cost, not simply the approach with the quickest initial build. Capture material issues, the proposed response, the assumption and who will revisit it. The review should end with a clear decision: proceed with the chosen approach, change the approach, or defer until a material issue can be resolved.
5. ***Make cross-functional collaboration continuous.*** A single review does not mean the lifecycle view is complete. Keep the people who hold operational, support, security, financial, product and customer communications evidence connected to the initiative at the points where their information can change a decision. Share the intended outcome and the recorded assumptions, so that a

concern is assessed against the same purpose rather than treated as a private escalation. When a new condition appears, ask whether it changes value, cost, quality or risk across the whole lifecycle.

6. *Compare prediction with early evidence.* At the end of the first release, return to the Lifecycle Value Review. Compare predicted lifecycle implications with early operational evidence: what support was needed, what monitoring revealed, what customer contacts showed, and what conditions required further work. Update the business case. This is not a search for blame when an assumption proves wrong. It is the step that makes future cost analysis more credible and stops a decision from remaining frozen at the point of approval.

The exercise

Run this exercise before approval to build for the next major initiative. Invite the people who can speak to development, operations, support, finance, security, product and the relevant customer communications or data protection considerations. Prepare the initial business case, the intended value and the proposed approach. Give the group 60 minutes for a Lifecycle Value Review.

- Spend the first 10 minutes stating the value the initiative is meant to create and the customer or organisational outcome it should support. Read the initial build assumption without treating it as fixed.
- Use the next 20 minutes to identify quality questions early: security, accessibility, operational support, customer communications, data protection, performance, legal constraints and maintainability

where they apply. Capture material quality, cost or risk issues that could change the approach.

- Spend 20 minutes tracing the lifecycle cost implications: creating, maintaining, supporting, securing, changing and retiring the work. Ask what each function would need to do, what could recur, and which assumption needs evidence after release.
- Use the final 10 minutes to decide whether the initiative can proceed, needs a changed approach or must wait for an issue to be resolved. Record the number of material quality, cost or risk issues identified before development and the person responsible for each follow-up.

At the end of the first release, reconvene briefly. Compare the predicted lifecycle implications with early operational evidence and update the business case. Retain the review record with the initiative; it shows whether cross-functional evidence changed the work while that change was still possible.

What to measure

The artefact is the *Lifecycle Value Review*. Before approval to build, it records the number of material quality, cost or risk issues identified, the lifecycle implications considered, the assumptions made and the agreed response. Read it at the approval decision and again at the end of the first release. At that point, compare predicted lifecycle implications with early operational evidence and update the business case. A growing list of late issues is not a target to celebrate; it is a prompt to bring the relevant perspective earlier next time. Read it with the same cross-functional group where a changed assumption

affects the intended outcome or could alter the chosen approach before that effect is handed downstream and harder to correct later.

One thing to do this week

For the next initiative waiting for approval, invite one colleague outside development who will live with the result after release. Ask them one question: what will this require to operate, support, secure or retire? Record the answer beside the build estimate and bring it to the approval discussion.

You now have six pillars, a cycle and a set of instruments. They give you a way to examine how teams work, choose a change, observe its effect and decide what to do next. In this final pillar, quality keeps that cycle connected to the value a customer receives, the cost the organisation carries and the people whose work makes both visible.

The fourteen instruments

An instrument in TEAM IQ is a working artefact: a card, board, log, threshold, contract or review that gives the team evidence it can use. It is not a separate reporting layer. Each instrument belongs in the work itself, where a target is set, a claim is tested, a handoff is agreed or a decision is made. The point is to replace meetings that depend on memory, confidence or status colour with evidence that the people doing the work can inspect.

Cycle stage	The question the stage answers	Instruments
Set	What valuable change are we aiming to create?	T.A.R.G.E.T Value Index (TVI); Value-to-Effort Ratio (VER); Target Risk Burn-Down (TRB)
Communicate	What do we know, what are we assuming, and what has changed?	C.L.E.A.R.L.Y Evidence Standard (CES); Expression of Value Delta (EVD)
Own	Who owns this outcome, and what evidence will prove it?	D.R.E.A.M Outcome Contract (DOC); Quantified D.R.E.A.M Boundaries (QDB)
Learn	What are we testing, and what uncertainty are we reducing?	M.I.N.D.S.E.T Value Engineering (MVE); Risk-First Learning Loops (RLL)
Decide and improve	What should we continue, change, stop, or scale?	D.I.R.E.C.T Value Routing (DVR); Insight Risk Triggers (IRT); D.I.R.E.C.T Decision Audits (DDA); 7 C's Value Engineering Standard (CVE); 7 C's Outcome Metrics (COM)

T.A.R.G.E.T Value Index (TVI). The TVI produces a card containing the stakeholder value to change, its scale, the current baseline, the required movement, a guardrail, an owner and a review rhythm. Use it when the team first sets a material outcome and read it again whenever work is prioritised or reviewed. It replaces a broad ambition, a delivery date without a value measure, and a plan that mistakes completed activity for success.

Value-to-Effort Ratio (VER). VER produces a comparable view of estimated value impact divided by estimated cost or effort, with confidence recorded alongside the estimate. Use it to rank the next cycle's options and again when evidence changes the estimate. It replaces a backlog ordered by volume, seniority, habit or the last person to make the case.

Target Risk Burn-Down (TRB). TRB produces a visible, prioritised risk view: an uncertain event, its impact and probability, its exposure score, the smallest reduction experiment and the evidence from review. Use it before commitment, maintain it weekly for an active initiative, and bring the highest exposures into planning. It replaces a risk register that is completed for governance and then left behind until a problem becomes expensive.

C.L.E.A.R.L.Y Evidence Standard (CES). CES produces an evidence log in which a significant claim is identified as evidence, an assumption or a hypothesis with a test plan; the later learning is recorded as well. Use it in planning, review and decision meetings, especially where a confident assertion could harden into a commitment. It replaces untested opinion presented as fact and makes challenge less

personal because the team can inspect the claim rather than judge the speaker.

Expression of Value Delta (EVD). EVD produces an update that states what changed, which value, quality, risk or operational measure should move, the before and current state, the learning, and the next action. Use it in stand-ups, delivery reviews and retrospectives after a change or test. It replaces a percentage-complete report or an activity update that cannot say what changed for a customer, the organisation or the team's understanding.

D.R.E.A.M Outcome Contract (DOC). A DOC produces a short outcome commitment: the intended change, relevant measure, baseline and intended movement, evidence, timeframe, dependencies and the learning rule if the result does not occur. Use it when major work is committed for a cycle and review it at the agreed date. It replaces an assignment phrased only as a task and a review that asks whether someone was busy rather than whether the expected result appeared.

Quantified D.R.E.A.M Boundaries (QDB). QDB produces an ownership board that assigns one primary owner and named contributors to each material outcome, measure or risk, with decision boundaries and measurable handoff conditions. Use it when work crosses roles, when a handoff has failed, and at regular ownership reviews. It replaces implied responsibility, duplicate ownership and the claim that a handoff was complete merely because something was sent.

M.I.N.D.S.E.T Value Engineering (MVE). MVE produces a testable hypothesis in a fixed form: if the team takes an action, a named measure should move from a baseline to a target within a timeframe,

for a stated reason. It also records the smallest feasible test, the observed result and the decision to adapt, scale, stop or test again. Use it before significant build work begins and wherever value rests on an untested assumption. It replaces premature certainty and a large commitment made before the team knows whether its central belief holds.

Risk-First Learning Loops (RLL). RLL produces a risk-first delivery plan and an updated learning record: the highest current TRB exposures, what can be tested this cycle, the smallest safe test, the evidence gained and who needs to know it. Use it at cycle planning and when new information changes risk. It replaces the habit of polishing low-risk work while the uncertainty most able to invalidate the initiative waits until later.

D.I.R.E.C.T Value Routing (DVR). DVR produces a recorded choice of the next move, grounded in the TVI, its guardrails, VER, current evidence and confidence. Use it when the team has viable options or when an existing route needs to change. It replaces a decision delayed until a hierarchy meeting, or a choice justified only by preference, because the purpose and decision evidence are visible to the people closest to the work.

Insight Risk Triggers (IRT). IRT produces predefined thresholds: the measure or risk to watch, its trigger level and date, the response rule, decision owner and evidence source. Set them before delivery begins and read them on the agreed review cadence. They replace an open-ended instruction to monitor a problem and make clear when the team must pause, pivot, escalate or choose a new test.

D.I.R.E.C.T Decision Audits (DDA). A DDA produces a reviewable record of whether a material decision had an explicit purpose, relevant evidence, visible assumptions and risks, fair comparison of options, clear authority, and a documented review point. Use it to log material decisions and audit a sample after a defined period. It replaces reconstructing a decision from meeting memories and judging a decision-maker only by an outcome that may have remained uncertain.

7 C's Value Engineering Standard (CVE). CVE produces a quality agreement: value target, quality scale, good-enough threshold, guardrail, cost of the next increment, and a stop-or-continue rule. Use it before development and at review when more polish competes with another priority. It replaces subjective arguments about what "done" means and work that continues beyond the agreed value requirement without an explicit reason.

7 C's Outcome Metrics (COM). COM produces a quality scorecard with customer and business measures, supported by technical guardrails, data sources, owners, cadence and decision use. Use it for a released feature or service at the stated review points, including 7, 30 and 60 days where those fit the work. It replaces a technical dashboard used as the only account of quality, such as test coverage, defects or uptime without evidence of customer or business effect.

References

Entries marked *verified* were checked against the publication itself. Where a source is recorded without full publication details, that is stated rather than filled from another source.

1. Woolley, A. W., Chabris, C. F., Pentland, A., Hashmi, N., & Malone, T. W. (2010). "Evidence for a Collective Intelligence Factor in the Performance of Human Groups." *Science*, 330(6004), 686–688. DOI 10.1126/science.1193147. *Verified*. Two studies with 699 people working in groups of two to five. The general collective-intelligence factor they identify is not strongly correlated with the average or maximum individual intelligence of group members, but is correlated with members' average social sensitivity, the equality of conversational turn-taking, and the proportion of women in the group.
2. Google (n.d.). "Understand team effectiveness." *re:Work* guide, Project Aristotle. rework.withgoogle.com/en/guides/understanding-team-effectiveness. *Verified*. Five dynamics distinguish effective teams: psychological safety, dependability, structure and clarity, meaning, and impact. Psychological safety was the most important of the five. The research ran at Google from 2011 for about four years.
3. Katzenbach, J. R., & Smith, D. K. (1993). "The Discipline of Teams." *Harvard Business Review*, 71(2), 111–120. *Verified*. A team is a small number of people with complementary skills, committed to a common purpose, a set of performance goals and an approach for which they hold themselves mutually accountable. The distinction between a team and a working group turns

- on performance. The authors' research covered hundreds of people in more than 50 teams across 30 companies.
4. Szatmari, B., Deichmann, D., & van den Ende, J. (2025). "The increasing value of status in low-performing organizations: Evidence from the video game industry." *Strategic Organization*, 23(4), 604–629. Published online 2024. DOI 10.1177/14761270241271021. *Verified*. The relationship between a team's status and its performance is much stronger in organisations with low previous performance; where recent performance has been high, status has far less influence, because there is no expected resource scarcity pushing decision makers towards high-status teams.
 5. Tovey, M. (ed.) (2008). *Collective Intelligence: Creating a Prosperous World at Peace*. Oakton, Virginia: Earth Intelligence Network. Foreword by Yochai Benkler. *Verified*. An edited collection on collective intelligence; a general source for the field rather than the source of any figure in this book.
 6. Inner Circle. *UK wasting £37b a year on failed Agile IT projects*. innercircle.com.au/uk-wasting-37b-a-year-on-failed-agile-it-projects/. The year is not recorded in the evidence register, and the publisher's page returned no content when checked, so the figure rests on the secondary source that cites it.
 7. *Scrum Inc.* Source for the statement that 47% of Agile transformations fail. The title, year and publication venue are not recorded in the evidence register.

8. *Cynefin*. Video at youtube.com/watch?v=N7oz366X0-8. The title, year and publisher details are not recorded in the evidence register.
9. *Snowden, D.* Paper on Cynefin described in the evidence register as a Harvard publication. The title, year and precise venue are not recorded.
10. *Moltke the Elder*. The observation that no plan survives first contact with the enemy, quoted in the evidence register. The source text, year and publisher are not recorded.
11. *Tyson, M.* "Everyone has a plan until they get punched in the face", quoted in the evidence register. The source, year and venue are not recorded.

About the author

Andrew Phillips has 28 years of experience delivering technology and change programmes. His programme work has included Nationwide Building Society, Vodafone, the Home Office, DEFRA, the Ministry of Defence's Defence Digital, Boeing Defence UK, LexisNexis Risk Solutions, Micro Focus, Atos and UKCloud.

At Defence Digital, he has trained 356 executives in Agile. In a cloud transformation at Micro Focus, the documented results include £45 million in future savings and £4 million in immediate savings, migration of 42 product groups to AWS landing zones, and a 50% reduction in virtual-machine footprint across 14 data centres. His work at Vodafone includes a 40% reduction in development cycle time.

His awards include Computing Cloud Project of the Year 2020 and the Nvidia Best Newcomer Cloud Project 2018. He has also won nine motorsport championships.

Working with TEAM IQ Creator

This book cannot see your teams. It cannot listen to the meeting where a risk is softened, examine a handoff that arrives without the information needed, compare a stated target with the work actually being funded, or tell you which of the six pillars is holding a team back. The instruments give you a way to look, but the diagnosis still depends on real evidence from the people, artefacts, measures and decisions in front of you. A score without that evidence may be a useful prompt; it is not a finding.

Start with the free material. For many teams, it is enough. The TEAM IQ Scorecard on the website takes eight minutes: 24 statements across the six pillars, scored out of 100, with the weakest pillar and two exercises returned. The free monthly workshop takes one pillar at a time in a 60-minute live working session. It includes a practical exercise, questions in the room, printable attendee notes and a replay available for 14 days. The chapters and instruments in this book are also intended for direct use. A team that chooses one real constraint, uses the relevant artefact and reviews the evidence can make a serious start without buying anything.

There are three paid routes when an outside view or more sustained work is useful. The first is the *TEAM IQ Diagnostic Session*, priced at £1,200 + VAT. It is a 90-minute session on one area of concern with the team that owns it. It produces a written Team IQ Score, a prioritised action plan and two exercises to start immediately. The £1,200 fee is credited in full against a corporate diagnostic or corporate programme if you continue within six months. The session is a complete piece of work; the plan is written so that a team can use it without further work from TEAM IQ Creator.

The *Corporate TEAM IQ Diagnostic* starts at £18,000 + VAT for up to five teams. For six to 12 teams, the stated price is £32,000 + VAT; work above 12 teams is scoped before a price is set. It measures all six pillars across every delivery team, benchmarks them on one scale, and produces a prioritised 90-day plan and a board presentation. This route provides the full picture across the teams. It is not presented as the training-and-exercise phase: the price page places training and exercises in the corporate programme, which is a useful distinction to make before anyone commits budget.

The *Corporate TEAM IQ Programme* starts at £45,000 + VAT. It runs for 12 to 16 weeks across all six pillars and includes training and exercises for every team, dashboards, coaching for team leads and quarterly re-measurement. The diagnostic fee is credited in full when an organisation proceeds. These are stated starting prices, before VAT. Travel and accommodation for on-site work are charged at cost, and third-party tools or licences are not included unless they are named first.

Work starts with a conversation about what is happening and what evidence already exists. From there, the aim is the smallest useful step: perhaps the free scorecard and an instrument from this book, perhaps one diagnostic session focused on a live constraint, or, where the evidence points to a wider pattern, a corporate diagnostic. There is no need to begin with a programme before the scale of the issue is known.

The standard for this work is the same one used throughout the book. Published findings, the method's own claims and the evidence from a particular team should not be presented as though they are the

same thing. A measure should have a named source, owner, baseline, review point and limit. Assumptions should be visible. Where the evidence is incomplete, the honest response is to say so and define the next test, not to claim certainty or promise an outcome.